

Module V

Convolutional Neural Networks, LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet, Visualizing Convolutional Neural Networks, Guided Backpropagation, Deep Dream, Deep Art,

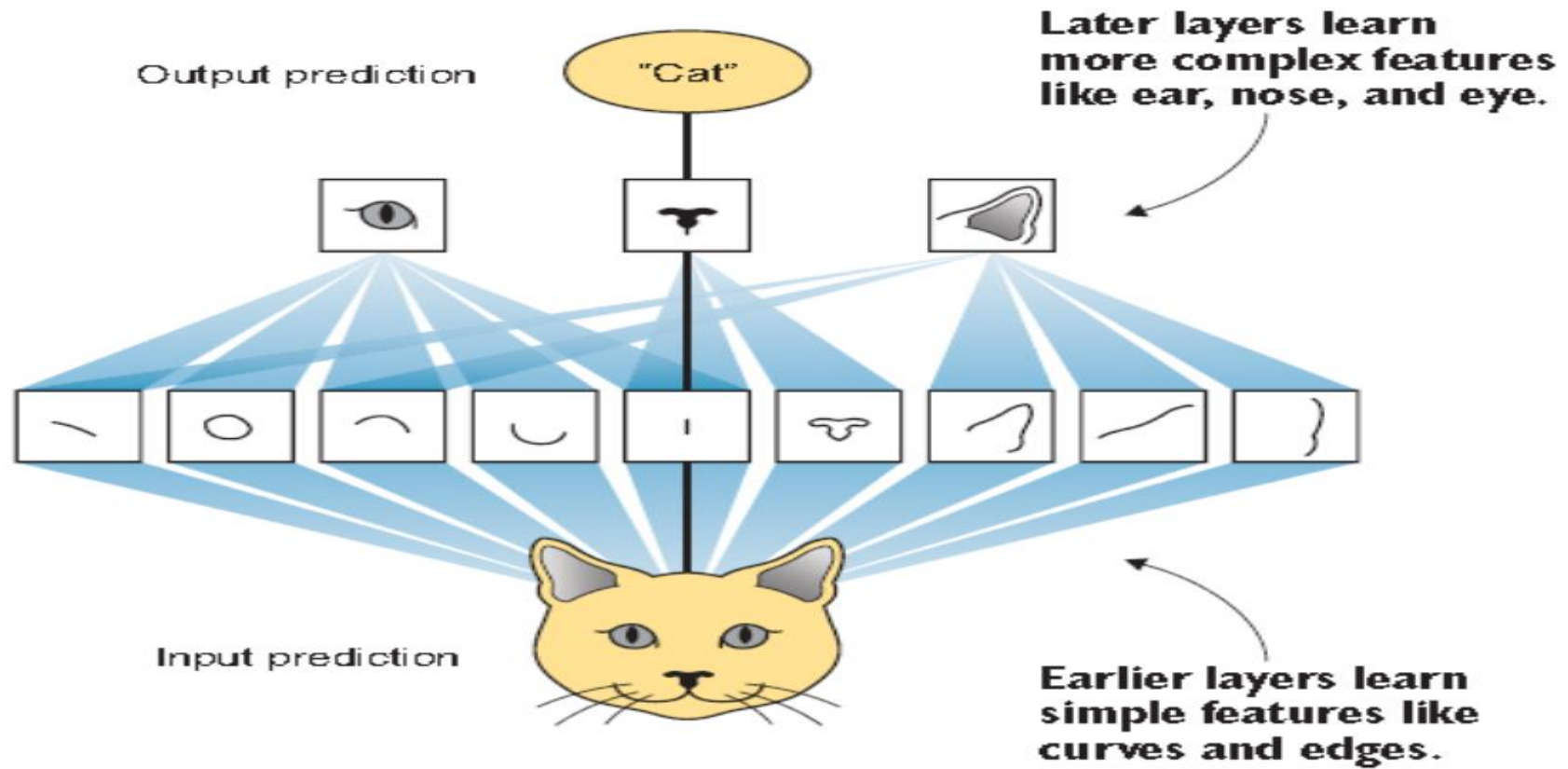
Fooling Convolutional Neural Networks, Recurrent Neural Networks, Backpropagation through

time (BPTT), Vanishing and Exploding Gradients, Truncated BPTT, GRU, **LSTMs**. Encoder Decoder Models, Attention Mechanism, Attention over images, Introduction to GANs.

CNN architecture

- first layer of convolutions learns some basic features (edges and lines),
- the next layer learns features that are a little more complex (circles, squares, and so on),
- the following layer finds even more complex features (like parts of the face, a car wheel, dog whiskers, and the like), and so on.

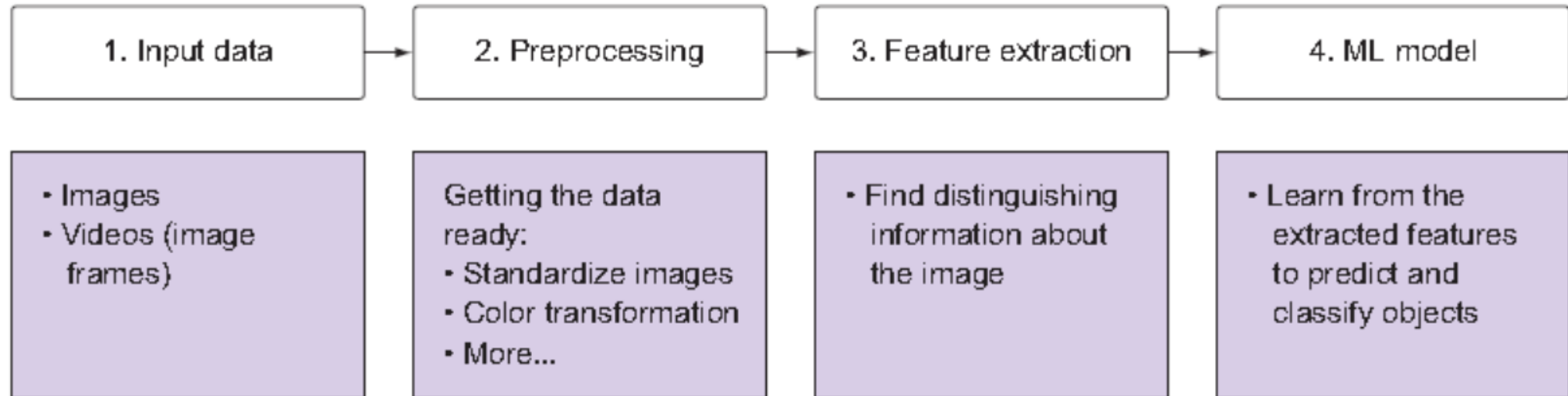
CNN architecture



CNN architecture

- CNN architecture follows the same pattern as neural networks:
- we stack neurons in hidden layers on top of each other;
- weights are randomly initiated and learned during network training; and we apply activation functions,
- calculate the error ($y - \hat{y}$),
- and backpropagate the error to update the weights.
- This process is the same.
- The difference is that we use convolutional layers instead of regular fully connected layers for the feature learning part.

CNN architecture



The image classification pipeline consists of four components

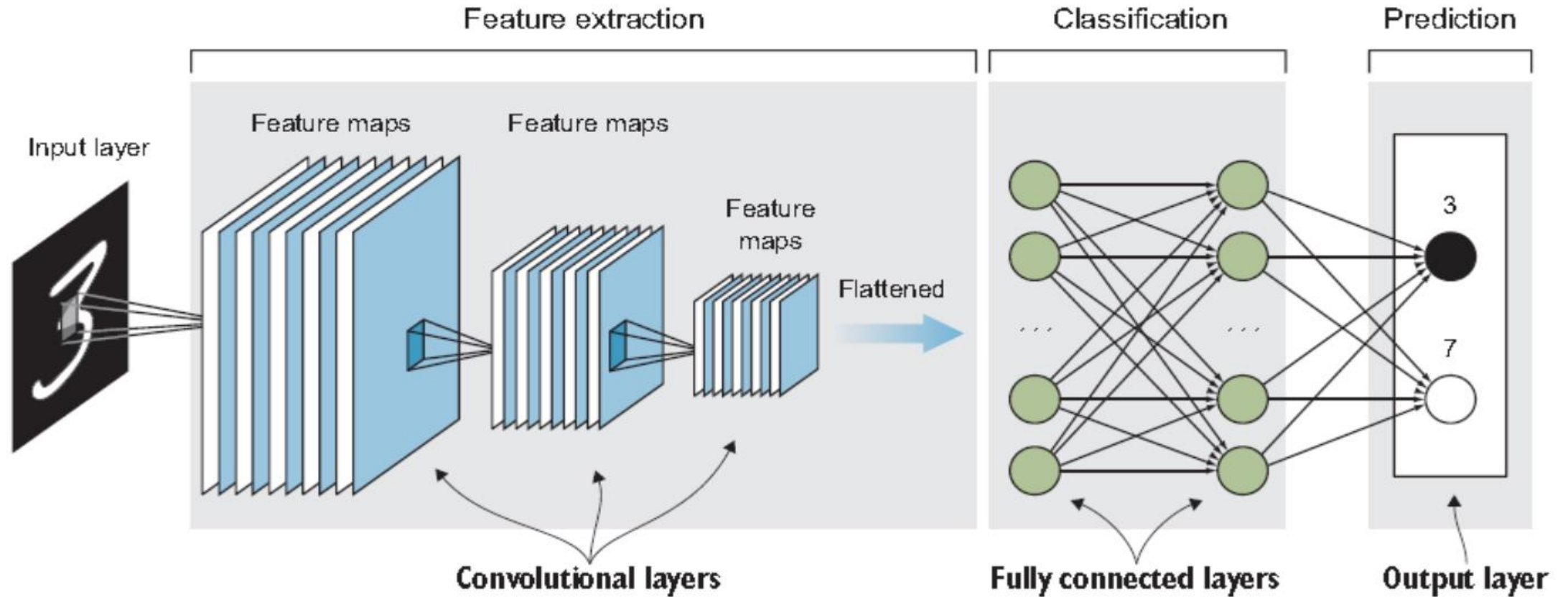
CNN architecture

- The fully connected layers aren't doing a good job of **feature extraction** (step 3), so let's replace that with locally connected layers (convolutional layers).
- On the other hand, fully connected layers do a great job **of classifying the extracted features** (step 4),
- so let's keep them for the classification part.

CNN architecture

- The high-level architecture of CNNs
- Input layer
- Convolutional layers for feature extraction
- Fully connected layers for classification
- Output prediction

CNN architecture



The CNN architecture consists of the following: input layer, convolutional layers, fully connected layers, and output prediction.

CNN architecture

- Feed the raw image to the convolutional layers
- The image passes through the CNN layers to detect patterns and extract features called *feature maps*.
- The output of this step is then flattened to a vector of the learned features of the image.
- image dimensions shrink after each layer,
- and the number of feature maps (the layer depth) increases
until we have a long array of small features in the last layer of the feature extraction part.

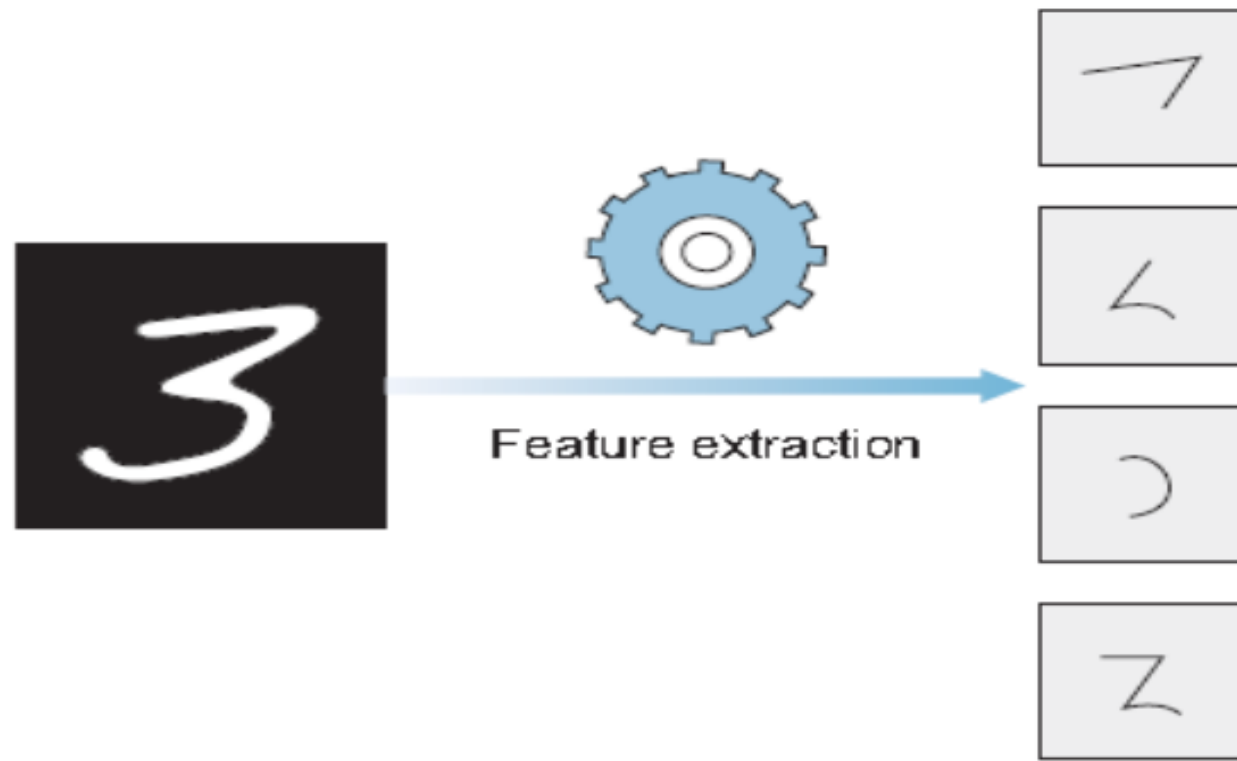
CNN architecture

- The flattened feature vector is fed to the fully connected layers to classify the extracted features of the image.
- The neural network fires the node that represents the correct prediction of the image.
- we are classifying two classes (3 and 7).
- Thus the output layer will have two nodes: one to represent the digit 3, and one for the digit 7.

CNN architecture

- idea of neural networks is that neurons learn features from the input.
- In CNNs, a ***feature map*** is the output of one filter applied to the previous layer.
- It is called a feature map because it is a mapping of where a certain kind of feature is found in the image.
- CNNs look for features such as straight lines, edges, or even objects. Whenever they spot these features, they report them to the feature map.

CNN architecture



feature-extraction step as breaking large images into smaller pieces of features and stacking them into a vector.

For example, an image of the digit 3 is one image (depth = 1) and is broken into smaller images that contain specific features of the digit 3 .

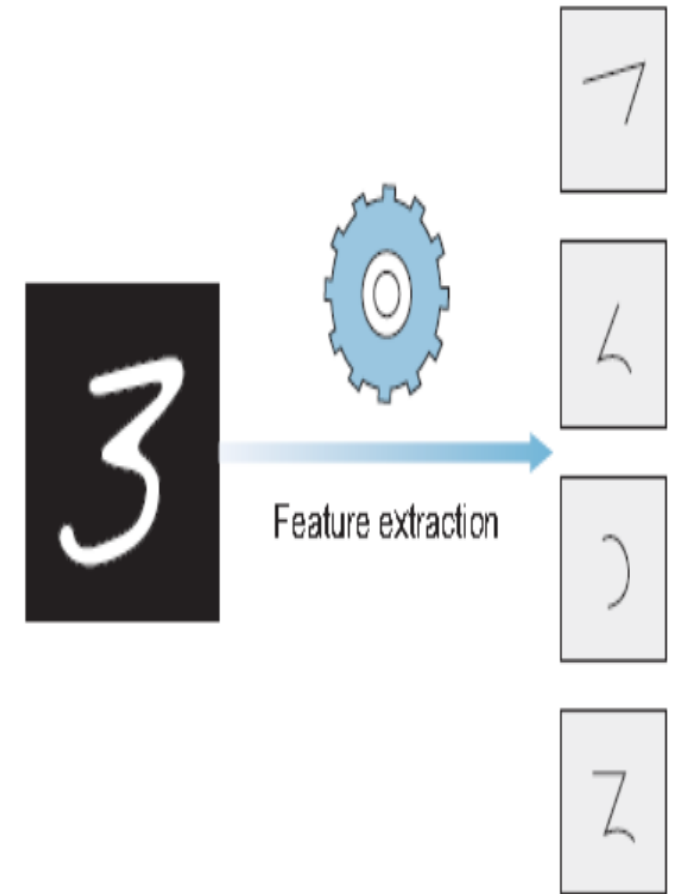
If it is broken into four features, then the depth equals 4.

it shrinks in dimensions, and the layer gets deeper because it contains more images of smaller features

An image is broken into smaller images that contain distinctive features.

A closer look at classification

- After feature extraction is complete, we add fully connected layers (a regular MLP) to look at the features vector.
- “The first feature (top) has what looks like an edge:
- this could be 3, or 7, or maybe an ugly 2. I’m not sure;
- let’s look at the second feature. Hmm, this is definitely not a 7 because it has a curve,”
- and so on until the MLP is confident that the image is the digit 3.

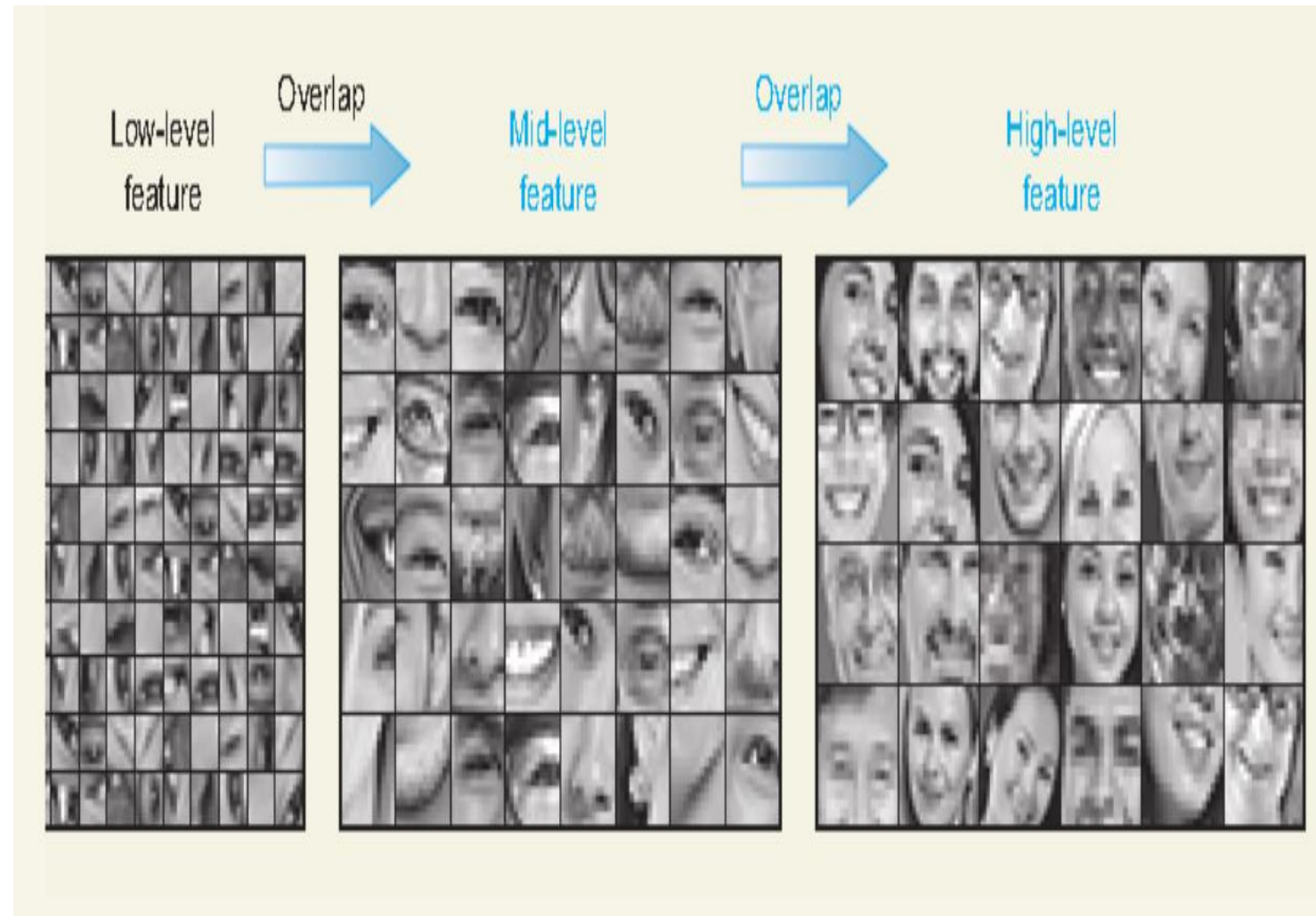


How CNNs learn patterns

- It is important to note that a CNN doesn't go from the image input to the features vector directly in one layer.
- This usually happens **in tens or hundreds of layers**.
- So the first layer usually learns very basic features like lines and edges, and the second assembles those lines into recognizable shapes, corners, and circles.
- Then, in the deeper layers, the network learns more complex shapes
- such as human hands, eyes, ears, and so on.

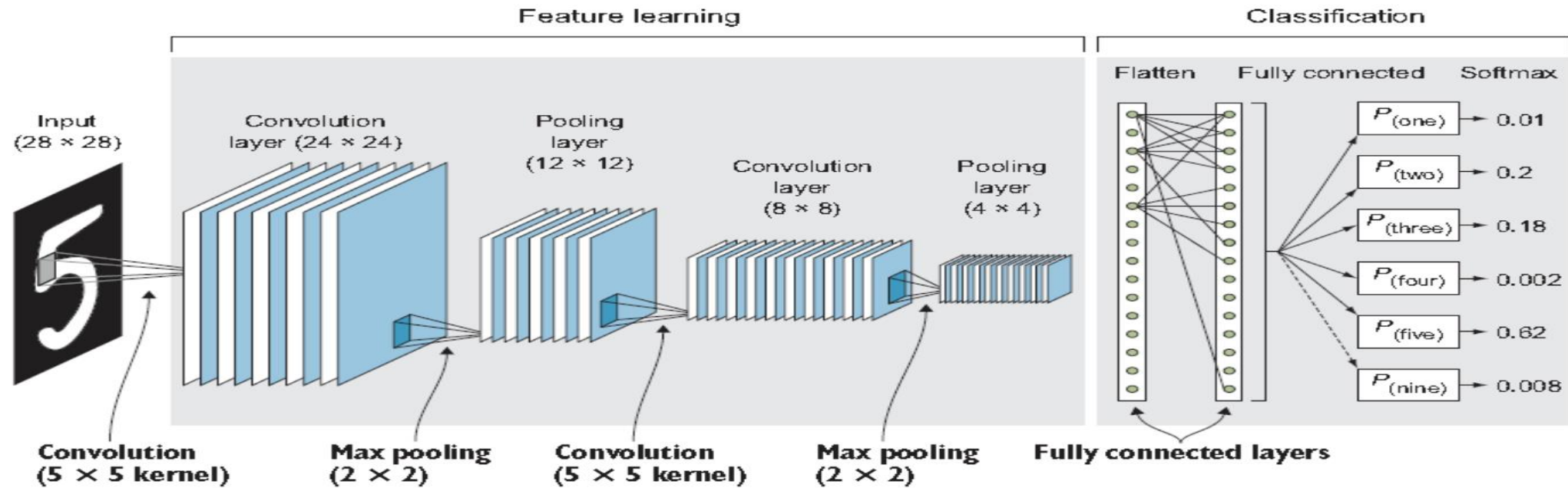
How CNNs learn patterns

- A simplified version of how CNNs learn faces
- Input image
- + Layer 1 $\Rightarrow$ patterns
- + Layer 2 $\Rightarrow$ patterns within patterns
- + Layer 3 $\Rightarrow$ patterns within patterns within patterns
- ... and so on



Basic components of a CNN

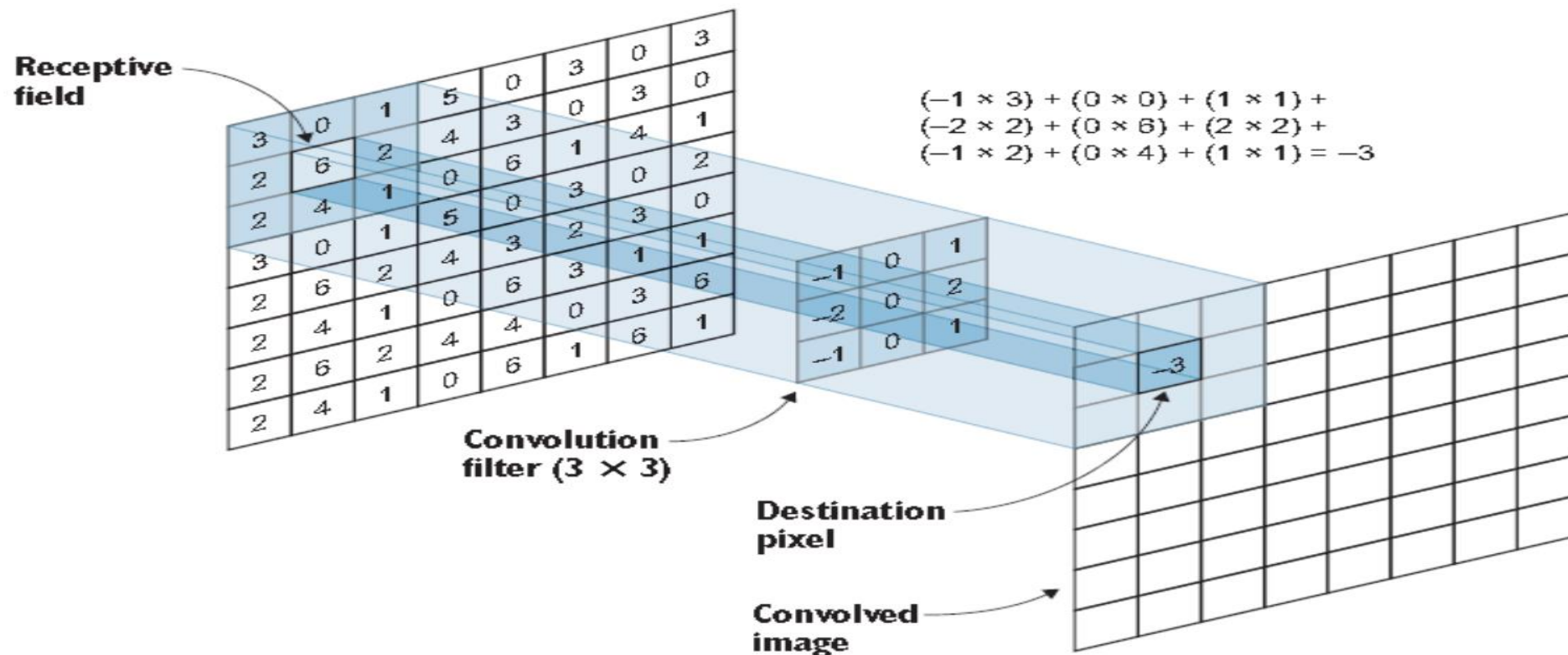
- 1 Convolutional layer (CONV)
- 2 Pooling layer (POOL)
- 3 Fully connected layer (FC)



CNN architecture: INPUT $\Rightarrow$ CONV $\Rightarrow$ RELU $\Rightarrow$ POOL $\Rightarrow$ CONV $\Rightarrow$ RELU $\Rightarrow$ POOL $\Rightarrow$ FC $\Rightarrow$ SOFTMAX

Convolutional layers

- Convolutional layers act like a feature finder window that slides over the image pixel by pixel to extract meaningful features that identify the objects in the image



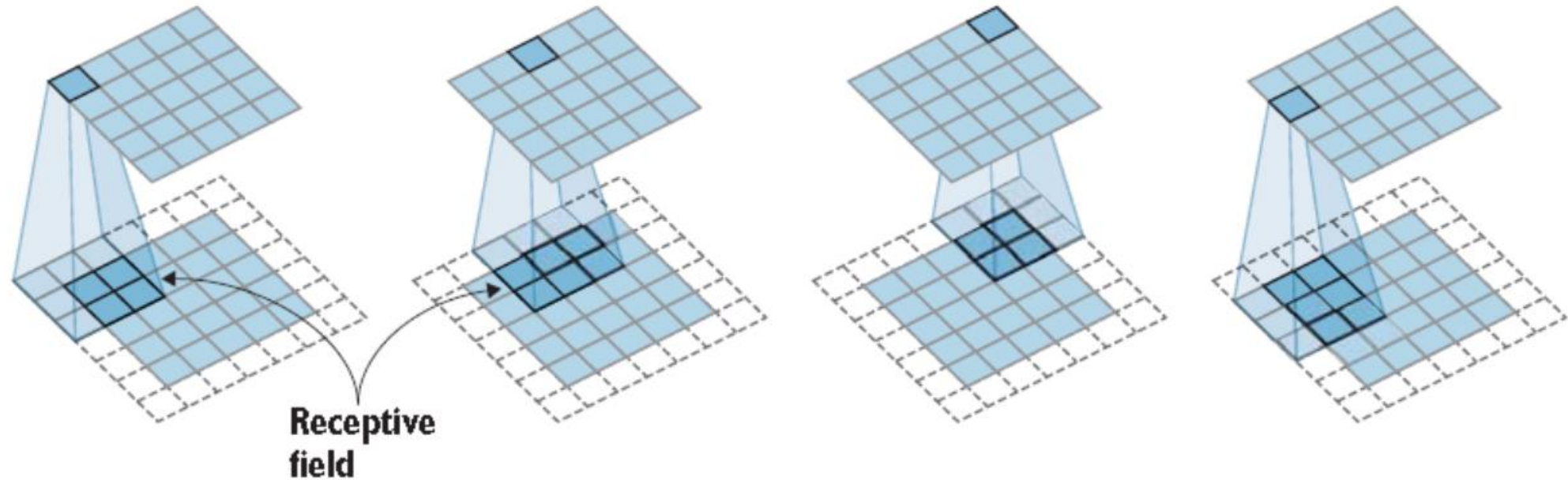
Convolutional layers

- WHAT IS CONVOLUTION?
- In mathematics, convolution is the operation of two functions to produce a third modified function.
- In the context of CNNs, the first function is the input image, and the second function is the convolutional filter.
- We will perform some mathematical operations to produce a modified image with new pixel values.
- By sliding the convolutional filter over the input image, the network breaks the image into little chunks and processes those chunks individually to assemble the modified image, a feature map

Convolutional layers

- some facts about convolution filters:
- The small 3×3 matrix in the middle *is* the convolution filter, also **called a *kernel***.
- The kernel slides over the original image pixel by pixel and does some math calculations to get the values of the new “convolved” image on the next layer.
- The area of the image that the filter convolves is called the *receptive field*

Convolutional layers



The kernel slides over the original image pixel by pixel and calculates the convolved image on the next layer. The convolved area is called the ***receptive field***.

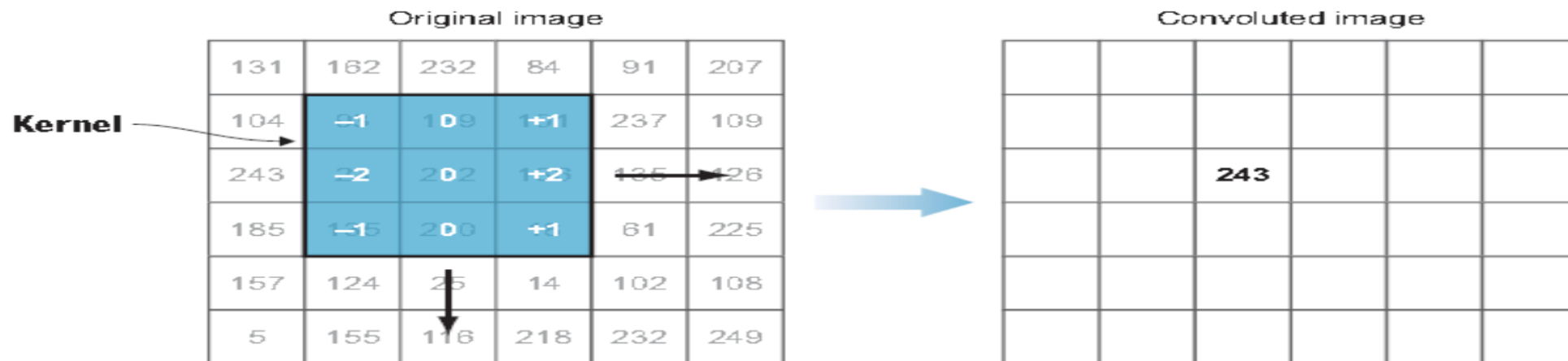
Convolutional layers

- What are the kernel values? In CNNs, the convolution matrix *is* the weights.
- This means they are also *randomly initialized* and the values are *learned* by the network.
- CONVOLUTIONAL OPERATIONS
- weighted sum = $x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + \dots + x_n \cdot w_n + b$.
- So we multiply each pixel in the receptive field by the corresponding pixel in the convolution filter and sum them all together to get the value of the center pixel in the new image

$$(93 \times -1) + (139 \times 0) + (101 \times 1) + (26 \times -2) + (252 \times 0) + (196 \times 2) + (135 \times -1) + (240 \times 0) + (48 \times 1) = 243$$

Convolutional layers

- The filter (or kernel) slides over the whole image. Each time, we multiply every corresponding pixel element-wise and then add them all together to create a new image with new pixel values.
- This convolved image is called a *feature map* or *activation map*.



Multiplying each pixel in the receptive field by the corresponding pixel in the convolution filter and summing them gives the value of the center pixel in the new image

You would agree that with the convolution process, we will quickly lose the pixels along the periphery. As we have seen in Figure 2-3, a 5x5 image became a 3x3 image, and this loss will increase with a greater number of layers. To tackle this problem, we have the concept of padding. In padding, we add some pixels to an image which is getting processed. For example, we can pad the image with zeros as shown in Figure 2-6. The usage of padding results in a better analysis and better results from the convolutional Neural Networks.

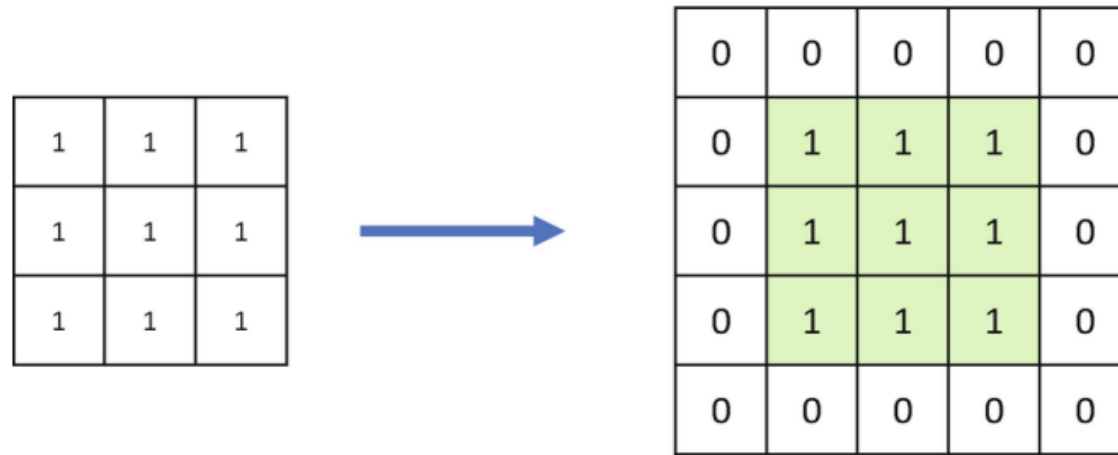


Figure 2-6 Zero padding has been added to the input image. Convolution as the process reduces the number of pixels, padding allows us to tackle it

Now we have understood the prime components of a CNN. Let us combine the concepts and create a small process. If we have an image of size $(n \times n)$ and we apply a filter of size f with a stride of s and padding p , then the output of the process will be $((n+2p-f)/s+1, (n+2p-f)/s+1)$ (Equation 2-1)

Applying filters to learn features

- We are doing all this so the network extracts features from the image.
- filters are used to filter out unwanted information or amplify features in an image.
- These filters are matrices of numbers that convolve with the input image to modify it.
- Look at this edge-detection filter
- When this kernel (K) is convolved with the input image $F(x,y)$,
- it creates a new convolved image (a feature map)
- that amplifies the edges.



0	-1	0
-1	4	-1
0	-1	0

Applying filters to learn features

Input image



Convolution kernel
with optimized weights

$$\begin{matrix} * & \begin{matrix} \begin{matrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{matrix} & = \end{matrix} \end{matrix}$$

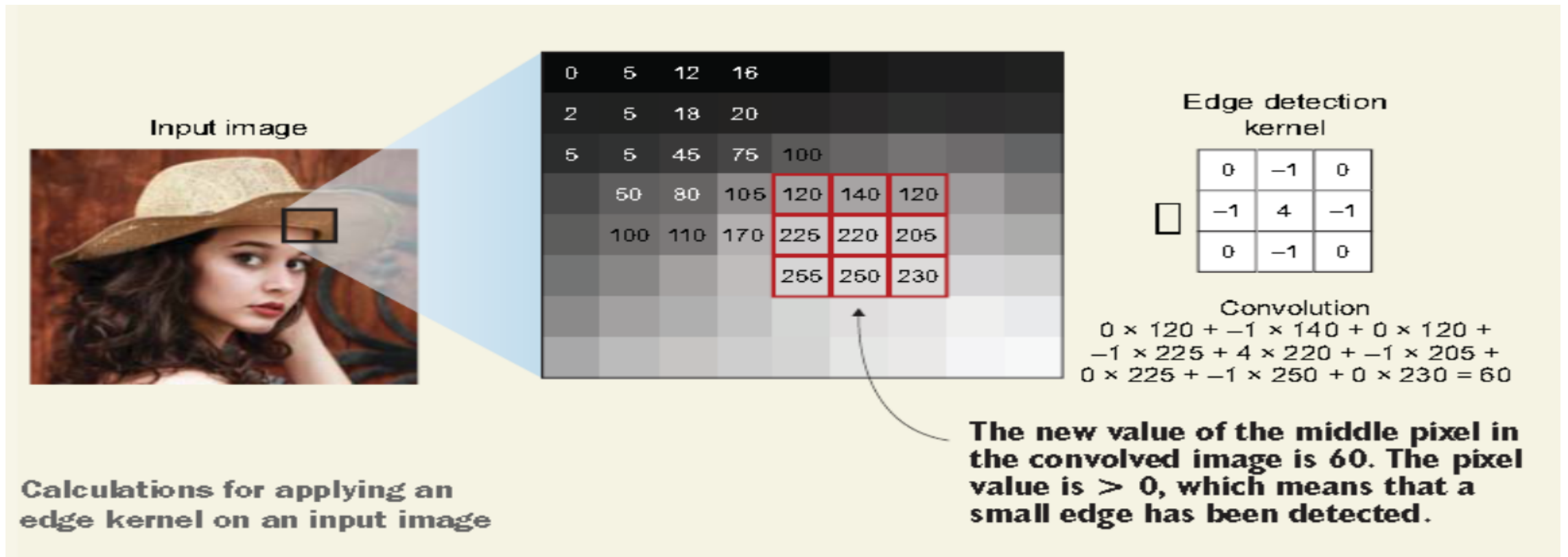
Convolved image
(feature map)



Applying an edge detection kernel on an image

Applying filters to learn features

- To understand how the convolution happens, let's zoom in on a small piece of the image.



Applying filters to learn features

- This image shows the convolution calculations in one area of the image to compute the value of one pixel.
- We compute the values of all the pixels by sliding the kernel over the input image pixel by pixel and applying the same convolution process.
- These kernels are often called *weights* because they determine how important a pixel is in forming a new output image. Similar to what we discussed about MLP and weights,
- these weights represent the importance of the feature on the output.
- In images, the input features are the pixel values.

Applying filters to learn features

- Other filters can be applied to detect different types of features.
- For example, some filters detect horizontal edges, others detect vertical edges,
- still others detect more complex shapes like corners, and so on.
- The point is that these filters, when applied in the convolutional layers, yield the feature-learning behavior.
- first they learn simple features like edges and straight lines, and later layers learn more complex features.

Applying filters to learn features

- Each convolutional layer contains one or more convolutional filters.
- The number of filters in each convolutional layer determines the depth of the next layer, because each filter produces its own feature map (convolved image).
- Let's look at the convolutional layers in Keras to see how they work:

```
from keras.layers import Conv2D
```

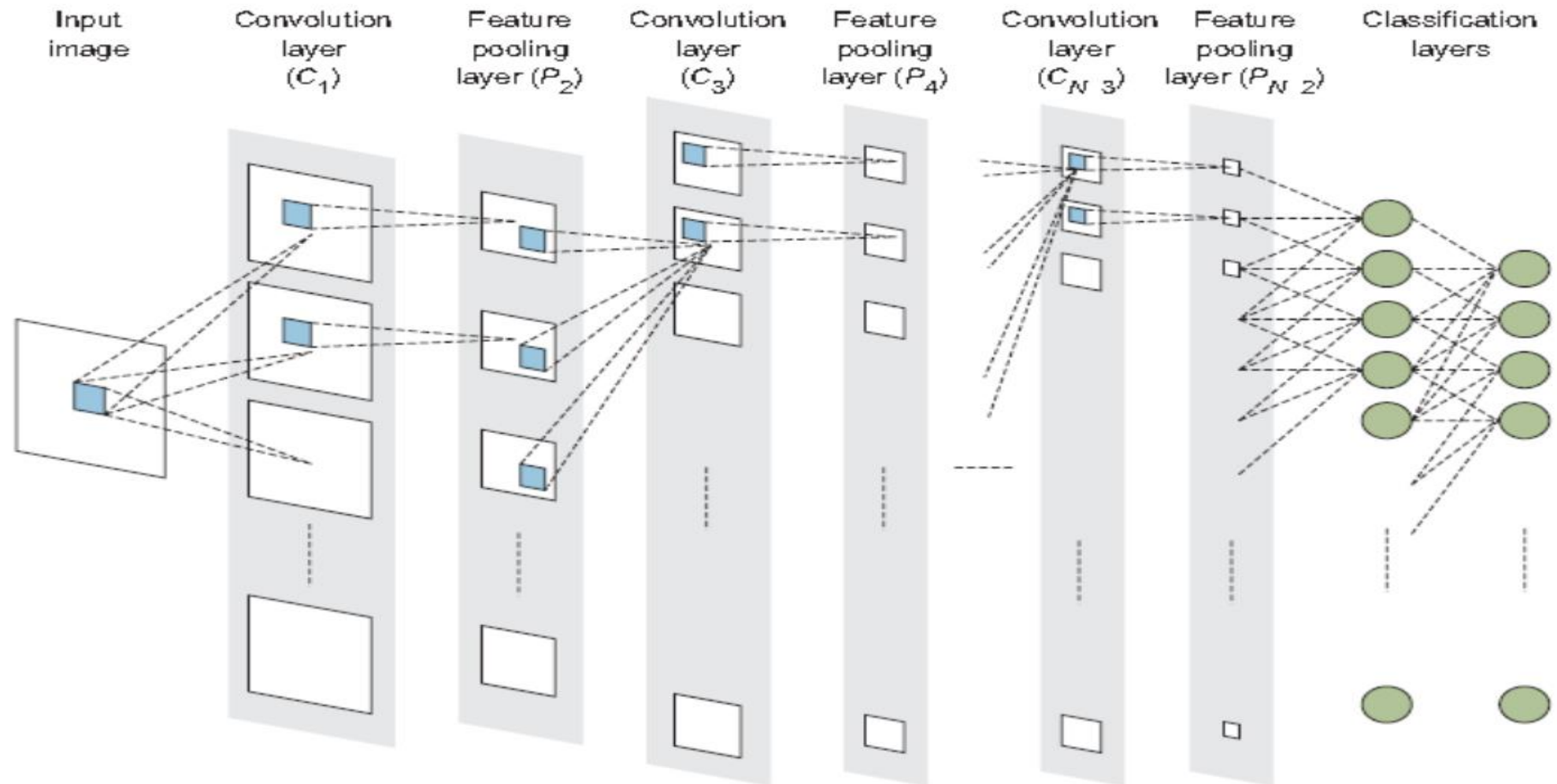
```
model.add(Conv2D(filters=16, kernel_size=2, strides='1', padding='same',  
                  activation='relu'))
```

Applying filters to learn features

- **four hyperparameters that** control the size and depth of the output volume:
- Filters: the number of convolutional filters in each layer. This represents the depth of its output.
- Kernel size: the size of the convolutional filter matrix. Sizes vary: **2×2 , 3×3 , 5×5** .
- Stride.
- Padding.

NUMBER OF FILTERS IN THE CONVOLUTIONAL LAYER

- Each convolutional layer has one or more filters.

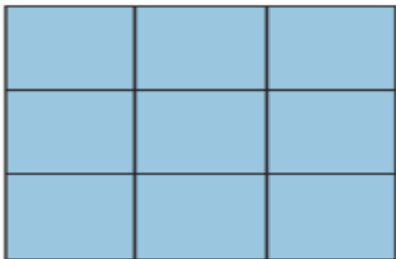


NUMBER OF FILTERS IN THE CONVOLUTIONAL LAYER

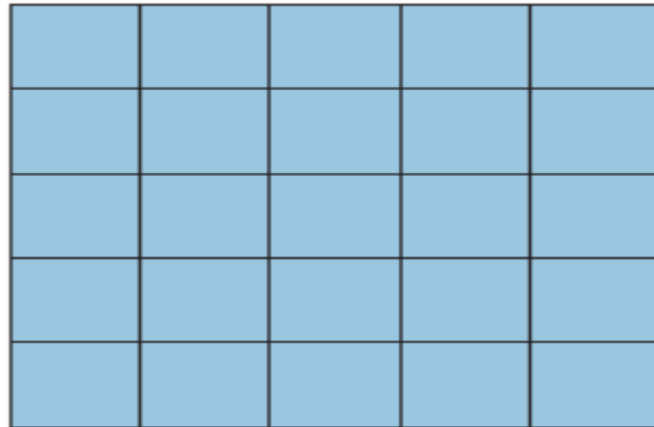
- with CNNs, **the convolutional layers are the hidden layers.**
- And to increase the number of neurons in hidden layers, we increase the number of kernels in convolutional layers.
- Each kernel unit is considered a neuron.
- For example, if we have a 3×3 kernel in the convolutional layer, this means we have 9 hidden units in this layer.
- When we add another 3×3 kernel, we have 18 hidden units.
- Add another one, and we have 27, and so on.
- So, by increasing the number of kernels in a convolutional layer,
- we increase the number of hidden units, which makes our network more complex and able to detect more complex patterns

KERNEL SIZE

- a convolution filter is also known as a *kernel*.
- It is a matrix of weights that slides over the image to extract features.
- The kernel size refers to the dimensions of the convolution filter (width times height)



3 × 3



5 × 5

the bigger the kernel_size, the deeper the network, which means the better it learns. However, this comes with higher computational complexity and might lead to overfitting

STRIDES AND PADDING

- *Strides*—The amount by which the filter slides over the image. For example, to slide the convolution filter one pixel at a time, the strides value is 1.
- If we want to jump **two pixels at a time**, the **strides value is 2**.
- **Strides of 3 or more are uncommon** and rare in practice.
- Jumping pixels produces smaller output volumes spatially.
- **Strides of 1 will make the output image roughly the same height and width of the input image,**
- **while strides of 2 will make the output image roughly half of the input image size**

Padding

- *zero-padding* because we add zeros around the border of an image.
- Padding = 2 adds two layers of zeros around the border.

Padding = 2 Pad

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	123	94	2	4	0	0
0	0	11	3	22	192	0	0
0	0	12	4	23	34	0	0
0	0	194	83	12	94	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Pad

keep all the important details of the image and transfer them to the next layer (when the strides value is 1 and the padding value is same);
Or ignore some of the spatial information of the image to make the processing computationally more affordable

Padding

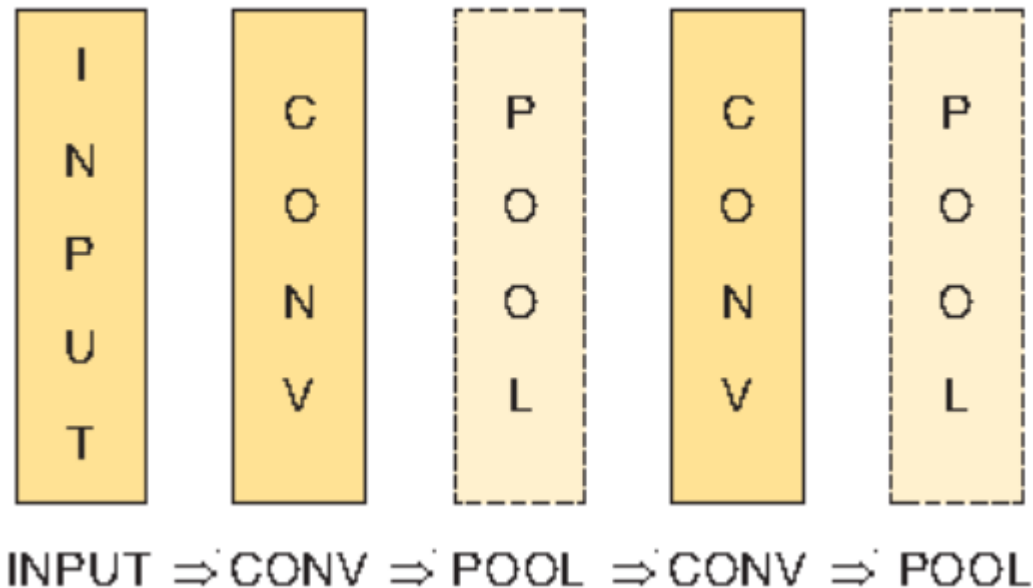
- **adding the pooling layer to reduce the size of the image** to focus on the extracted features.
- For now, know that strides and padding hyperparameters are meant to control the behavior of the convolutional layer and the size of its output:
whether to pass on all image details or ignore some of them.

Pooling layers or subsampling

- **Adding more convolutional layers increases the depth of the output layer**, which leads to increasing the number of parameters that the network needs to optimize (learn).
- You can see that adding several convolutional layers (usually tens or even hundreds) will produce a huge number of parameters (weights). This increase in network dimensionality increases the time and space complexity of the mathematical operations that take place in the learning process.
- ***Subsampling or pooling*** helps reduce the size of the network by reducing the number of parameters passed to the next layer.

Pooling layers or subsampling

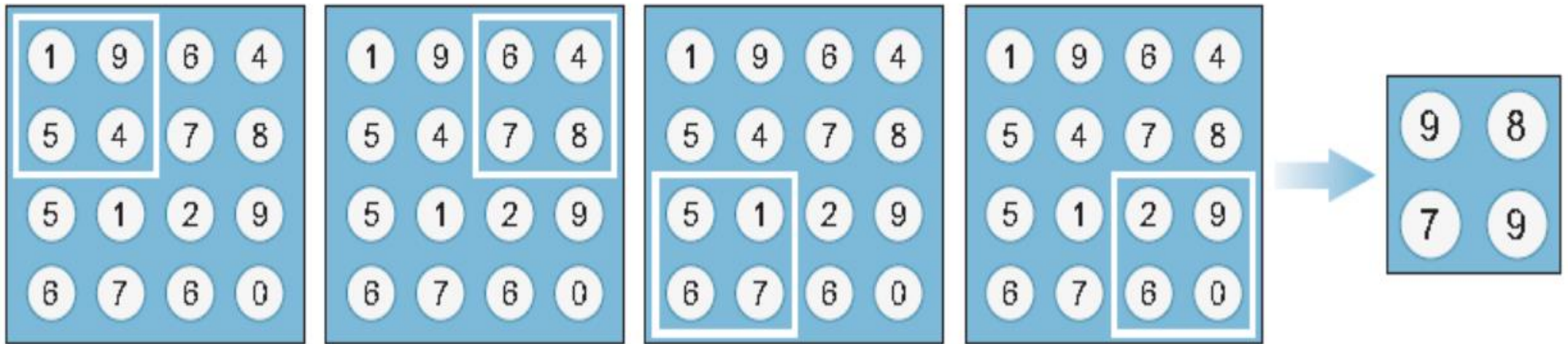
- The pooling operation resizes its input by applying a summary statistical function, such as a **maximum** or **average**, to reduce the overall number of parameters passed on to the next layer



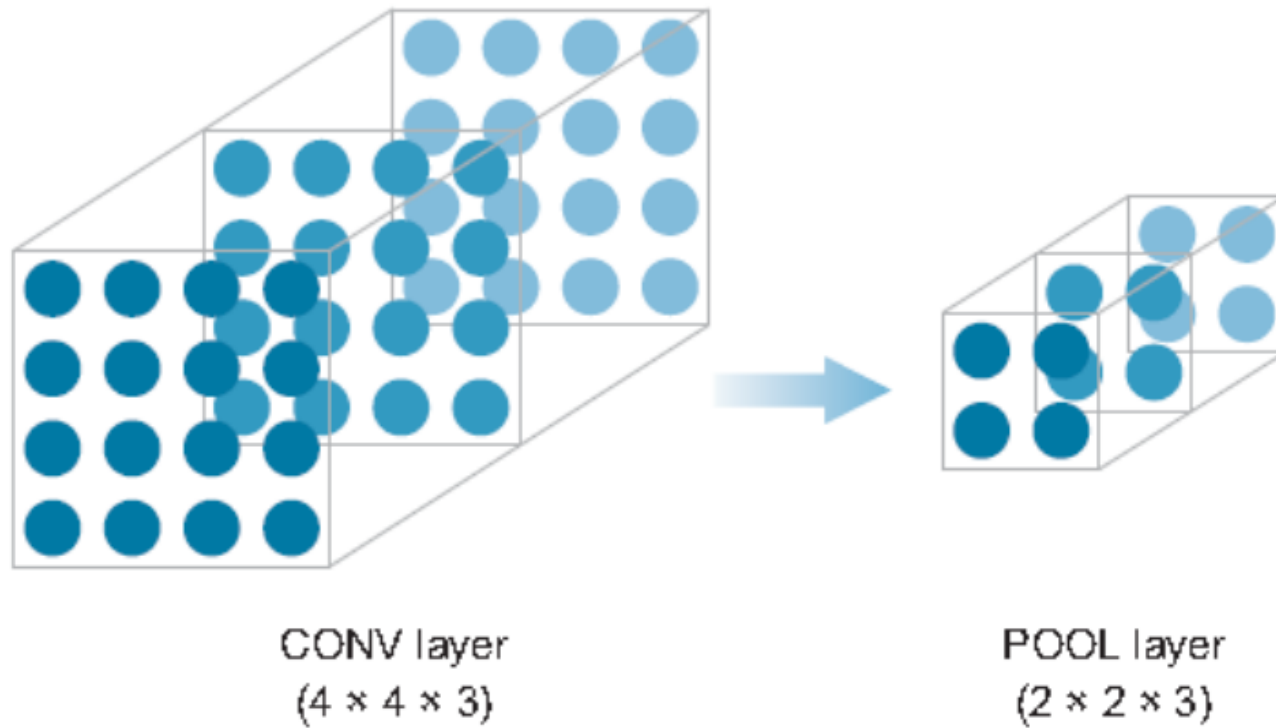
Pooling layers are added after every one or two convolutional layers.

MAX POOLING VS. AVERAGE POOLING

- you see a pooling filter with a **size of 2×2** and **strides of 2** (the filter jumps 2 pixels when sliding over the image).
- This pooling layer reduces feature map size from 4×4 down to 2×2 .



Max Pooling or Avg



If the convolutional layer has three feature maps, the pooling layer's output will have three smaller feature maps.

Max Pooling or Avg

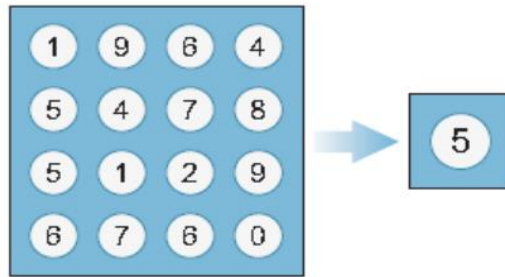
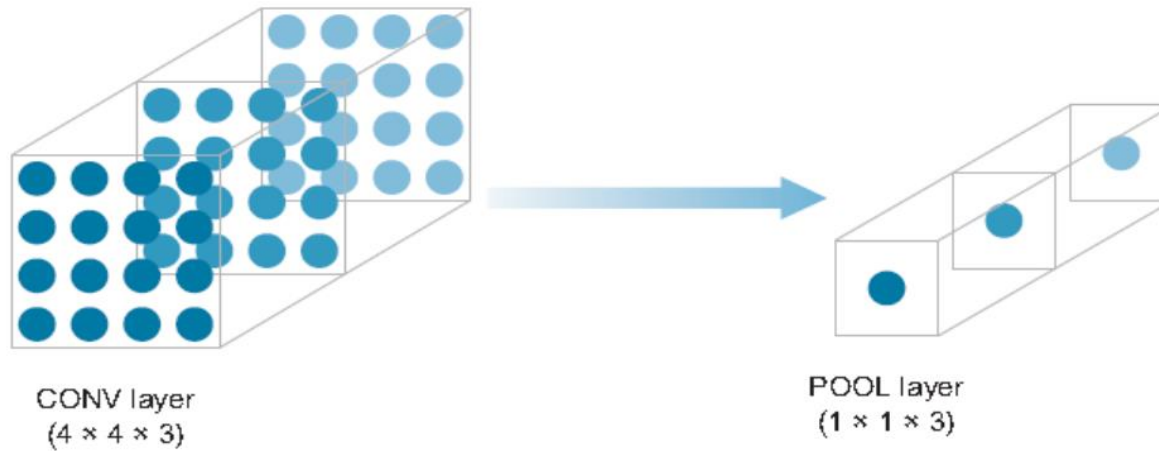


Figure 3.23 Global average pooling calculates the average values of all the pixels in a feature map.

Global average pooling calculates the average values of all the pixels in a feature map..



WHY USE A POOLING LAYER?



Original



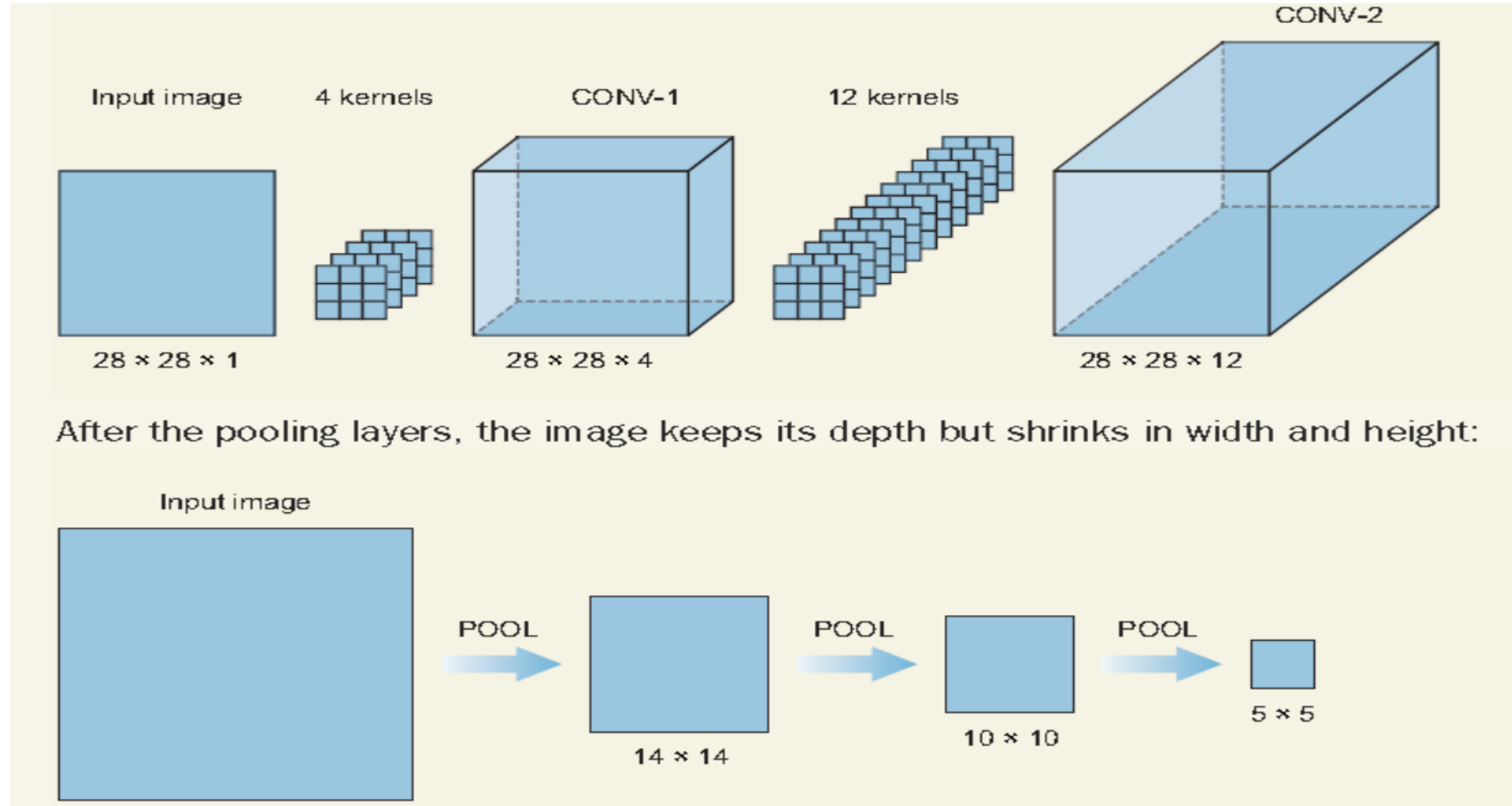
Downsampled

Pooling layers reduce image resolution and keep the image's important features.

Visualize what happens after each layer

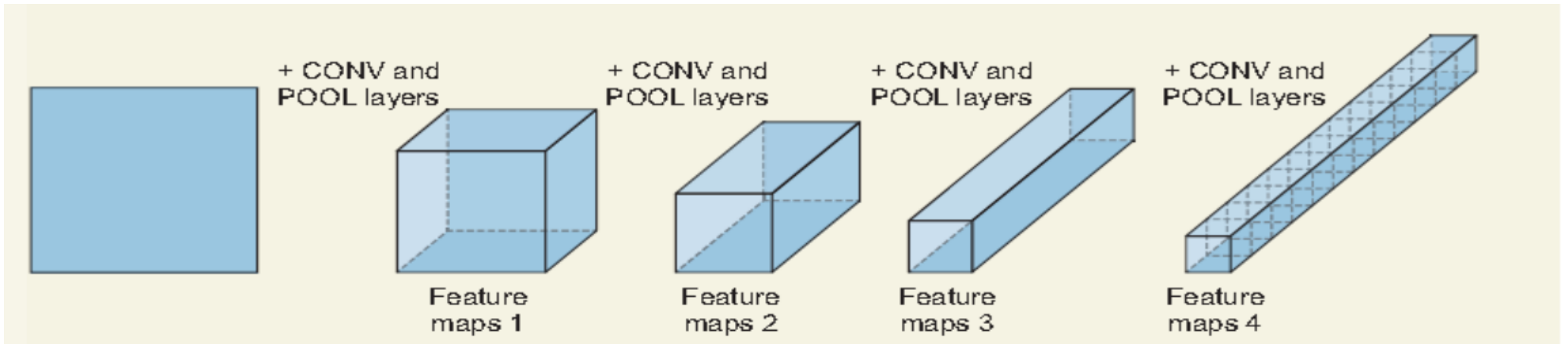
- suppose the input image is 28×28 (like in the MNIST dataset). When
- we add a CONV_1 layer (with filters of 4, strides of 1, and padding of same), the output will be the **same width and height** dimensions but with depth of 4 ($28 \times 28 \times 4$).
- Now we add a CONV_2 layer with the same hyperparameters but more filters (12), and we get deeper output: **$28 \times 28 \times 12$** .

Visualize what happens after each layer



Visualize what happens after each layer

- Putting the convolutional and pooling together



- The output of the convolutional and pooling layers produces a feature tube ($5 \times 5 \times 40$) that is *almost* ready to be classified.
- the flattened layer will have dimensions of $(1, m)$ where $m = 5 \times 5 \times 40 = 1,000$ neurons

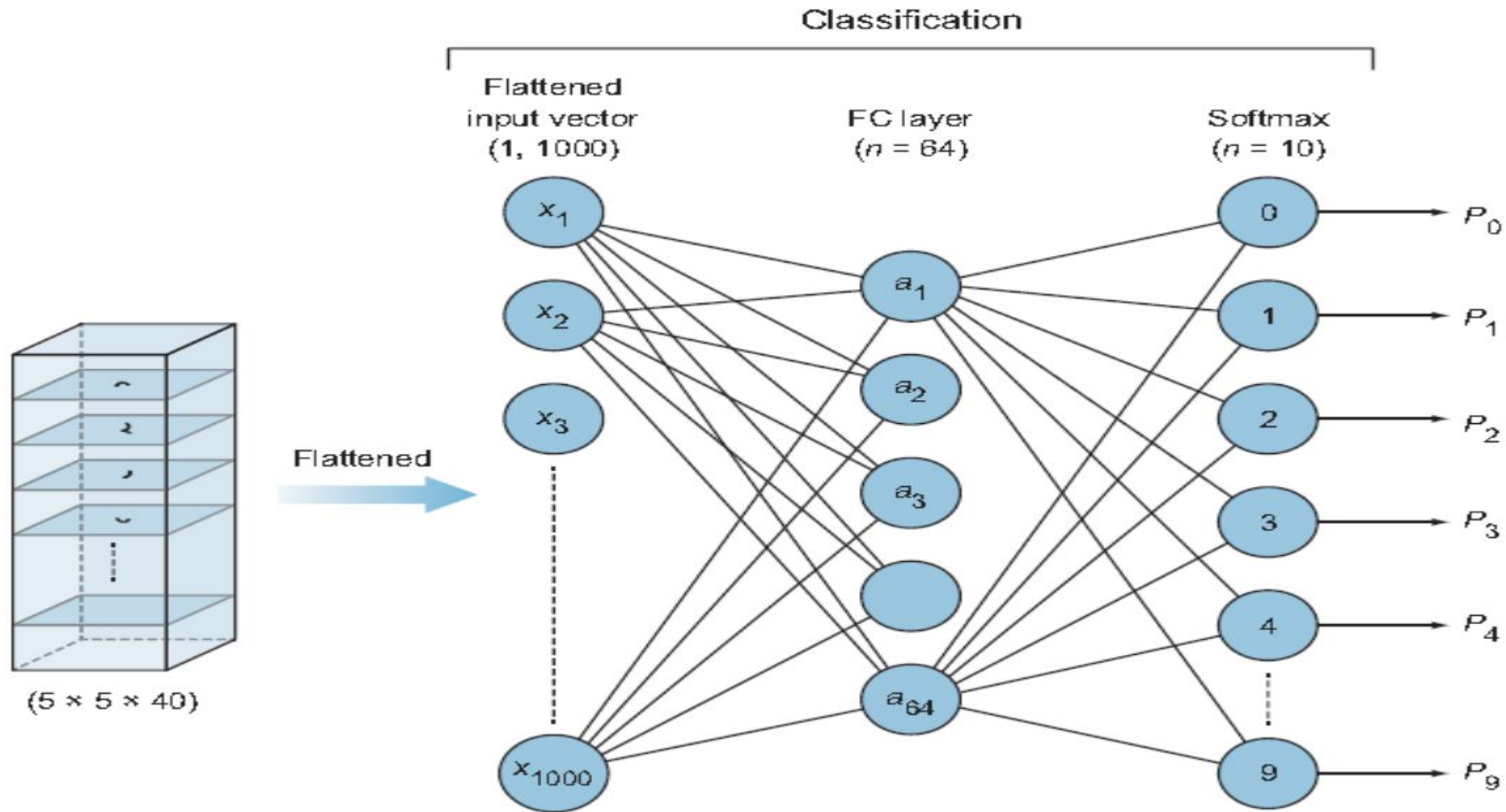
Fully connected layers

- After passing the image through the feature-learning process using convolutional and pooling layers, we have extracted all the features and put them in a long tube.
- Now it is time to use these extracted features to classify images.
- WHY USE FULLY CONNECTED LAYERS?
- MLPs work great in classification problems
- MLPs lose a lot of valuable information when extracting features from an image—we have to flatten the image before feeding it to the network—
- whereas convolutional layers can process raw images.
- Now we have the features extracted, and after we flatten them, we can use regular MLPs to classify them.

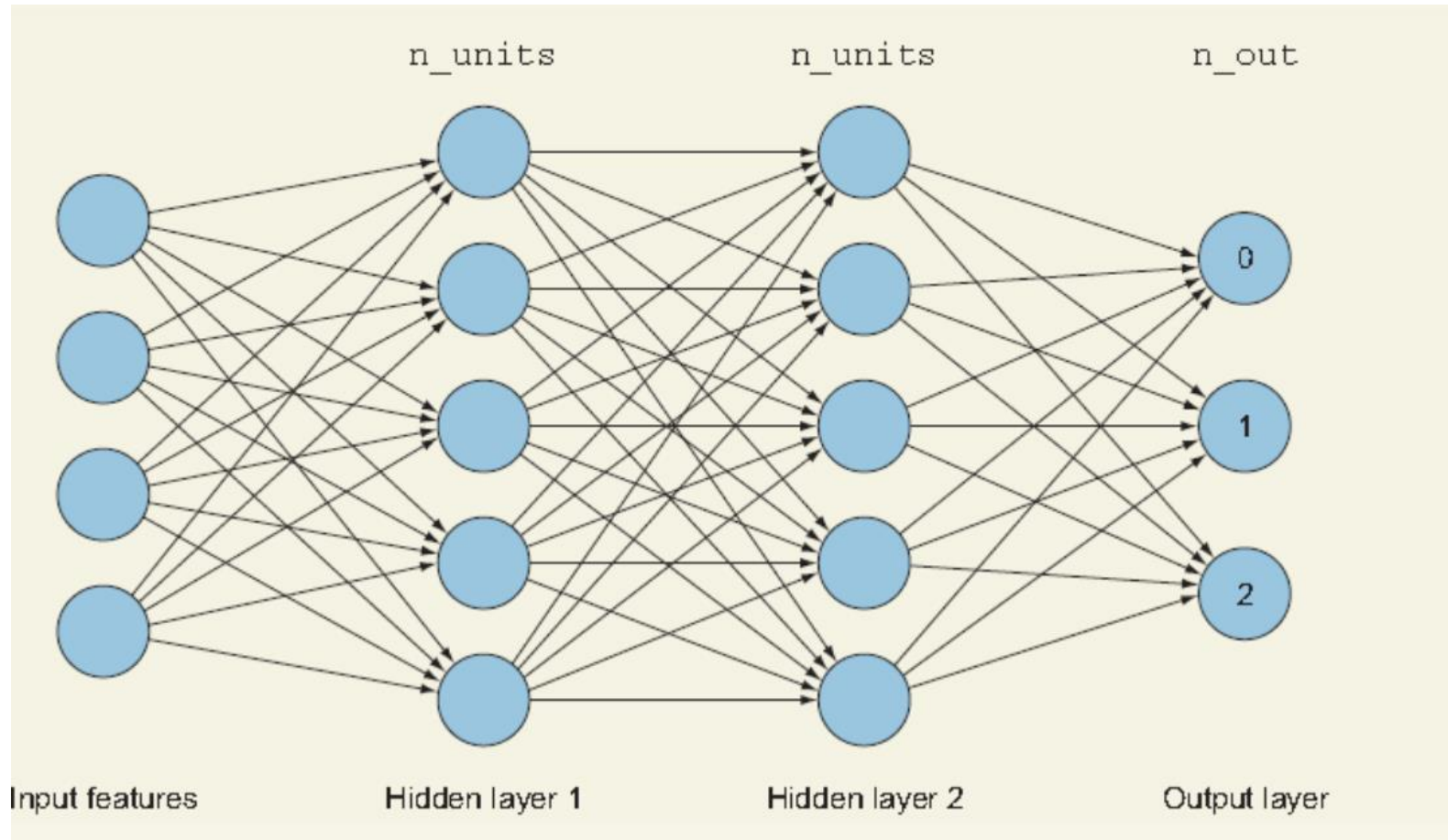
MLP multilayer perceptrons or Fully Connected

- **MLP architecture**
- ***Input flattened vector***—if the features tube has the dimensions of $5 \times 5 \times 40$, the flattened vector will be (1, 1000).
- ***Hidden layer***—We add one or more fully connected layers, and each layer has one or more neurons.
- ***Output layer*** recommended using the softmax activation function for
- classification problems involving more than two classes.
- In this example, we are classifying digits from 0 to 9: 10 classes

MLP multilayer perceptrons or Fully Connected



MLP multilayer perceptrons or Fully Connected

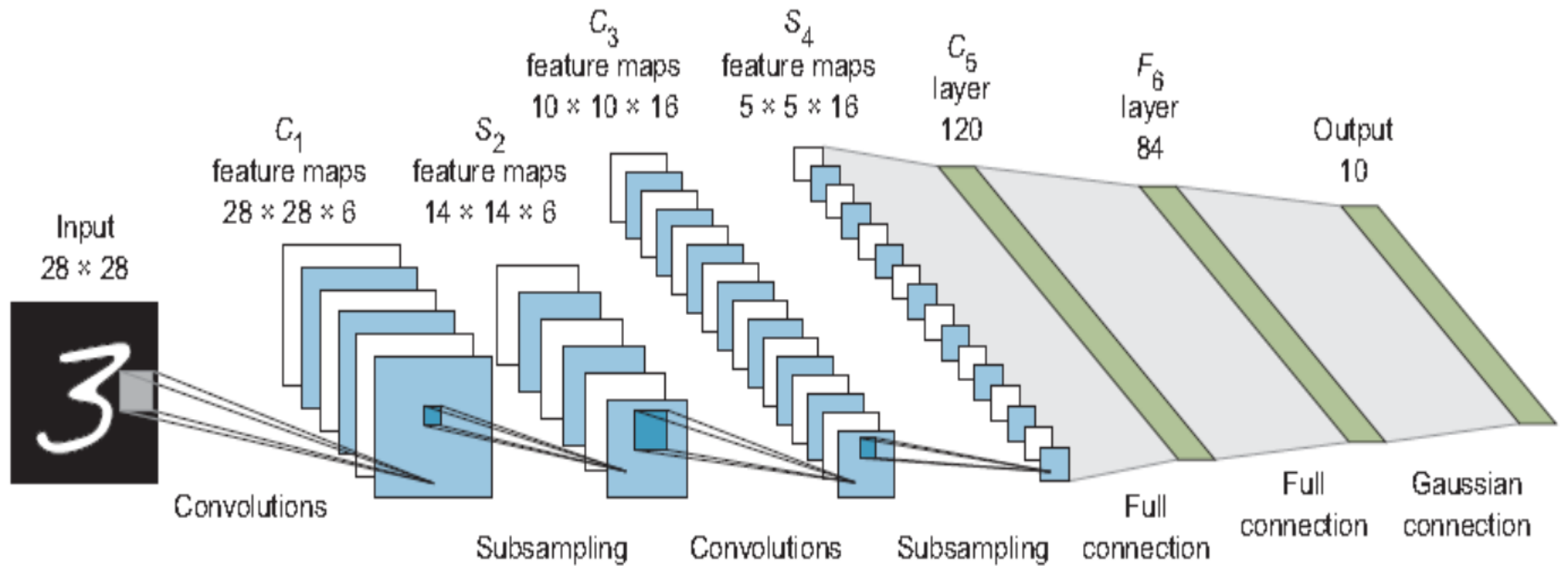


all the nodes from one layer are connected to all the nodes in the previous and next layers. They are also called *dense layers*.

LeNet-5

- The architecture is composed of five weight layers, and hence the name LeNet-5:
- **three convolutional layers and two fully connected layers**
- INPUT IMAGE $\Rightarrow$ C1 $\Rightarrow$ TANH $\Rightarrow$ S2 $\Rightarrow$ C3 $\Rightarrow$ TANH $\Rightarrow$ S4 $\Rightarrow$ C5 $\Rightarrow$ TANH $\Rightarrow$ FC6 $\Rightarrow$ SOFTMAX7
- where C is a convolutional layer, S is a subsampling or pooling layer, and FC is a fully connected layer

LeNet-5

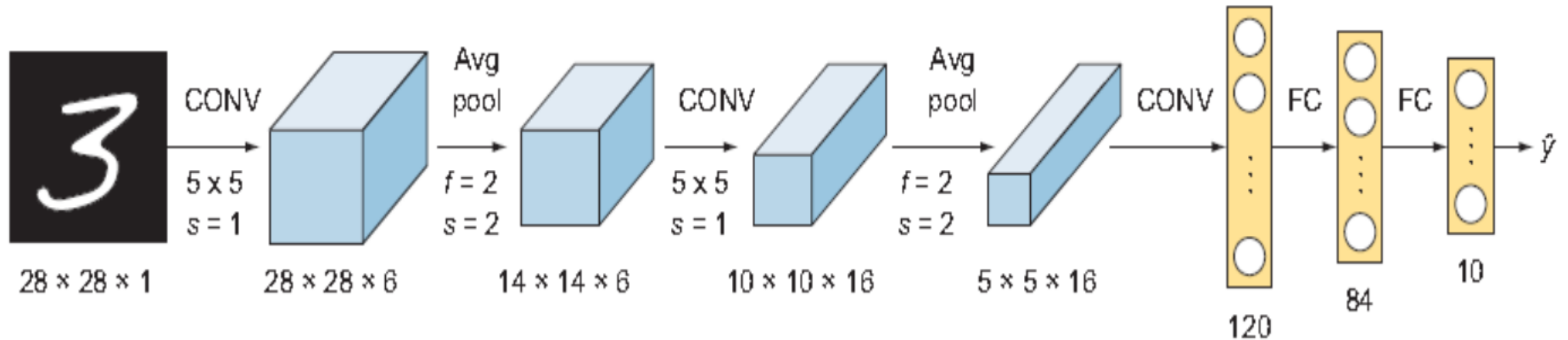


LeNet-5

- **LeNet-5 implementation in Keras**
- *Number of filters in each convolutional layer :*
- C1 has 6, C3 has 16, C5 has 120 layers
- *Kernel size of each convolutional layer— 5X5*
- **Subsampling (pooling) layers**—pool_size is 2, used *average pooling*
- **Activation function** tanh activation function
- When you train LeNet-5 on the MNIST dataset, you will get above 99% accuracy

LeNet-5

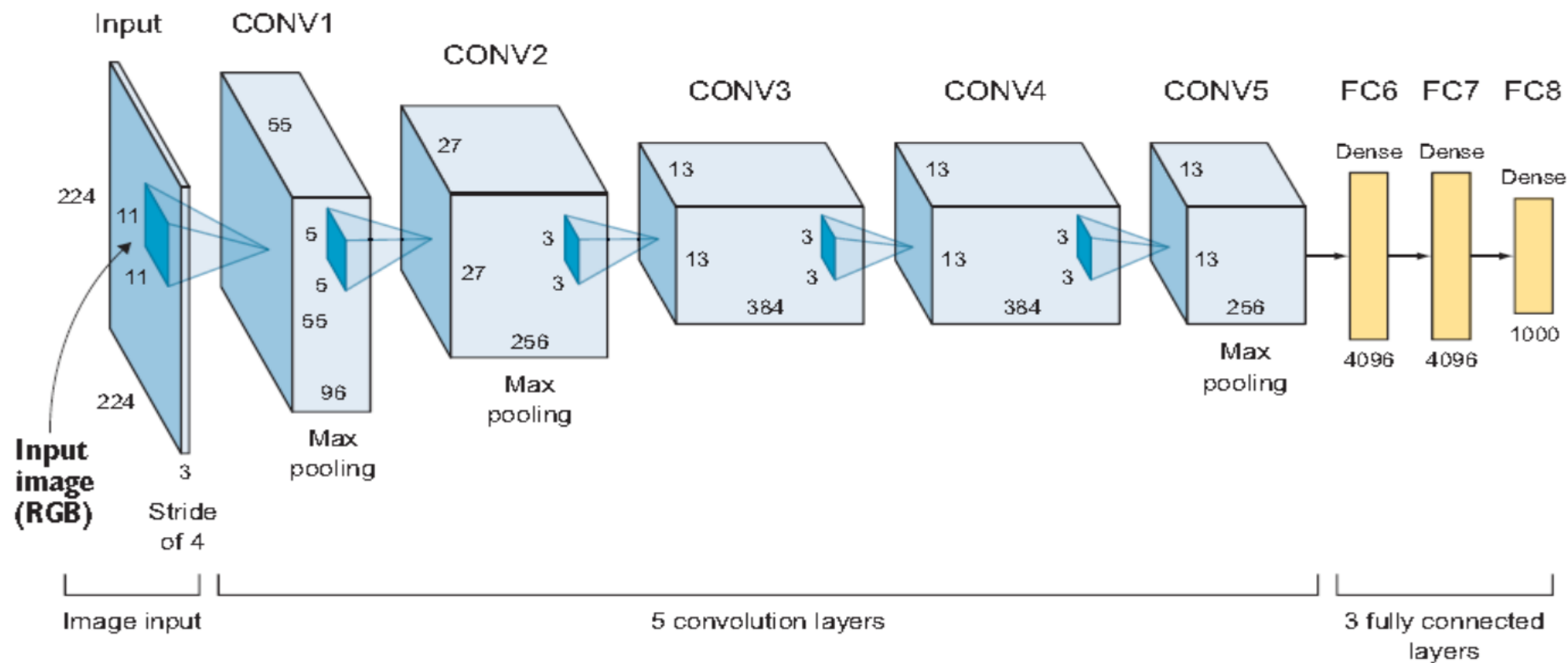
- The LeNet architecture consists of convolutional kernels of size 5×5 ; pooling layers; an activation function (tanh); and three fully connected layers with 120, 84, and 10 neurons, respectively



AlexNet

- LeNet performs very well on the MNIST dataset. But it turns out that the MNIST dataset is very simple because it contains **grayscale images** (1 channel) and classifies into only 10 classes, which makes it an easier challenge.
- The main motivation behind Alex-Net was to build a deeper network that can learn more complex functions.
- AlexNet was the winner of the ILSVRC image classification competition in 2012.
- Krizhevsky et al. created the neural network architecture and trained it on 1.2 million **high-resolution images** into 1,000 different classes of the ImageNet dataset.

AlexNet Architecture



AlexNet Architecture

- AlexNet has a lot of similarities to LeNet but is much deeper (more hidden layers) and bigger (more filters per layer).
- They have similar building blocks: a series of convolutional and pooling layers stacked on top of each other followed by fully connected layers and a softmax.
- We've seen that LeNet has around 61,000 parameters, whereas AlexNet has about 60 million parameters and 650,000 neurons,
- which gives it a larger learning capacity to understand more complex features.
- This allowed AlexNet to achieve remarkable performance in the ILSVRC image classification competition in 2012.

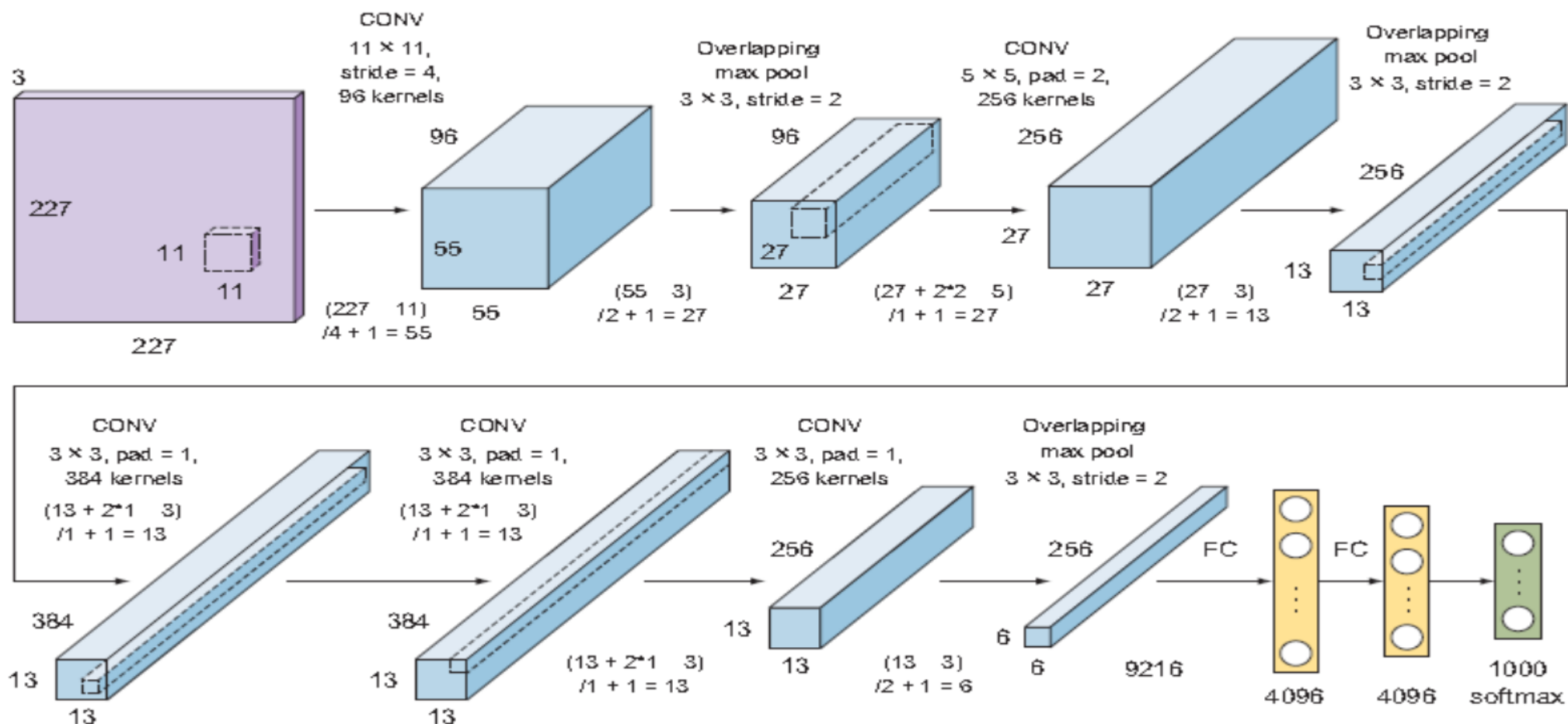
AlexNet architecture

- Convolutional layers with the following kernel sizes: 11×11 , 5×5 , and 3×3
- Max pooling layers for images downsampling
- Dropout layers to avoid overfitting
- Unlike LeNet, ReLU activation functions in the hidden layers and a softmax activation in the output layer.
- AlexNet consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully connected layers with a final 1000-way softmax that produces a distribution over the 1,000 class labels.
- INPUT IMAGE $\Rightarrow$ CONV1 $\Rightarrow$ POOL2 $\Rightarrow$ CONV3 $\Rightarrow$ POOL4 $\Rightarrow$ CONV5 $\Rightarrow$ CONV6 $\Rightarrow$ CONV7 $\Rightarrow$ POOL8 $\Rightarrow$ FC9 $\Rightarrow$ FC10 $\Rightarrow$ SOFTMAX7

AlexNet architecture

- AlexNet input starts with $227 \times 227 \times 3$ images
- *CONV1*—The authors used a large kernel size (11). They also used a large stride (4), which makes the input dimensions shrink by roughly a factor 4 (from 227×227 to 55×55).
- $\frac{(227-11)}{4} + 1 = 55$
- and the depth is the number of filters in the convolutional layer (96). The output dimensions are **$55 \times 55 \times 96$** .

AlexNet architecture



AlexNet contains eight weight layers: five convolutional and three fully connected. Two contain 4,096 neurons, and the output is fed to a 1,000-neuron softmax.

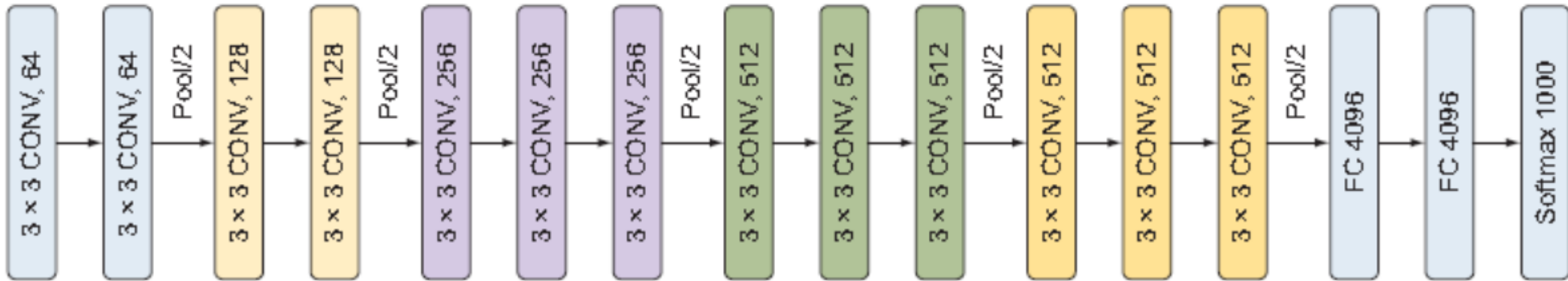
AlexNet architecture

- Similarly, we can calculate the output dimensions of the remaining layers:
- *CONV2*—Kernel size = 5, depth = 256, and stride = 1
- *POOL*—Size = 3×3 , which downsamples its input dimensions from 27×27 to 13×13
- *CONV3*—Kernel size = 3, depth = 384, and stride = 1
- *CONV4*—Kernel size = 3, depth = 384, and stride = 1
- *CONV5*—Kernel size = 3, depth = 256, and stride = 1
- *POOL*—Size = 3×3 , which downsamples its input from 13×13 to 6×6
- *Flatten layer*—Flattens the dimension volume $6 \times 6 \times 256$ to $1 \times 9,216$
- *FC* with 4,096 neurons
- *FC* with 4,096 neurons
- *Softmax layer* with 1,000 neurons

VGGNet

- VGGNet was developed in 2014 by the Visual Geometry Group at Oxford University.
- VGGNet is an even deeper network with more convolutional, pooling, and dense layers.
- VGGNet, also known as VGG16, consists of 16 weight layers: **13 convolutional layers and 3 fully connected.**
- All convolutional layers are 3×3 kernel-sized filters with a strides value of 1 and a padding value of same.
- All pooling layers have a 2×2 pool size and a strides value of 2.

VGGNet-16 architecture



The VGGNet architecture is developed by stacking 3×3 convolutional layers with 2×2 pooling layers inserted after several convolutional layers.

This is followed by the traditional classifier, which is composed of fully connected layers and a softmax.

the following regularization techniques

to avoid overfitting:

L2 regularization with weight decay of 5×10^{-4} .

Dropout regularization for the first two fully connected layers, with a dropout ratio set to 0.5.

Inception and GoogLeNet

- The Inception network came to the world in 2014 when a group of researchers at Google published their paper, “Going Deeper with Convolutions.”
- The main hallmark of this architecture is **building a deeper neural network while improving the utilization of the computing resources** inside the network.
- One particular incarnation of the **Inception network is called GoogLeNet** and was used in the team’s submission for ILSVRC 2014.
- It uses a network **22 layers deep (deeper than VGGNet)**
- **while reducing the number of parameters by 12 times** (from ~138 million to ~13 million) and achieving significantly more accurate results.

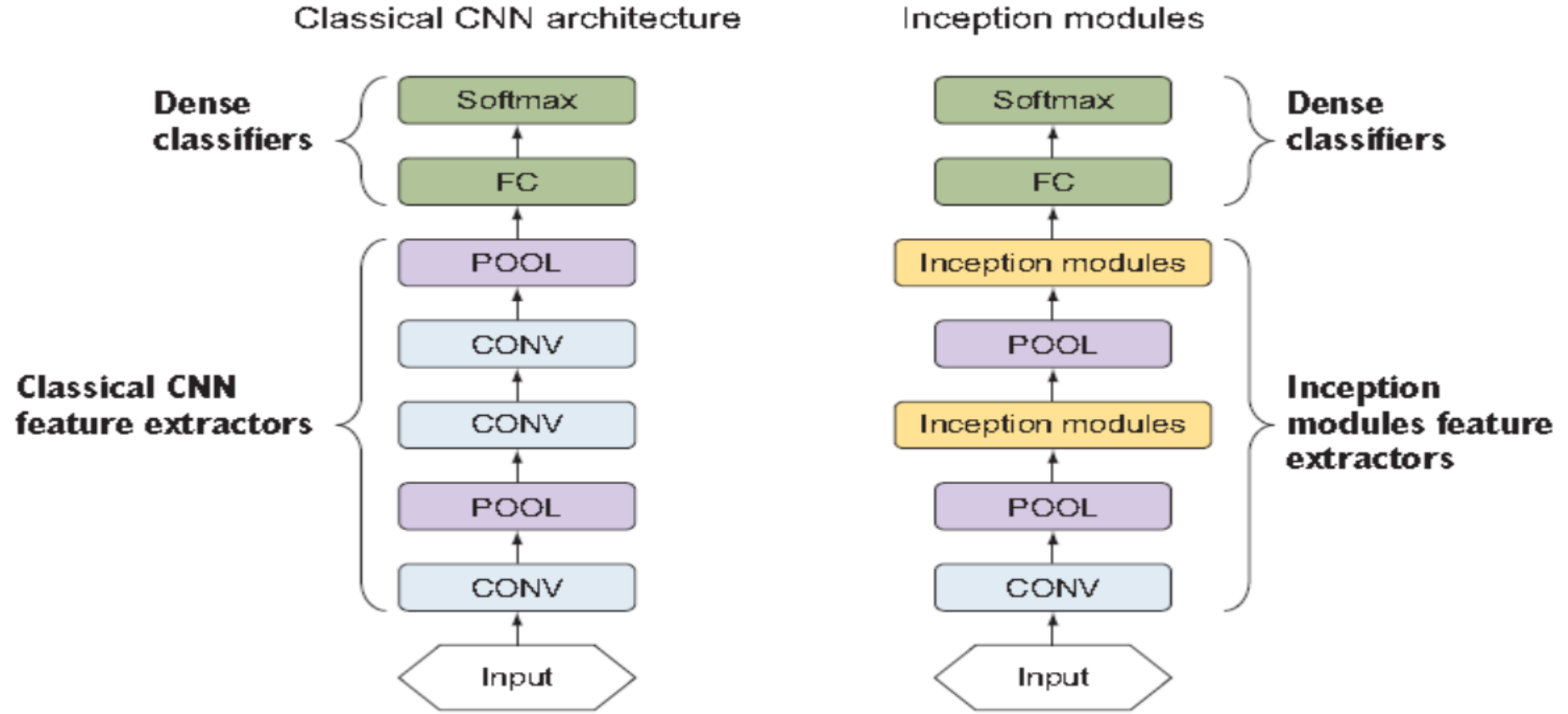
Inception and GoogLeNet

- ***Novel features of Inception***
- ***The kernel size of the convolutional layer***— 1×1 , 3×3 , 5×5 , and, in some cases, 11×11 (as in AlexNet).
- smaller kernels capture finer details of the image,
- whereas bigger filters will leave out minute details.
- ***When to use the pooling layer.***
- AlexNet uses pooling layers every one or two convolutional layers to downsize spatial features.
- VGGNet applies pooling after every two, three, or four convolutional layers as the network gets deeper.

Inception and GoogLeNet

- Inception says, “Instead of choosing a desired filter size in a convolutional layer and deciding where to place the pooling layers, let’s apply all of them all together in one block and call it the *inception module*.”
- **an inception module consisting of several convolutional layers with different kernel sizes.**
- The architecture is then developed by stacking the inception modules on top of each other.

Inception and GoogLeNet



Inception and GoogLeNet

- **In classical architectures like LeNet, AlexNet, and VGGNet,**
- we stack convolutional and pooling layers on top of each other to build the feature extractors.
- At the end, we add the dense fully connected layers to build the classifier.
- **In the Inception architecture,** we start with a convolutional layer and a pooling layer, stack the inception modules and pooling layers to build the feature extractors,
- and then add the regular dense classifier layers

Inception and GoogLeNet

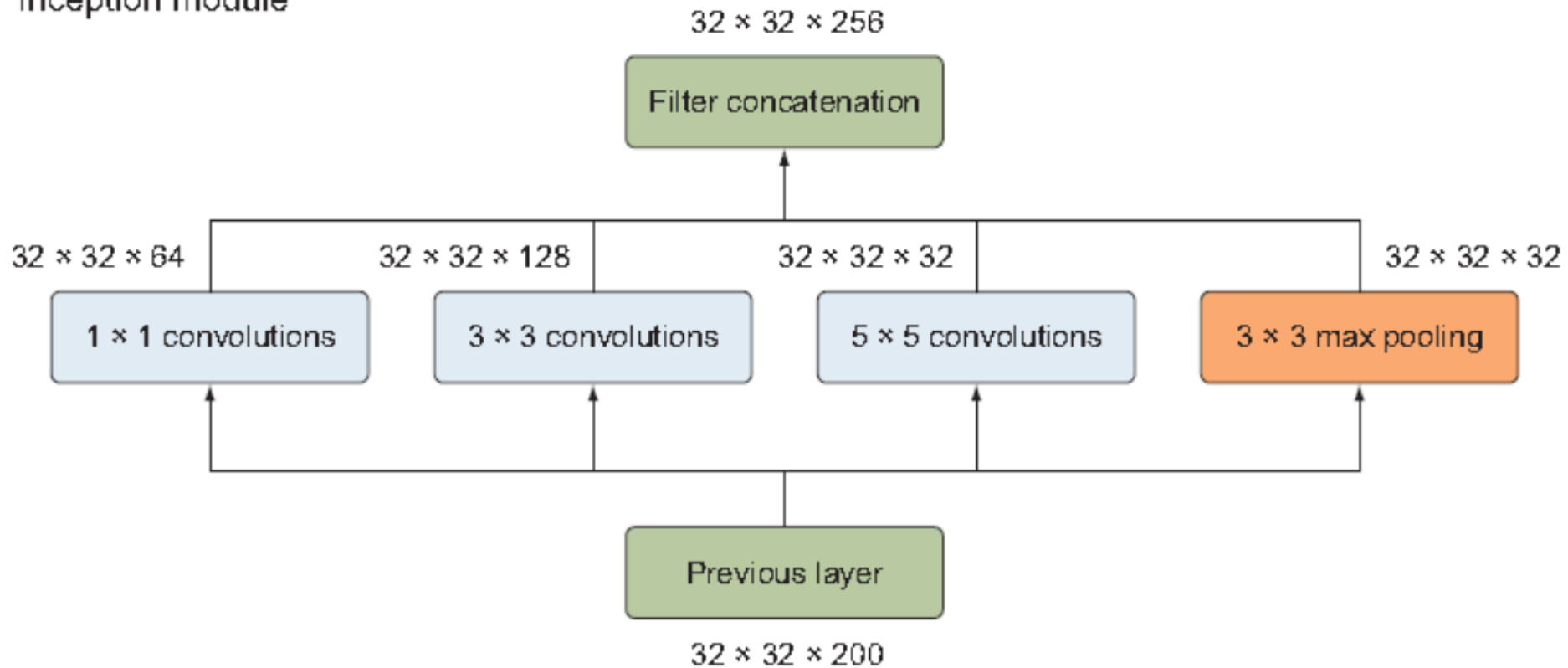
- *Inception module: Naive version*
- The inception module is a combination of four layers:
- **1×1 convolutional layer**
- 3×3 convolutional layer
- 5×5 convolutional layer
- **3×3 max-pooling layer**
- The outputs of these layers are concatenated into a single output volume forming the input of the next stage

Inception and GoogLeNet

- Suppose we have an input dimensional volume from the previous layer of size **$32 \times 32 \times 200$** .
- We feed this input to four convolutions simultaneously
- 1×1 convolutional layer with depth = 64 and padding = same.
- The output of this kernel = **$32 \times 32 \times 64$** .
- – 3×3 convolutional layer with depth = 128 and padding = same. Output = **$32 \times 32 \times 128$** .
- – 5×5 convolutional layer with depth = 32 and padding = same. Output = **$32 \times 32 \times 32$** .
- – 3×3 max-pooling layer with padding = same and strides = 1. Output = **$32 \times 32 \times 32$** .
- We concatenate the depth of the four outputs to create one output volume of dimensions **$32 \times 32 \times 256$** .
- **The same value results in padding** the input such that the output has the same length as the original input

Inception and GoogLeNet

Inception module



Now we have an inception module that takes an input volume of $32 \times 32 \times 200$ and outputs a volume of $32 \times 32 \times 256$

Inception module with dimensionality reduction

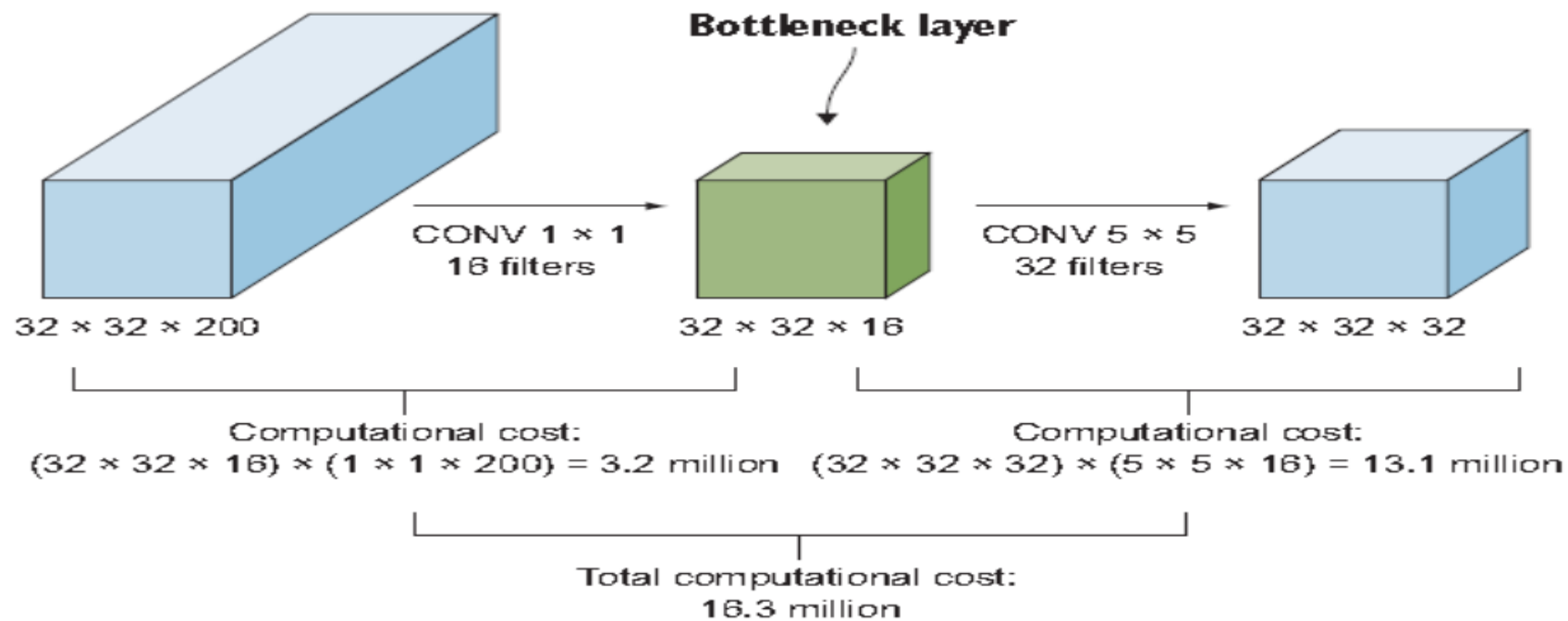
- The input volume with dimensions of $32 \times 32 \times 200$ will be fed to the 5×5 convolutional layer of 32 filters with dimensions = $5 \times 5 \times 32$. This means the total number of multiplications that the computer needs to compute is $32 \times 32 \times 200$ multiplied by $5 \times 5 \times 32$, which is more than 163 million operations.
- While we can perform this many operations with modern computers, this is still pretty expensive.
- **This is when the dimensionality reduction layers can be very useful**

Inception module with dimensionality reduction

- **DIMENSIONALITY REDUCTION LAYER (1×1 CONVOLUTIONAL LAYER)**
- The 1×1 convolutional layer can reduce the operational cost of 163 million operations to about a tenth of that.
- That is why it is called a *reduce layer*.
- The idea here is to add a 1×1 convolutional layer before the bigger kernels like the 3×3 and 5×5 convolutional layers, **to reduce their depth**, which in turn will reduce the number of operations.
- Suppose we have an input dimension volume of $32 \times 32 \times \mathbf{200}$.
- We then add a 1×1 convolutional layer with a depth of **16**.
- This reduces the dimension volume from **200 to 16 channels**.
- We can then apply the 5×5 convolutional layer on the output, which has much less depth

Inception module with dimensionality reduction

- Notice that $32 \times 32 \times 200$ input is processed through two convolutional layers and outputs a volume of dimensions $32 \times 32 \times 32$,



Dimensionality reduction is used to reduce the computational cost by reducing the depth of the layer.

Inception module with dimensionality reduction

- Computation

= operations in the 1×1 convolutional layer + operations in the 5×5 convolutional layer

= $(32 \times 32 \times 200)$ multiplied by $(1 \times 1 \times 16)$ + $(32 \times 32 \times 16)$ multiplied by $(5 \times 5 \times 32)$

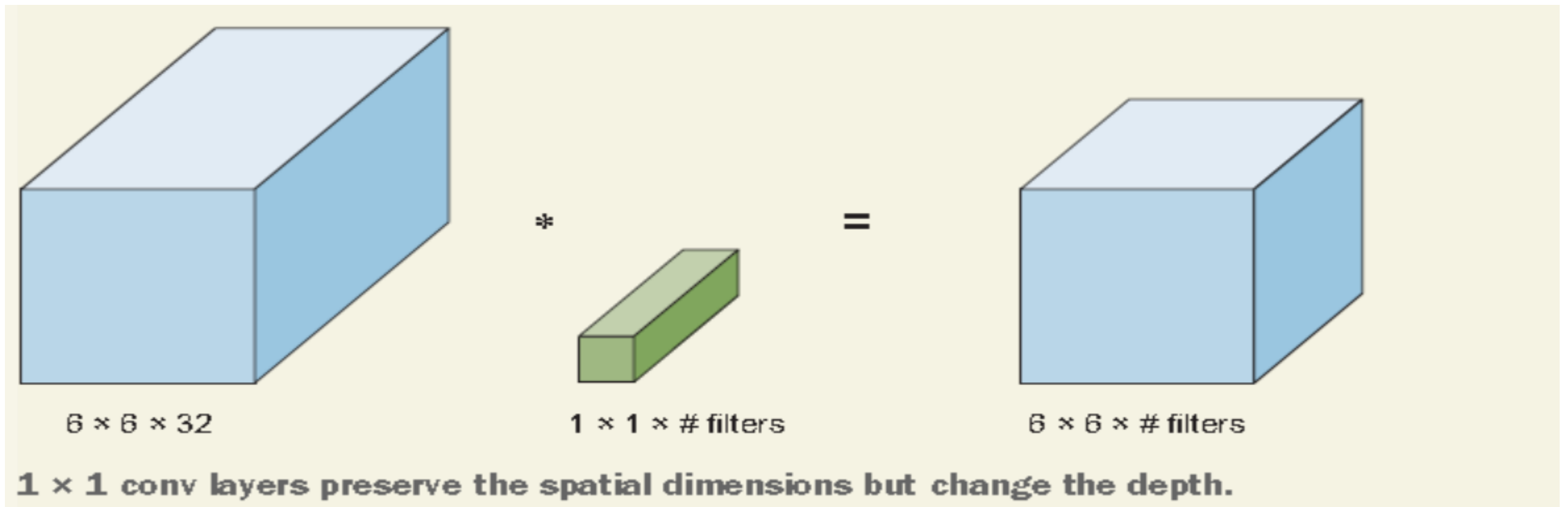
- = 3.2 million + 13.1 million

- The total number of multiplications in this operation is 16.3 million, which is a tenth of the 163 million multiplications that we calculated without the reduce layers

Dimensionality reduction is used to reduce the computational cost by reducing the depth of the layer.

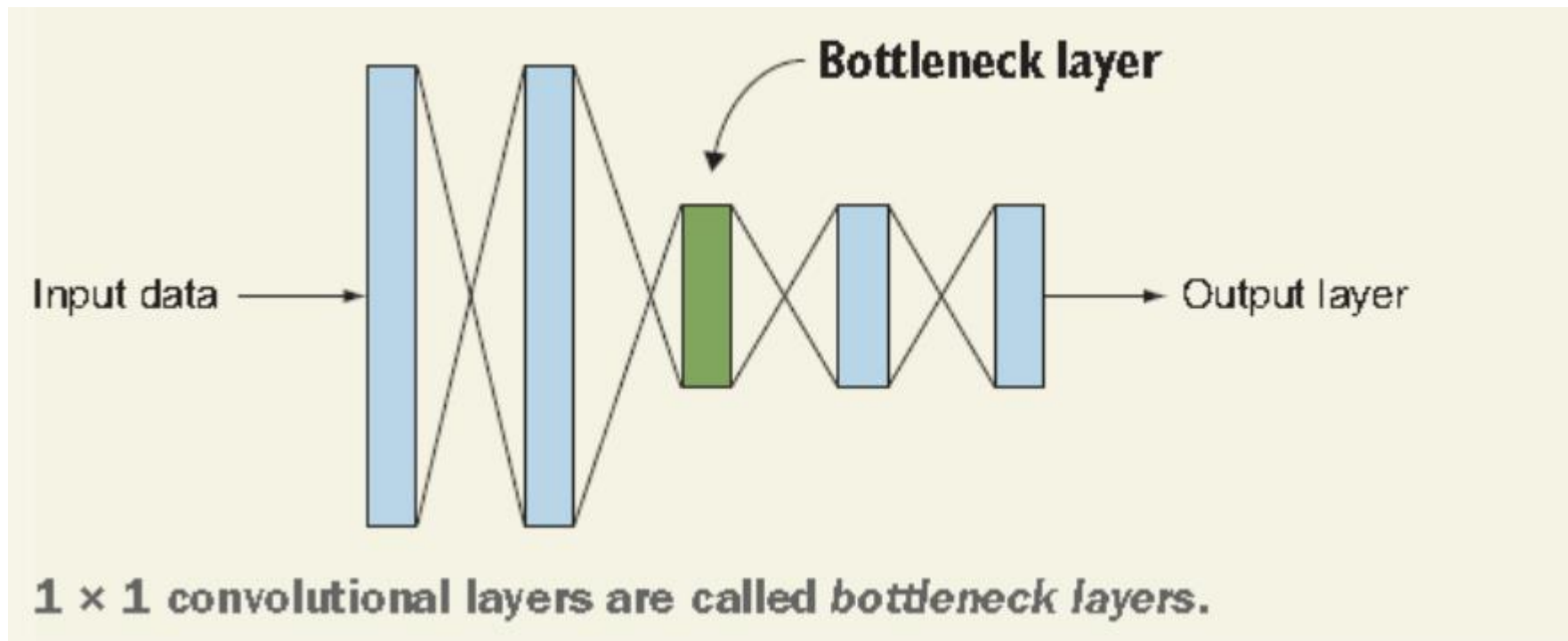
Inception module with dimensionality reduction

- The 1×1 convolutional layer
- The idea of the 1×1 convolutional layer is that it preserves the spatial dimensions (height and width) of the input volume but changes the number of channels of the volume (depth):



Inception module with dimensionality reduction

- The 1×1 convolutional layers are also known as *bottleneck layers* because the bottleneck is the smallest part of the bottle and reduce layers reduce the dimensionality of the network, making it look like a bottleneck:

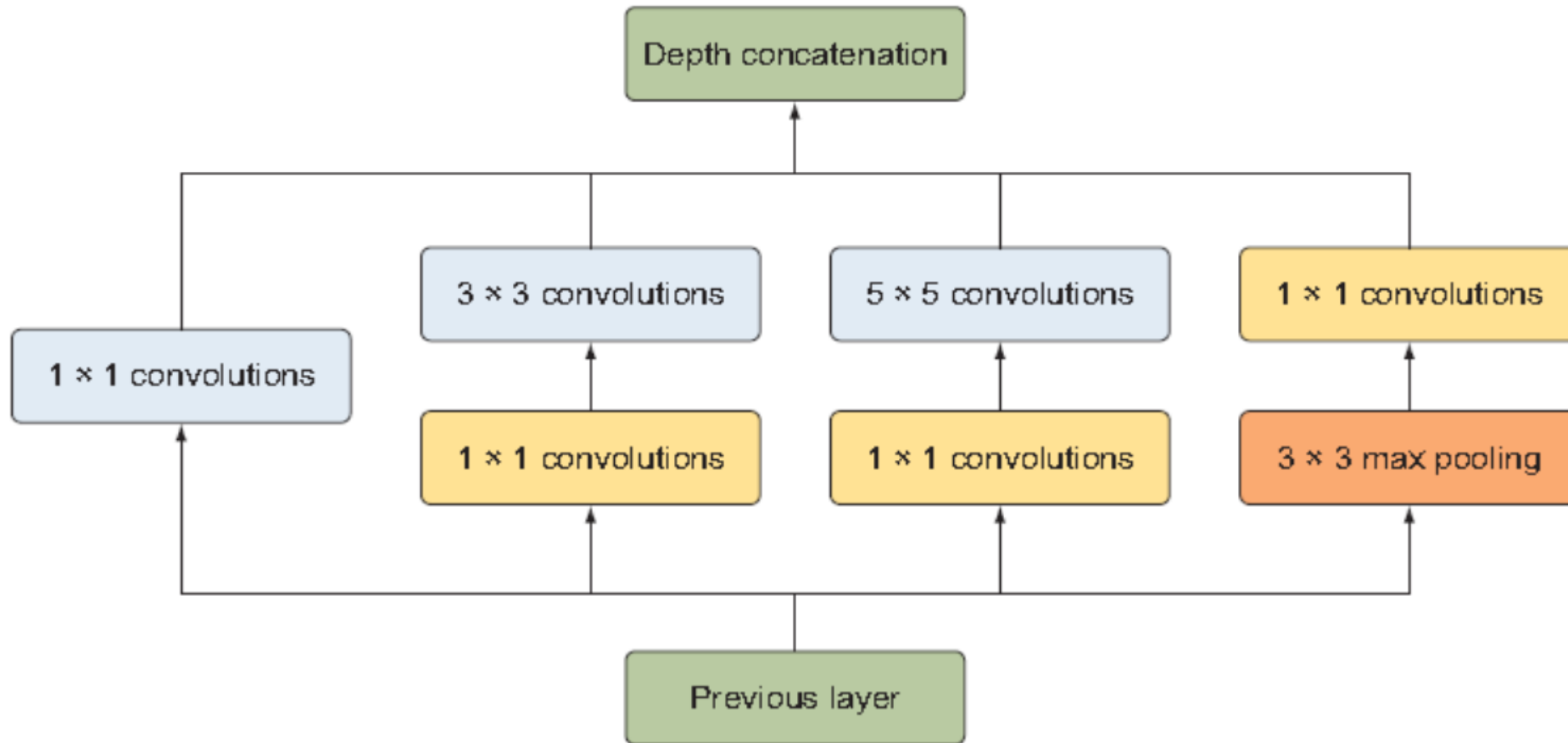


Inception module with dimensionality reduction

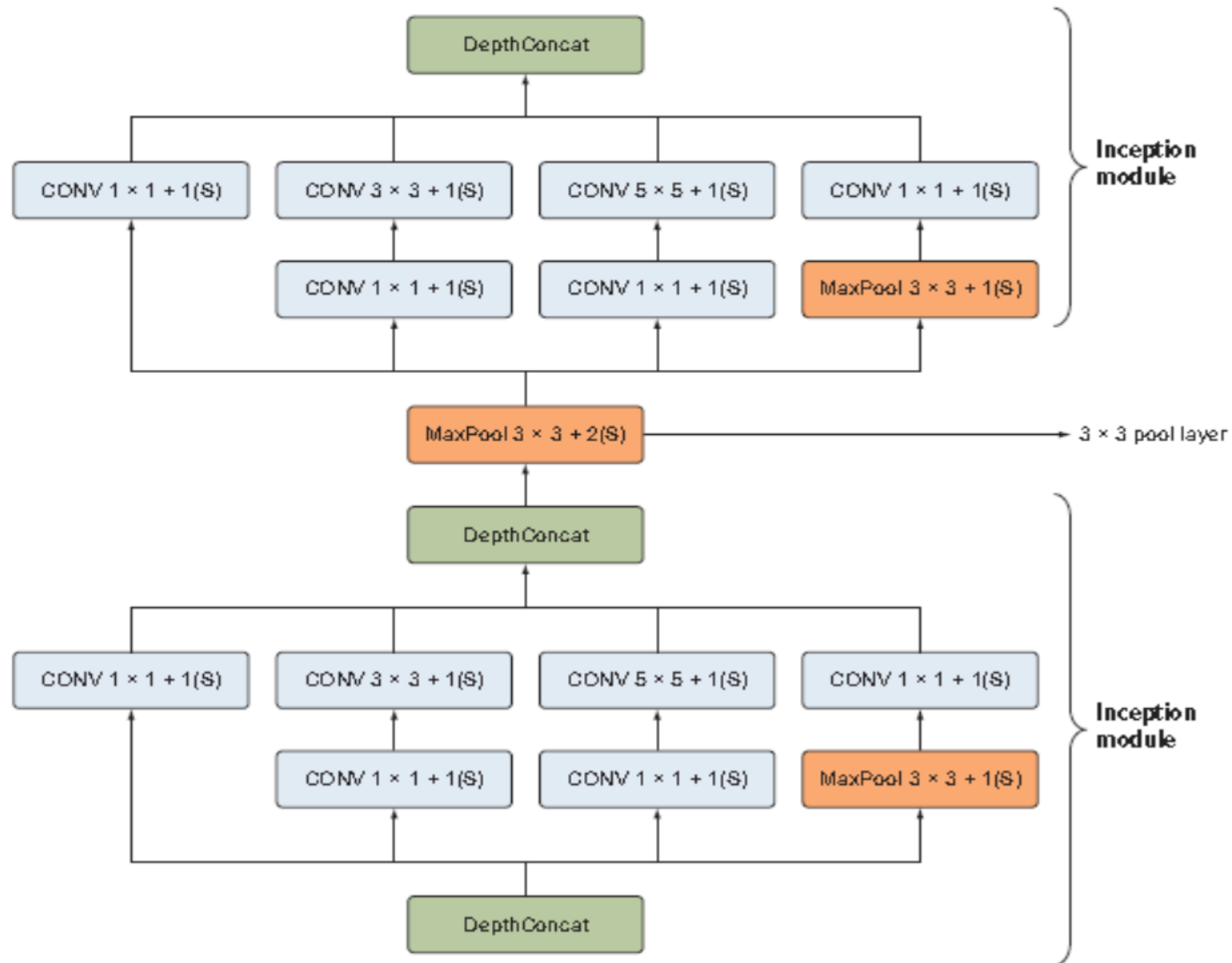
- We will add a 1×1 convolutional reduce layer before the 3×3 and 5×5 convolutional layers to reduce their computational cost.
- We will also add a 1×1 convolutional layer after the 3×3 max-pooling layer because pooling layers **don't reduce the depth for their inputs**.

Inception module with dimensionality reduction

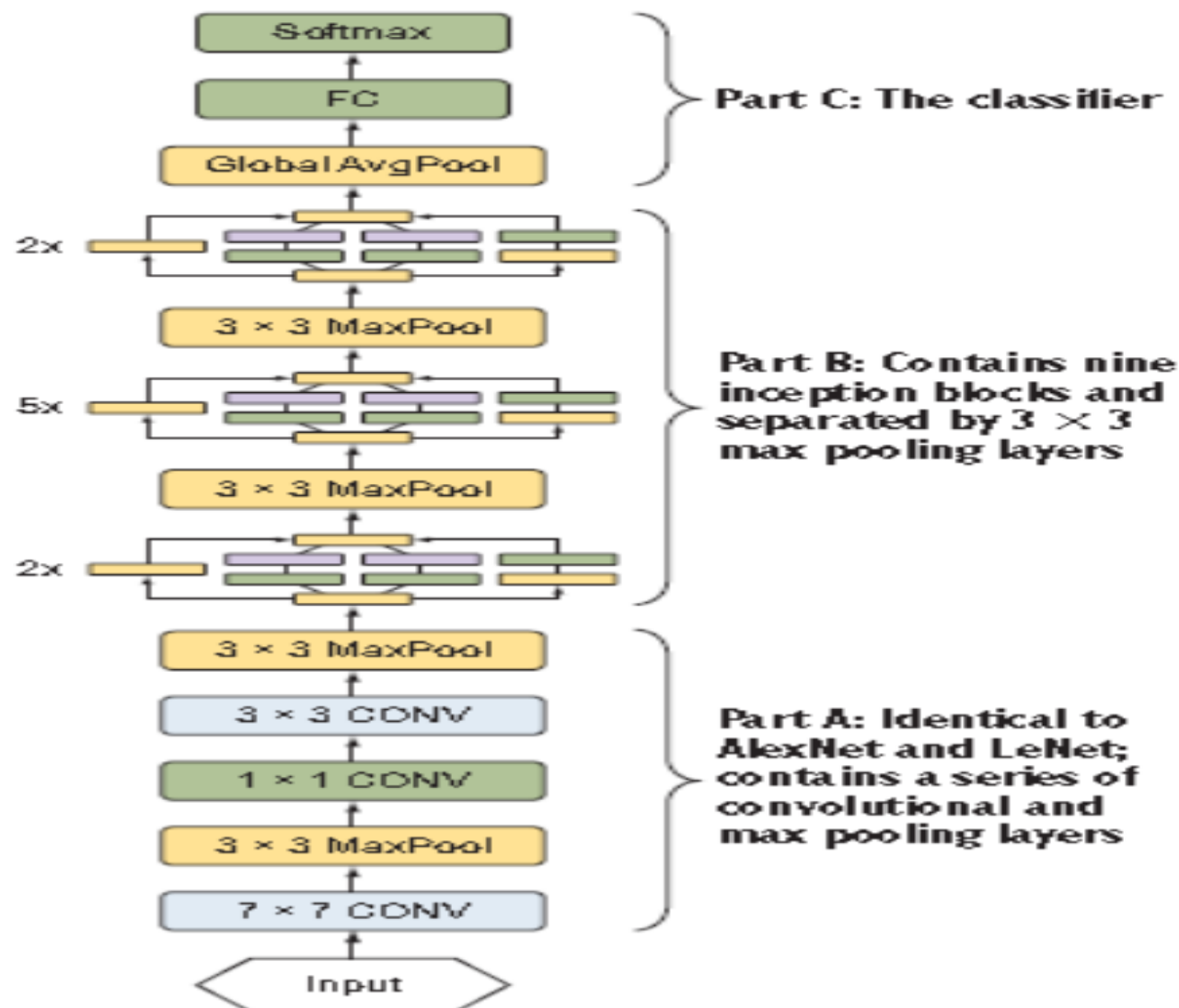
Inception module with dimensionality reduction



Inception architecture



GoogLeNet *architecture*



The full GoogLeNet model consists of three parts: the first part has the classical CNN architecture like AlexNet and LeNet, the second part is a stack of inceptions modules and pooling layers, and the third part is the traditional fully connected classifiers

GoogLeNet *architecture*

- To simplify this implementation, we are going to break down the GoogLeNet architecture into three parts:
- *Part A*—Identical to the AlexNet and LeNet architectures; contains a series of
- convolutional and pooling layers.
- *Part B*—Contains nine inception modules stacked as follows:
- **two inception modules + pooling layer +**
- **five inception modules + pooling layer +**
- **two inception modules.**
- *Part C*—The classifier part of the network, consisting of the fully connected and softmax layers.

ResNet

- The Residual Neural Network (ResNet) was developed in 2015 by a group from the Microsoft Research team.
- They introduced a novel *residual module* architecture **with skip connections**. The network also features **heavy batch normalization for the hidden layers**. This technique allowed the team to train very deep neural networks with **50,**
- **101, and 152 weight layers while still having lower complexity** than smaller networks like **VGGNet (19 layers)**.
- ResNet was able to achieve a top-5 error rate of 3.57% in the ILSVRC 2015 competition, **which beat the performance of all prior ConvNets**

ResNet

- you might have noticed that the deeper the network, the larger its learning capacity, and the better it extracts features from images.
- very deep networks are able to represent very complex functions,
- which allows the network to learn features at many different levels of abstraction, from edges (at the lower layers) to very complex features (at the deeper layers).
- one downside of adding too many layers is that doing so makes the network more **prone to overfit the training data**.
- This is not a major problem because we can use **regularization techniques like dropout, L2 regularization, and batch normalization to avoid overfitting**.
- So, if we can take care of the overfitting problem, wouldn't we want to build networks that are 50, 100, or even 150 layers deep?

The answer is yes.

ResNet

We definitely should try to build very deep neural networks. We need to fix just one other problem, to unblock the capability of building super-deep networks: a phenomenon called ***vanishing gradients***.

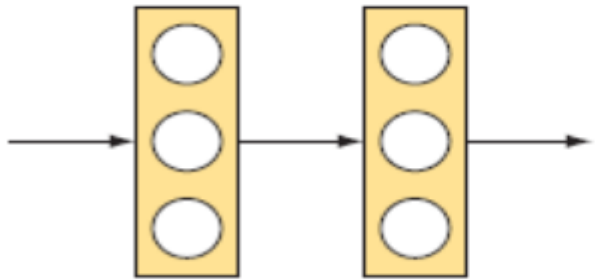
- As the network backpropagates the gradient of the error from the final layer back to the first layer, it is multiplied by the **weight matrix at each step**;
- thus the gradient can decrease exponentially quickly to zero, leading to a vanishing gradient phenomenon that prevents the earlier layers from learning.
- As a result, the network's performance gets saturated or even starts to degrade rapidly.
- In other cases, the gradient grows exponentially quickly and “explodes” to take very large values.
- This phenomenon is called ***exploding gradients***.

ResNet

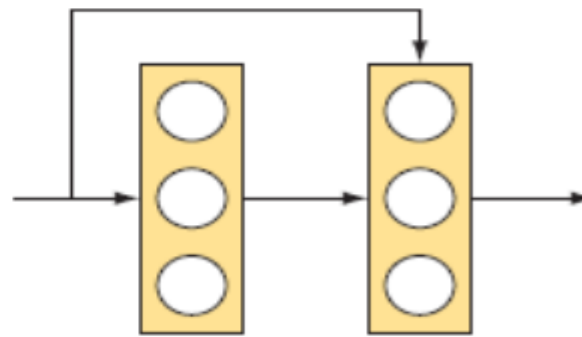
To solve the vanishing gradient problem.

- created a shortcut that allows the gradient to be directly backpropagated to earlier layers.
- These shortcuts are called *skip connections*:

Without skip connection



With skip connection

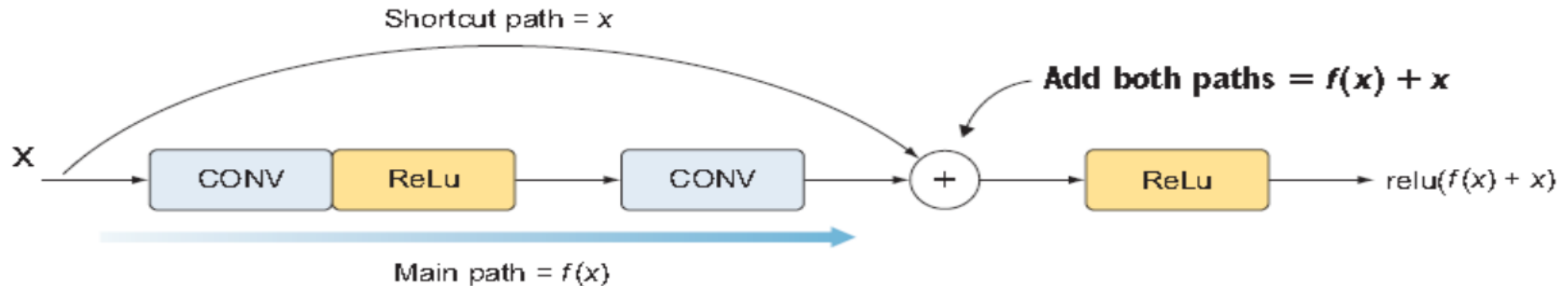


Traditional network without skip connections (left);
network with a skip connection (right).

ResNet

To solve the vanishing gradient problem.

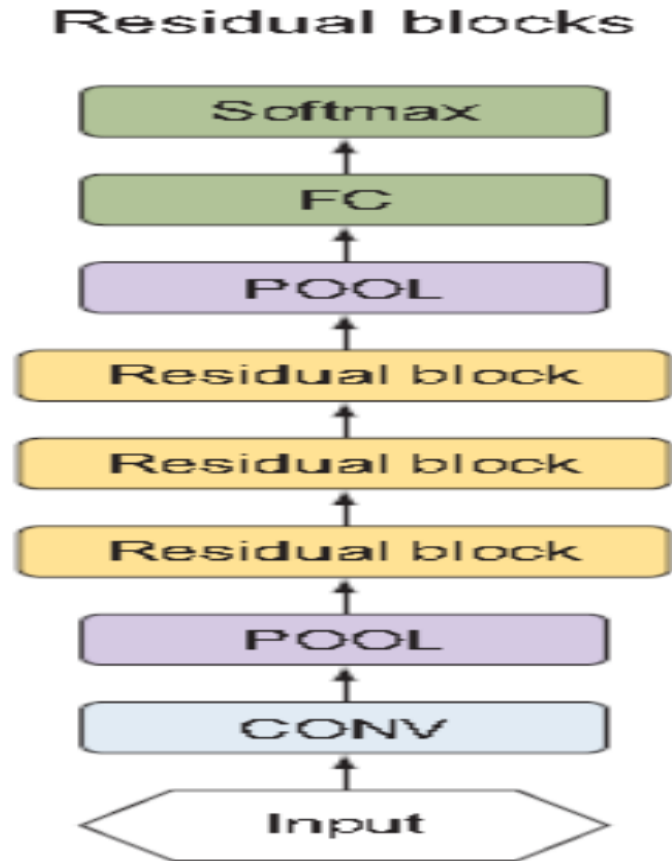
- they are used to flow information from earlier layers in the network to later layers, creating an alternate shortcut path for the gradient to flow through.
- Another important benefit of the skip connections is that they allow the model to learn an identity function,
- which ensures that the layer will perform at least as well as the previous layer.



Adding the paths and applying the ReLU activation function to solve the vanishing gradient problem that usually comes with very deep networks

ResNet

-



Feature extractors—To build the feature extractor part of ResNet, we start with a convolutional layer and a pooling layer and then stack residual blocks on top of each other to build the network.

When we are designing our ResNet network, we can add as many residual blocks as we want to build even deeper networks.

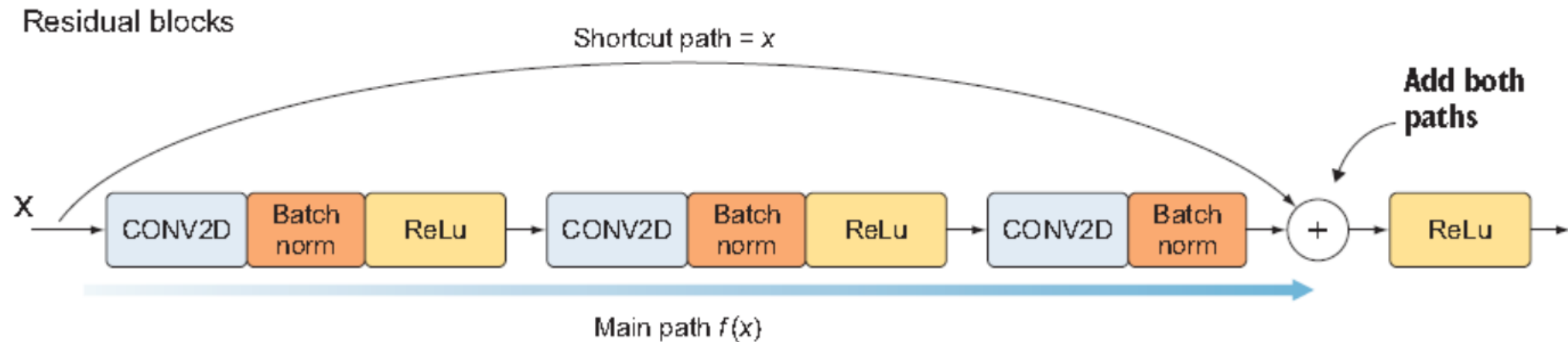
Classifiers—The classification part is still the same as we learned for other networks: fully connected layers followed by a softmax.

The residual network consists of a set of residual blocks

ResNet

- A residual module consists of two branches:
- ***Shortcut path*** Connects the input to an addition of second branch.
- ***Main path***—A series of convolutions and activations.
- main path consists of **3 convolutional layers with ReLU activations**.
- We also add **batch normalization** to each convolutional layer to **reduce overfitting and speed up training**.
- The **main path architecture** looks like this: **[CONV $\Rightarrow$ BN $\Rightarrow$ ReLU] $\times$ 3.**

ResNet



[CONV $\Rightarrow$ BN $\Rightarrow$ ReLU] $\times$ 3.

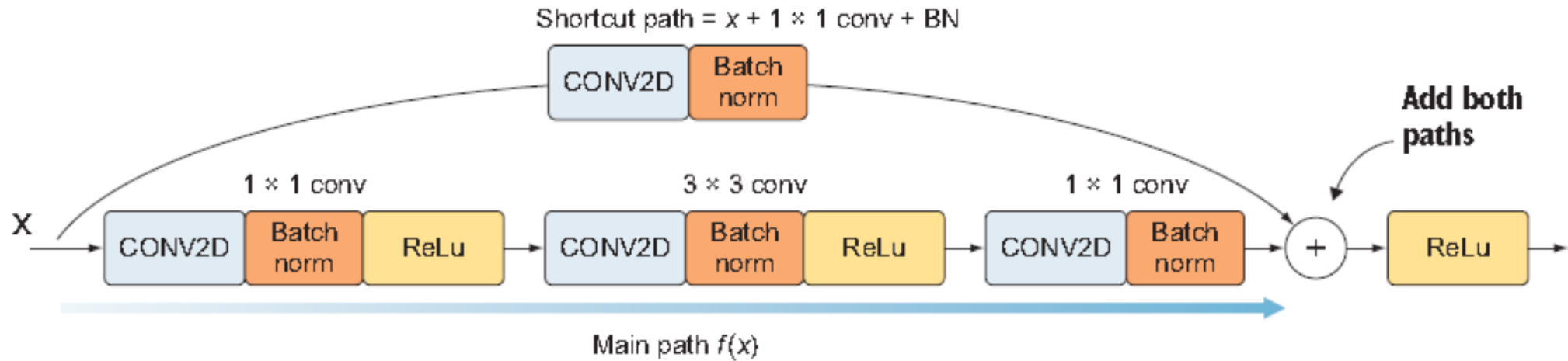
The output of the main path is added to the input value through the shortcut before they are fed to the ReLU function.

ResNet

- Notice that there are **no pooling layers in the residual block**.
- **dimension downsampling using bottleneck 1×1 convolutional layers**, similar to the Inception network.
- So, each residual block starts with a 1×1 convolutional layer to downsample the input dimension volume, and a 3×3 convolutional
- layer and another 1×1 convolutional layer to downsample the output.
- This is a good technique to keep control of the volume dimensions across many layers.
- This configuration is called a ***bottleneck residual block***

ResNet

Bottleneck residual block with reduce shortcut



To reduce the input dimensionality, we add a bottleneck layer (1×1 convolutional layer + batch normalization) to the shortcut path. This is called the ***reduce shortcut***.

ResNet

- **let's recap the discussion of residual blocks:**
- Residual blocks contain two paths: the shortcut path and the main path.
- The main path consists of three convolutional layers, and we add a batch normalization layer to them:
 - – **1 × 1 convolutional layer**
 - – **3 × 3 convolutional layer**
 - – **1 × 1 convolutional layer**
- **There are two ways to implement the shortcut path:**
 - – *Regular shortcut*—Add the input dimensions to the main path.
 - – *Reduce shortcut*—Add a convolutional layer in the shortcut path before merging with the main path.

Recurrent Neural Networks (RNN)

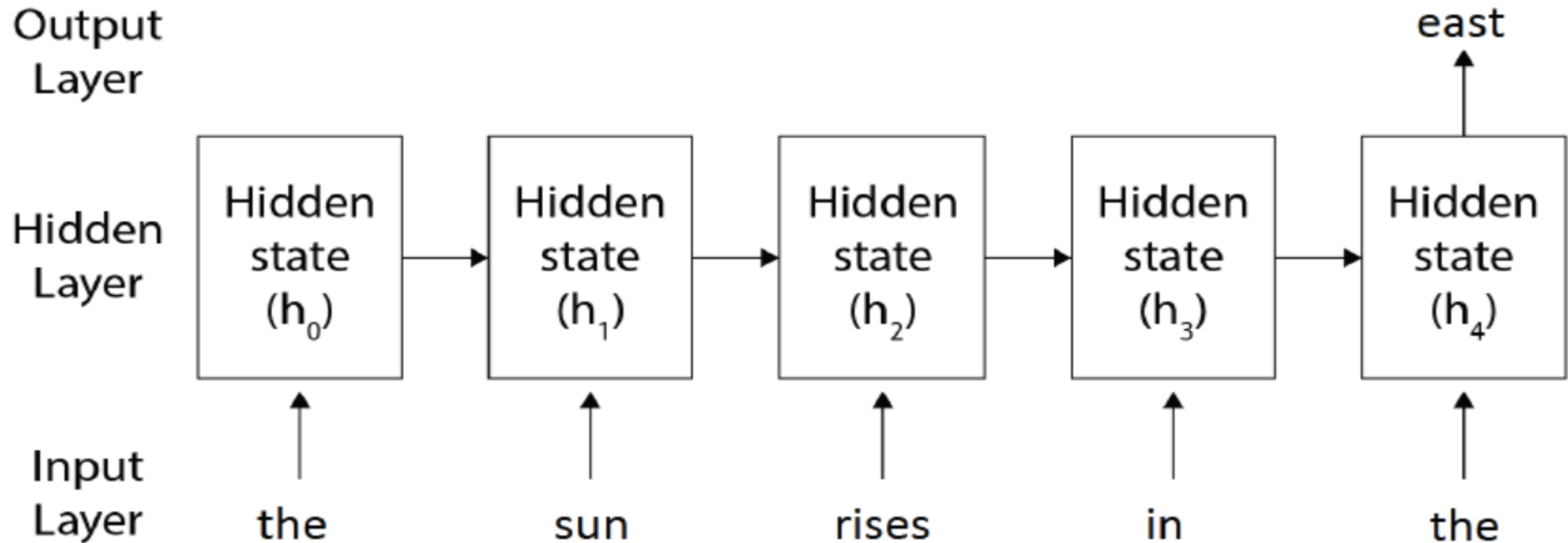
- *The Sun rises in the ____.*
- If we were asked to predict the blank term in the preceding sentence, we would probably say **east**.
- If we use a feedforward neural network to predict the blank, it would not predict the right word.
- each **input is independent of other input** and they make predictions based only **on the current input**, and they **don't remember previous input**.
- **Here is where we use RNNs.**
- They predict output not only based on the **current input**, but also on the **previous hidden state**

Recurrent Neural Networks (RNN)

- the previous hidden state acts like a memory and it captures the context of the sentence. With this context and the current input, we can predict the relevant word.
- ***The sun rises in the ____.***
- we first pass the word *the* as an input, and then we pass the next word, *sun*, as input; but along with this, we also pass the previous hidden state, h_0 .
- So, every time we pass the input word, we also pass a previous hidden state as an input.
- an **RNN uses the previous hidden state as memory** which captures and stores the contextual information (input) that the network has seen so far.

Recurrent Neural Networks (RNN)

- *The sun rises in the ____.*

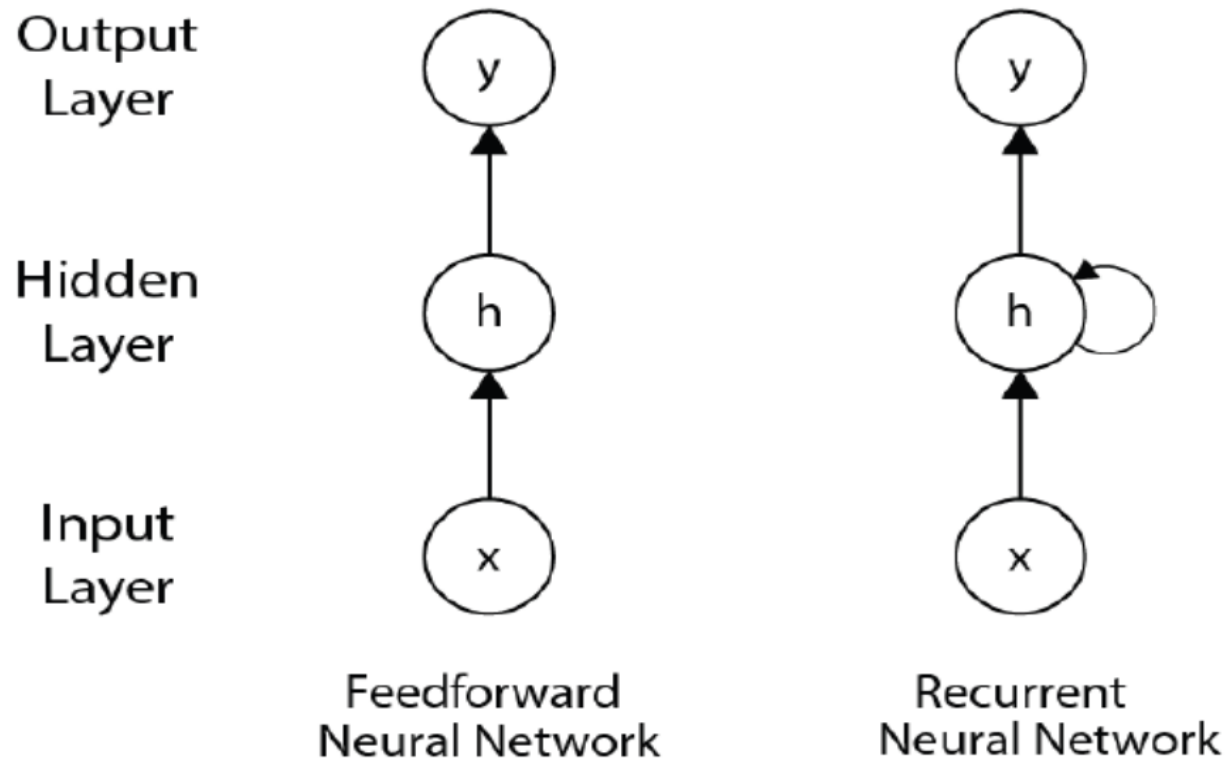


Recurrent Neural Networks (RNN)

- RNNs are widely applied for use cases that involve sequential data, such as
- time series, text, audio, speech, video, weather, and much more.
- They have been greatly used in various **natural language processing (NLP)** tasks, such as language translation, sentiment analysis, text generation, and so on.

Recurrent Neural Networks (RNN)

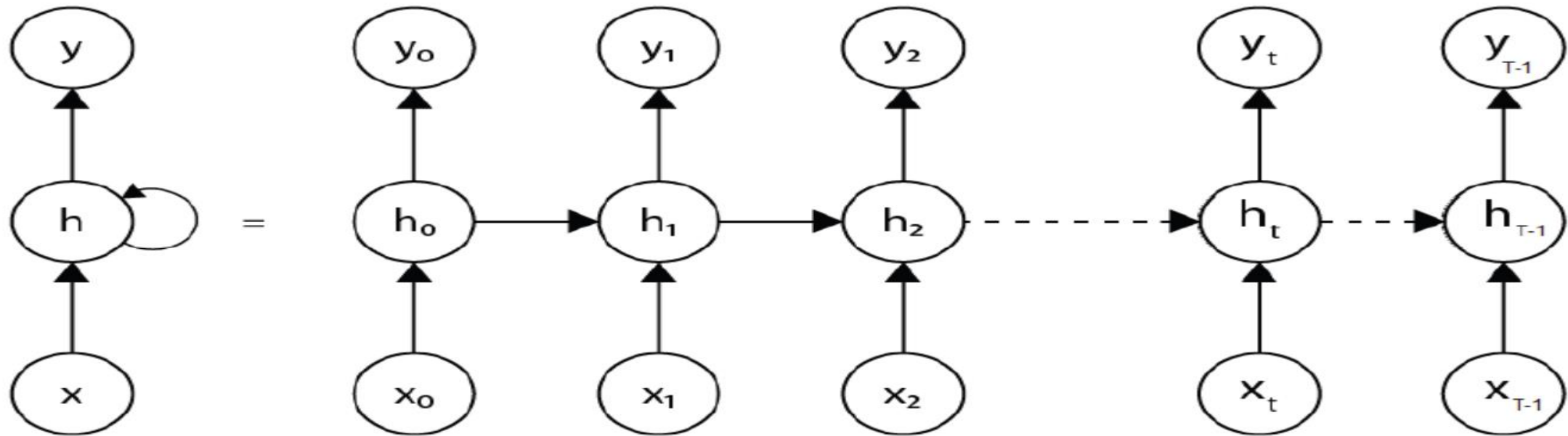
- A comparison between an RNN and a feedforward network



the RNN contains a looped connection in the hidden layer, which implies that we use the previous hidden state along with the input to predict the output.

Recurrent Neural Networks (RNN)

- Let's suppose that we have an input sentence with **T words**.
- then, we will have 0 to T-1 layers, one for each word,



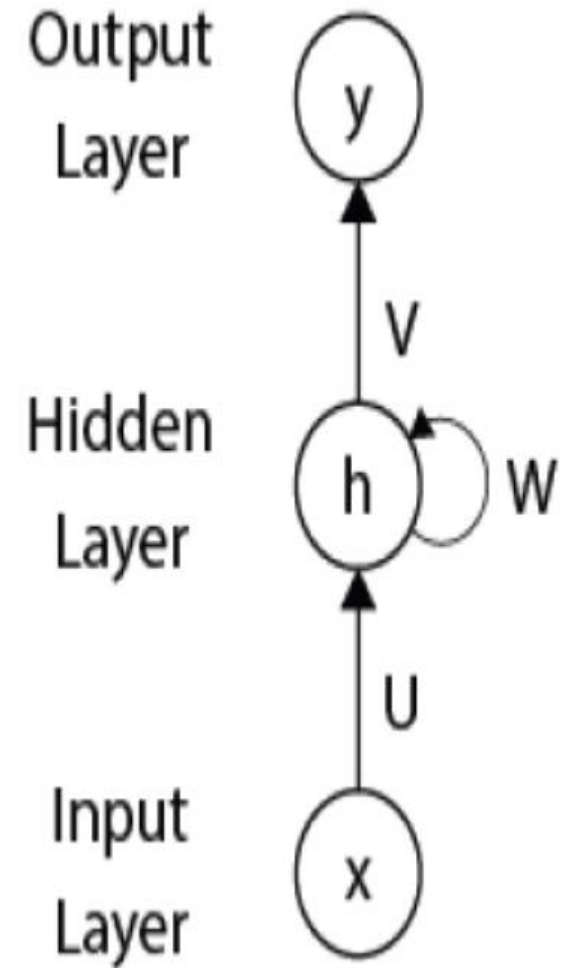
Recurrent Neural Networks (RNN)

- at the time step $t=1$, the output y_1 is predicted based on the current input x_1 and the previous hidden state h_0 .
- **Similarly**, at the time step $t=2$, the output y_2 is predicted based on the current input x_2 and the previous hidden state h_1 .
- This is how an RNN works; it takes the current input and the previous hidden state to predict the output.

Recurrent Neural Networks (RNN)

- **Forward propagation in RNNs**

- U represents the input to hidden layer weight matrix
- W represents the hidden to hidden layer weight matrix
- V represents the hidden to output layer weight matrix



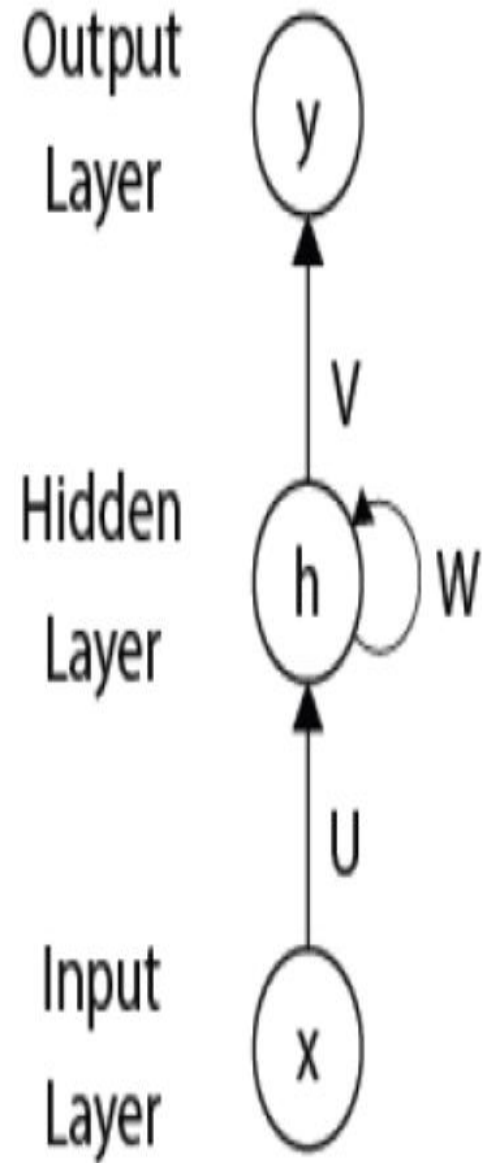
Recurrent Neural Networks (RNN)

- The hidden state h at a time step t :

$$h_t = \tanh(Ux_t + Wh_{t-1})$$

- *hidden state at a time step, $t =$*
- *$\tanh([input\ to\ hidden\ layer\ weight\ x\ input]$*
 $+ [hidden\ to\ hidden\ layer\ weight\ x\ previous\ hidden\ state])$.
- The output at a time step :

$$\hat{y}_t = \text{softmax}(Vh_t)$$



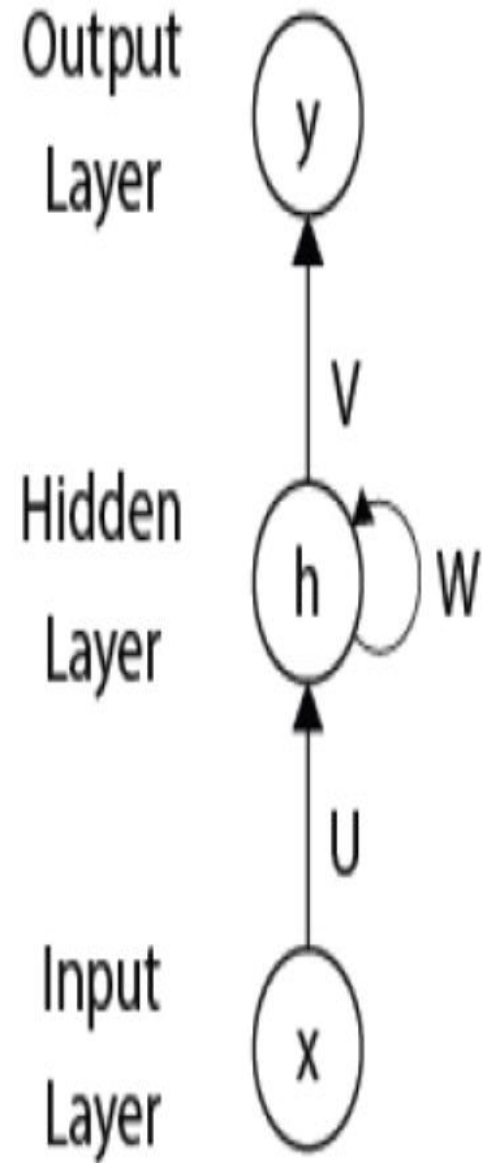
Recurrent Neural Networks (RNN)

- The hidden state h at a time step t :

$$h_t = \tanh(Ux_t + Wh_{t-1})$$

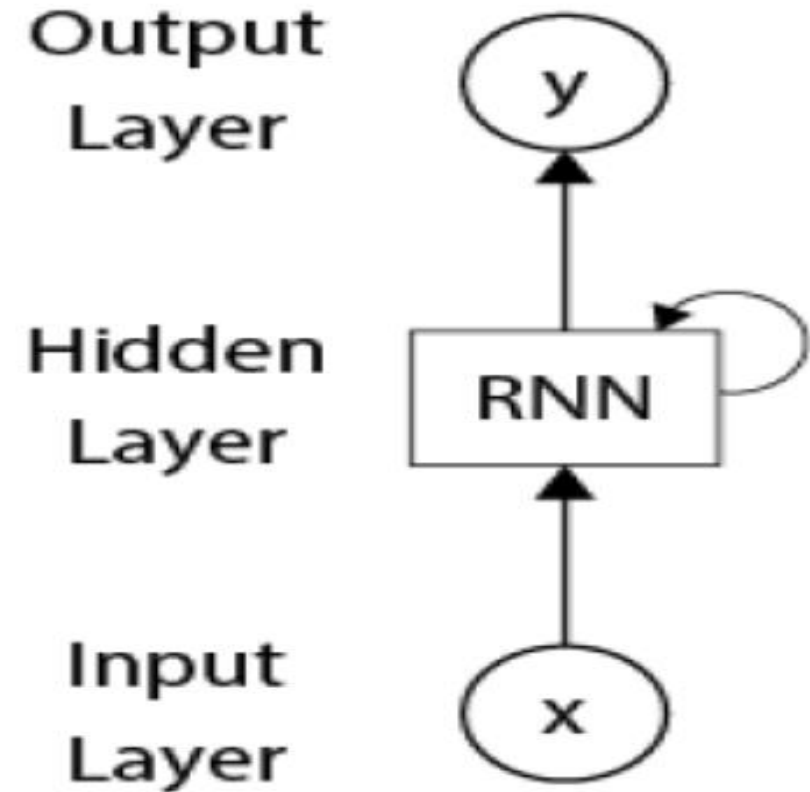
- *hidden state at a time step, $t =$*
- *$\tanh([input\ to\ hidden\ layer\ weight\ x\ input]$*
 $+ [hidden\ to\ hidden\ layer\ weight\ x\ previous\ hidden\ state])$.
- The output at a time step :

$$\hat{y}_t = \text{softmax}(Vh_t)$$



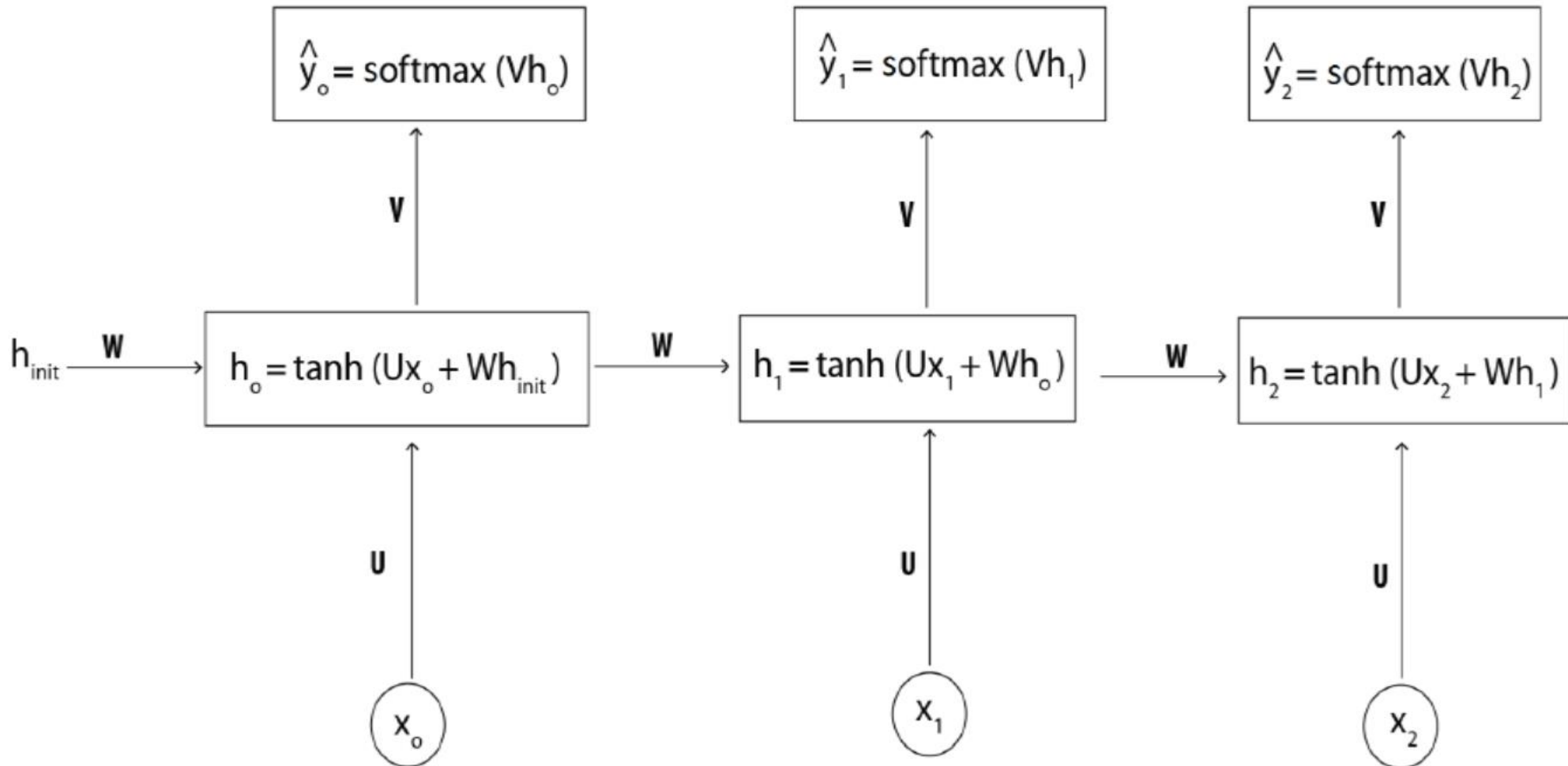
RNN

- the hidden layer is represented by an RNN block, which implies that our network is an RNN,
- and previous hidden states are used in predicting the output:



RNN

- forward propagation works in an unrolled version of an RNN



RNN

- We initialize the initial hidden state h_{init} with random values.
- As you can see in the preceding figure, the output, $\hat{y}_0$, is predicted based on the current input x_0 ,
- and the previous hidden state, h_{init} which is an initial hidden state, using the following formula

$$h_0 = \tanh(Ux_0 + Wh_{init})$$

$$\hat{y}_0 = \text{softmax}(Vh_0)$$

RNN

- Similarly

$$h_1 = \tanh(Ux_1 + Wh_0)$$

$$\hat{y}_1 = \text{softmax}(Vh_1)$$

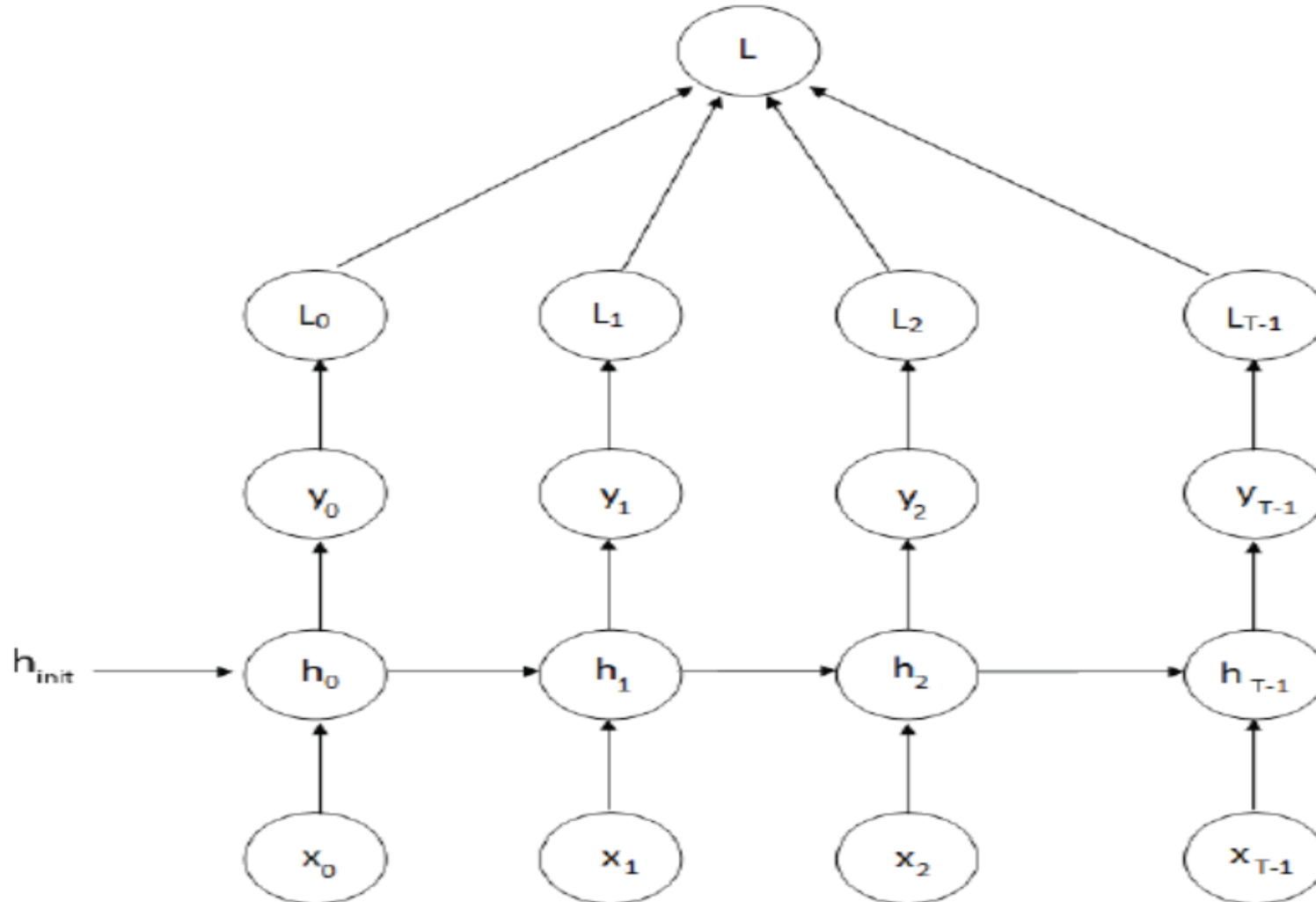
- Thus, in forward propagation to predict the output, RNN uses the **current input and the previous hidden state**

Backpropagating through time

- we compute the loss, L , at each time step, t ,
- We use the **crossentropy** loss as our loss function.
- The Loss L at a time step t , $L_t = -y_t \log(\hat{y}_t)$
- Where y_t is the actual output, and $\hat{y}_t$ is the predicted output at a time step t .
- The final loss is a sum of the loss at all the time steps.
- Suppose that we have $T-1$ layers;
- the final loss

$$L = \sum_{j=0}^{T-1} L_j$$

Backpropagating through time



our goal is to minimize the loss.
We can minimize the loss by
finding the optimal weights of
the RNN.

we have three weights in
RNNs:

input to hidden U ,
hidden to hidden W ,
and hidden to output V .

Backpropagating through time

- We begin by calculating the gradients of the loss function with respect to all the weights; then, we update the weights according to the weight update rule as follows
- $V = V - \alpha \frac{\partial L}{\partial V}$
- $W = W - \alpha \frac{\partial L}{\partial W}$
- $U = U - \alpha \frac{\partial L}{\partial U}$

Backpropagating through time

- , $L_t = -y_t \log(\hat{y}_t)$
- Thus, the gradient of the loss L with respect to $\hat{y}_t$
- $\frac{\partial L}{\partial \hat{y}_t} = -\frac{y_t}{\hat{y}_t}$
- Now, we will learn how to calculate the gradient of the loss with respect to all the weights one by one.

Backpropagating through time

- Gradients with respect to the hidden to output weight, V
- $h_t = \tanh(Ux_t + Wh_{t-1})$
- $\hat{y}_t = \text{Softmax}(Vh_t)$
- $L_t = -y_t \log(\hat{y}_t)$
- Suppose $Z_t = Vh_t$
- $h_t = \tanh(Ux_t + Wh_{t-1})$
- $\hat{y}_t = \text{Softmax}(Z_t)$
- $L_t = -y_t \log(\hat{y}_t)$

Backpropagating through time

- **After predicting the output $\hat{y}$** , we are in the final layer of the network. Since we are backpropagating,
- that is, going from the **output layer to the input layer**,
- our first weight would be **V**, which is the **hidden to output layer weight**.
- We have seen that the final loss is the sum of the loss over all the time steps, and similarly, the final gradient is the sum of gradients over all the time steps:

$$\frac{\partial L}{\partial V} = \frac{\partial L_0}{\partial V} + \frac{\partial L_1}{\partial V} + \frac{\partial L_2}{\partial V} + \dots + \frac{\partial L_{T-1}}{\partial V}$$

Backpropagating through time

- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta V}$
- $L_t = -y_t \log(\hat{y}_t)$
- we cannot calculate the gradient with respect to V directly from L_t , as there are no V terms in it. **So, we apply the chain rule.**
- Recall the forward propagation equation; there is a V term in $\hat{y}_t$:
- $\hat{y}_t = \sigma(Z_t)$ where $Z_t = Vh_t$
- First, we calculate a partial derivative of the loss with respect to $\hat{y}_t$, and
- then, from $\hat{y}_t$, we will calculate the partial derivative with respect to Z_t
- From Z_t , we can calculate the derivative with respect to V .
- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta \hat{y}_j} \frac{\delta \hat{y}_j}{\delta Z_j} \frac{\delta Z_j}{\delta V}$

Backpropagating through time

- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta \hat{y}_j} \frac{\delta \hat{y}_j}{\delta Z_j} \frac{\delta Z_j}{\delta V}$
- As we know that, $\hat{y}_t = \text{softmax}(Z_t)$, the gradient of loss with respect to $\mathbf{z}_j$ can be calculated as follows
- $\frac{\delta L_j}{\delta \mathbf{z}_j} = \frac{\delta L_j}{\delta \hat{y}_j} \frac{\delta \hat{y}_j}{\delta \mathbf{z}_j}$
- Substituting equation
- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta \mathbf{z}_j} \frac{\delta \mathbf{z}_j}{\delta V}$

Backpropagating through time

- $\frac{\delta L_j}{\delta z_t} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta \hat{y}_j} \frac{\delta \hat{y}_j}{\delta z_t}$
- Since $\frac{\partial L}{\partial \hat{y}_t} = - \frac{y_t}{\hat{y}_t}$
- $\frac{\delta L_j}{\delta z_t} = \sum_{j=0}^{T-1} - \frac{y_t}{\hat{y}_j} \frac{\delta \hat{y}_j}{\delta z_t}$
- Now, we will compute the term $\frac{\delta \hat{y}_j}{\delta z_t}$, **since we know** $\hat{y}_t = \text{softmax}(Z_t)$
- $\frac{\delta \hat{y}_j}{\delta z_t} = \text{Softmax}'$
- $\frac{\delta \hat{y}_j}{\delta z_t} = \begin{cases} -\hat{y}_j \hat{y}_t, & j \neq t \\ \hat{y}_t(1 - \hat{y}_t), & j = t \end{cases}$

Backpropagating through time

$$\begin{aligned}\frac{\partial L}{\partial z_t} &= -\frac{y_t}{\hat{y}_t} \hat{y}_t (1 - \hat{y}_t) + \sum_{j \neq t}^T \frac{y_j}{\hat{y}_j} \hat{y}_j \hat{y}_t \\&= -\frac{y_t}{\hat{y}_t} \hat{y}_t (1 - \hat{y}_t) + \sum_{j \neq t}^T \frac{y_j}{\hat{y}_j} \hat{y}_j \hat{y}_t \\&= -y_t (1 - \hat{y}_t) + \sum_{j \neq t}^T y_j \hat{y}_t \\&= -y_t + \hat{y}_t y_t + \hat{y}_t \sum_{j \neq t}^T y_j \\&= -y_t + \hat{y}_t \sum_{j=1}^T y_j \\&\quad \text{say } \sum_{j=1}^T y_j = 1, \text{ then we can write} \\&= -y_t + \hat{y}_t \\&= \hat{y}_t - y_t\end{aligned}$$

Backpropagating through time

- Thus, the final equation becomes: $\frac{\delta L_j}{\delta z_t} = \hat{y}_t - y_t$
- Now, we can substitute equation
- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} (\hat{y}_j - y_j) \frac{\partial z_j}{\partial V}$
- $z_t = V h_t$
- $\frac{\partial z_t}{\partial V} = h_t$
- $\frac{\partial L}{\partial V} = \sum_{j=0}^{T-1} (\hat{y}_j - y_j) * h_t$

Gradients with respect to hidden to hidden layer weights, W

- $\frac{\partial L}{\partial W} = \frac{\partial L_0}{\partial W} + \frac{\partial L_1}{\partial W} + \frac{\partial L_2}{\partial W} \dots \dots \dots + \frac{\partial L_{T-1}}{\partial W}$
- $\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta W}$
- First, let's compute gradient L_0 of loss, with respect to W , ie $\frac{\partial L_0}{\partial W}$
- We cannot compute derivative of L_0 with respect to W directly from as there are no terms in it.
- So, we use the chain rule to compute the gradients of loss with respect to .

Gradients with respect to hidden to hidden layer weights, W

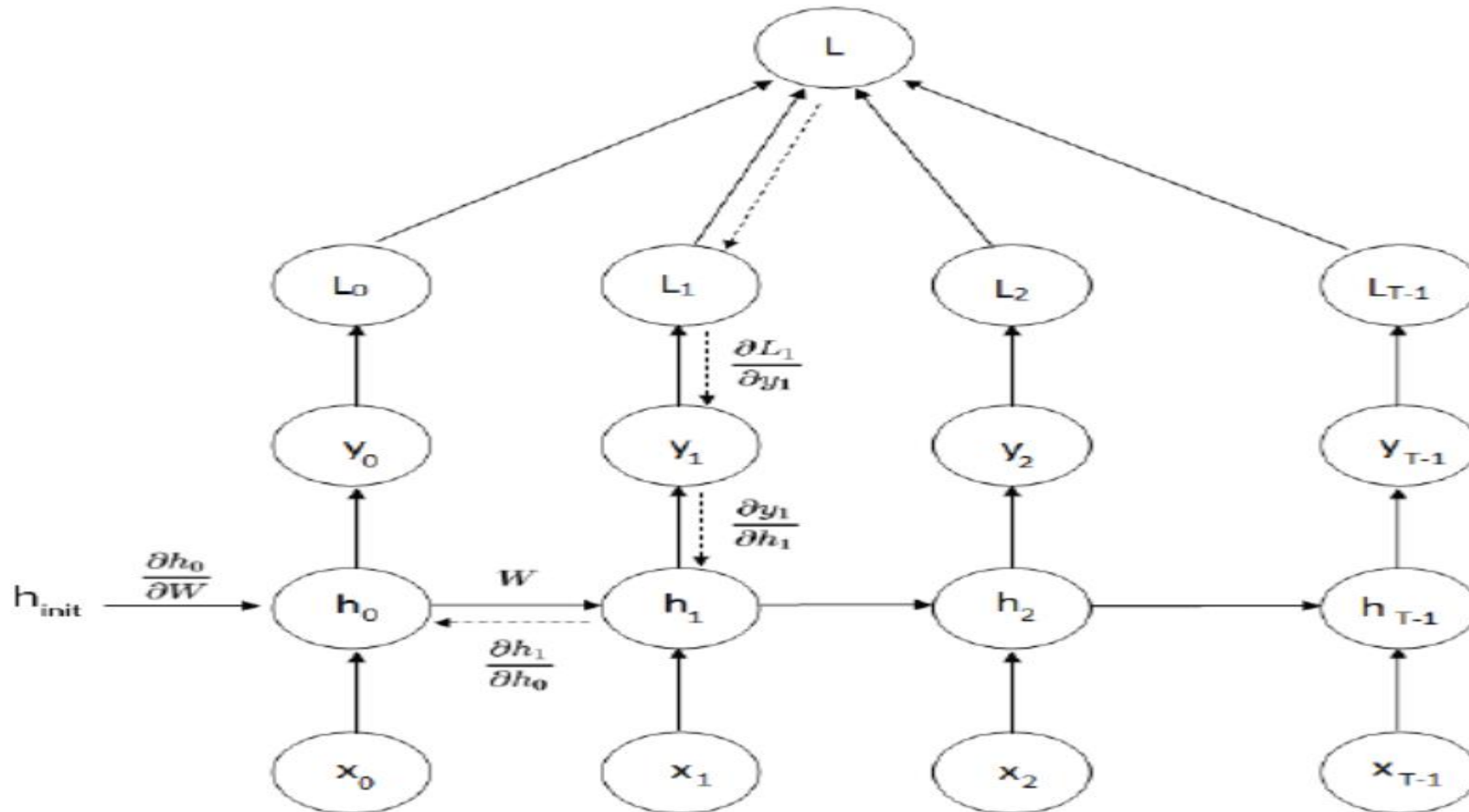
- $L_0 = -y_0 \log(\hat{y}_0)$
- $y_0 = \text{Softmax}(h_0 V)$
- $h_0 = \tanh(Ux_0 + Wh_{init})$
- First, we calculate the partial derivative of loss L_0 with respect to y_0 ;
- then, from y_0 , we calculate the partial derivative with respect to h_0 ; then, from h_0 , we can calculate the derivative with respect to W , as follows
- $$\frac{\partial L_0}{\partial W} = \frac{\partial L_0}{\partial y_0} \frac{\partial y_0}{\partial h_0} \frac{\partial h_0}{\partial W}$$
- Similarly
$$\frac{\partial L_1}{\partial W} = \frac{\partial L_1}{\partial y_1} \frac{\partial y_1}{\partial h_1} \frac{\partial h_1}{\partial W}$$

Gradients with respect to hidden to hidden layer weights, W

- Since $h_1 = \tanh(Ux_1 + Wh_0)$
- As you can see in the preceding equation, computing h_1 depends on h_0 and W
- , but h_0 is not a constant; it is a function again. So, we need to calculate the derivative with respect to that, as well.
- $$\frac{\partial L_1}{\partial W} = \frac{\partial L_1}{\partial y_1} \frac{\partial y_1}{\partial h_1} \frac{\partial h_1}{\partial W} + \frac{\partial L_1}{\partial y_1} \frac{\partial y_1}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W}$$
- **we can notice how h_1 is dependent on h_0 :**

Gradients w.r.t hidden to hidden layer weights, W

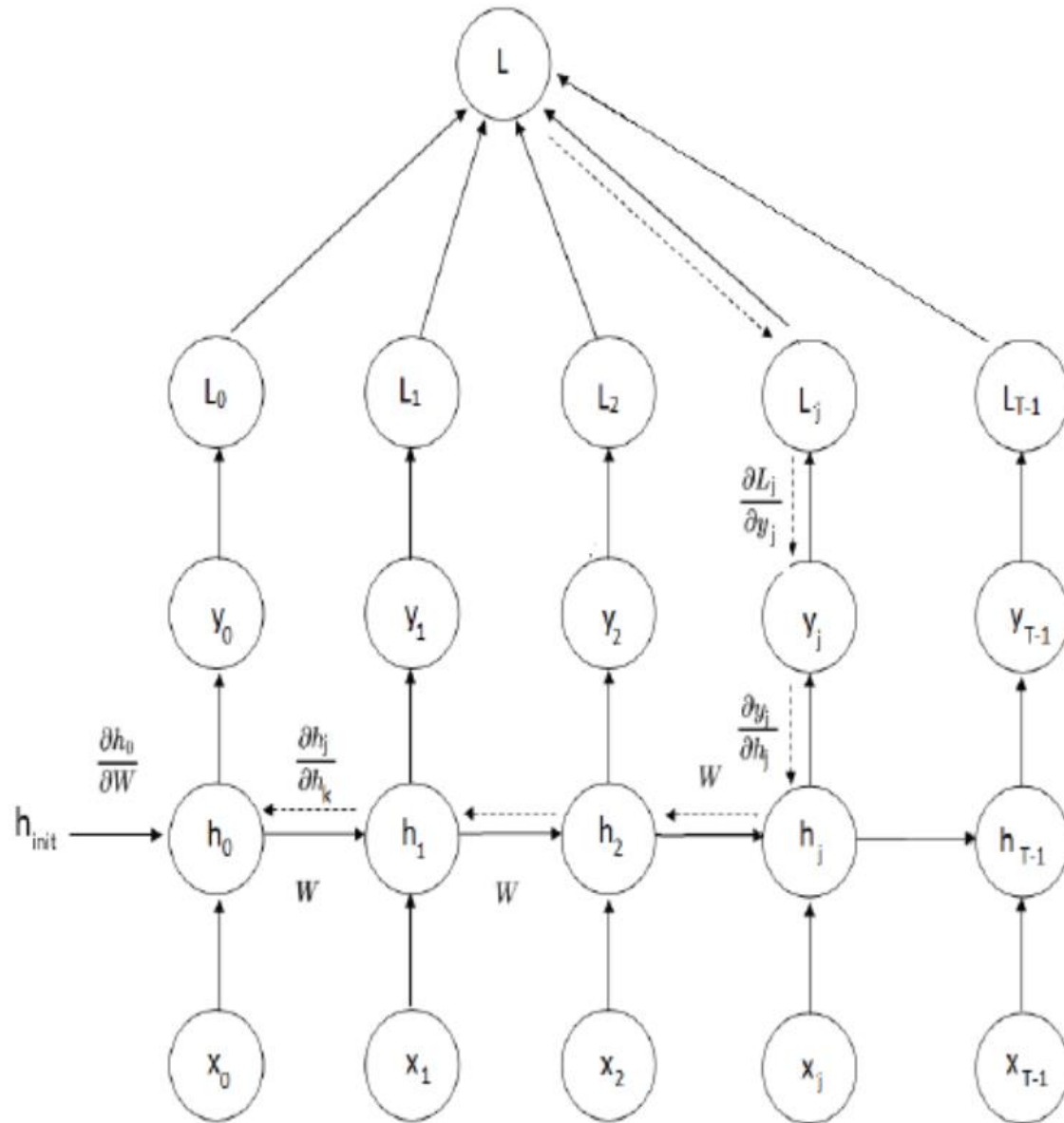
- $\frac{\partial L_1}{\partial W} = \frac{\partial L_1}{\partial y_1} \frac{\partial y_1}{\partial h_1} \frac{\partial h_1}{\partial W} + \frac{\partial L_1}{\partial y_1} \frac{\partial y_1}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W}$
- we can notice how h_1 is dependent on h_0 :



Gradients with respect to hidden to hidden layer weights, W

- *Now compute the gradient of loss, L_2 with respect to W*
- $$\frac{\partial L_2}{\partial W} = \frac{\partial L_2}{\partial y_2} \frac{\partial y_2}{\partial h_2} \frac{\partial h_2}{\partial W}$$
- Since $h_2 = \tanh(Ux_2 + Wh_1)$
- **we can notice how h_2 is dependent on h_1 , where h_1 is again function of h_0 :**
- $$\frac{\partial L_2}{\partial W} = \frac{\partial L_2}{\partial y_2} \frac{\partial y_2}{\partial h_2} \frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial y_2} \frac{\partial y_2}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial y_2} \frac{\partial y_2}{\partial h_2} \frac{\partial h_2}{\partial h_0} \frac{\partial h_0}{\partial W}$$
- This is because in RNNs, the hidden state at a time t is dependent on a
- hidden state at a time $t-1$, which implies that the current hidden state is
- always dependent on the previous hidden state

Gradients wrt hidden to hidden layer weights, W



Thus, we can write, gradient of loss L_j with respect to w becomes:

$$\frac{\partial L_j}{\partial W} = \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \frac{\partial h_j}{\partial h_k} \frac{\partial h_k}{\partial W}$$

$\sum_{k=0}^j$ implies the sum over all the hidden state h_k

Gradients wrt hidden to hidden layer weights, W

$$\frac{\partial L_j}{\partial W} = \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \frac{\partial h_j}{\partial h_k} \frac{\partial h_k}{\partial W}$$

$$\frac{\partial h_j}{\partial h_k} = \prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}}$$

Assume that $j=3$ and $k=0$; then, the preceding equation becomes:

$$\frac{\partial h_3}{\partial h_0} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0}$$

$$\frac{\partial L_j}{\partial W} = \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial W}$$

Gradients wrt hidden to hidden layer weights, W

$$\frac{\partial L_j}{\partial W} = \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial W}$$

final loss is the sum of loss across all the time steps:

$$\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \frac{\delta L_j}{\delta W}$$

$$\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial W}$$

- $\sum_{j=0}^{T-1}$ implies the sum of loss across all the time steps

$\sum_{k=0}^j$ is the summation over hidden states

final equation for computing gradient of loss with respect to W , is given as:

$$\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial W}$$

$$\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \sum_{k=0}^j \frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial W}$$

$$\frac{\partial L_j}{\partial y_j} \frac{\partial y_j}{\partial h_j} = \hat{y}_j - y_j$$

$$\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}}$$

$$h_m = \tanh(Ux_m + Wh_{m-1})$$

The derivative of $\tanh(x)$ is $1 - \tanh^2(x)$, so we can write:

$$\frac{\partial h_m}{\partial h_{m-1}} = W^T \text{diag}(1 - \tanh^2(Wh_{m-1} + Ux_m))$$

Gradients wrt hidden to hidden layer weights, W

- $\frac{\partial h_k}{\partial W} = h_k - 1$

$$\frac{\partial L}{\partial W} = \sum_{j=0}^{T-1} \sum_{k=0}^j (\hat{y}_j - y_j) \prod_{m=k+1}^j W^T \text{diag}(1 - \tanh^2(W h_{m-1} + U x_m)) \otimes h_{k-1}$$

Gradients with respect to input to the hidden layer weight, U

$$\frac{\partial L}{\partial U} = \sum_{j=0}^{T-1} \sum_{k=0}^j \frac{\partial L_j}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial h_j} \left(\prod_{m=k+1}^j \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial U}$$

$$h_k = \tanh(Ux_k + Wh_{k-1}).$$

$$\frac{\partial h_k}{\partial U} = x_k$$

$$\boxed{\frac{\partial L}{\partial U} = \sum_{j=0}^{T-1} \sum_{k=0}^j (\hat{y}_j - y_j) \prod_{m=k+1}^j W^T \text{diag}(1 - \tanh^2(Wh_{m-1} + Ux_m)) \otimes x_k}$$

Vanishing and exploding gradients problem

- While computing the derivatives of loss with respect to W and U , we saw that we have to traverse all the way back to the first hidden state, as each hidden state at a time t is dependent on its previous hidden state at a time $t-1$

For instance, the gradient of loss L_2 with respect to W is given as:

$$\frac{\partial L_2}{\partial W} = \frac{\partial L_2}{\partial y_2} \frac{\partial y_2}{\partial h_2} \frac{\partial h_2}{\partial W}$$

we can't calculate the derivative of h_2 with respect to w directly $(\frac{\partial h_2}{\partial W})$

$$h_2 = \tanh(Ux_2 + Wh_1)$$

is a function that is dependent on h_1 and W .

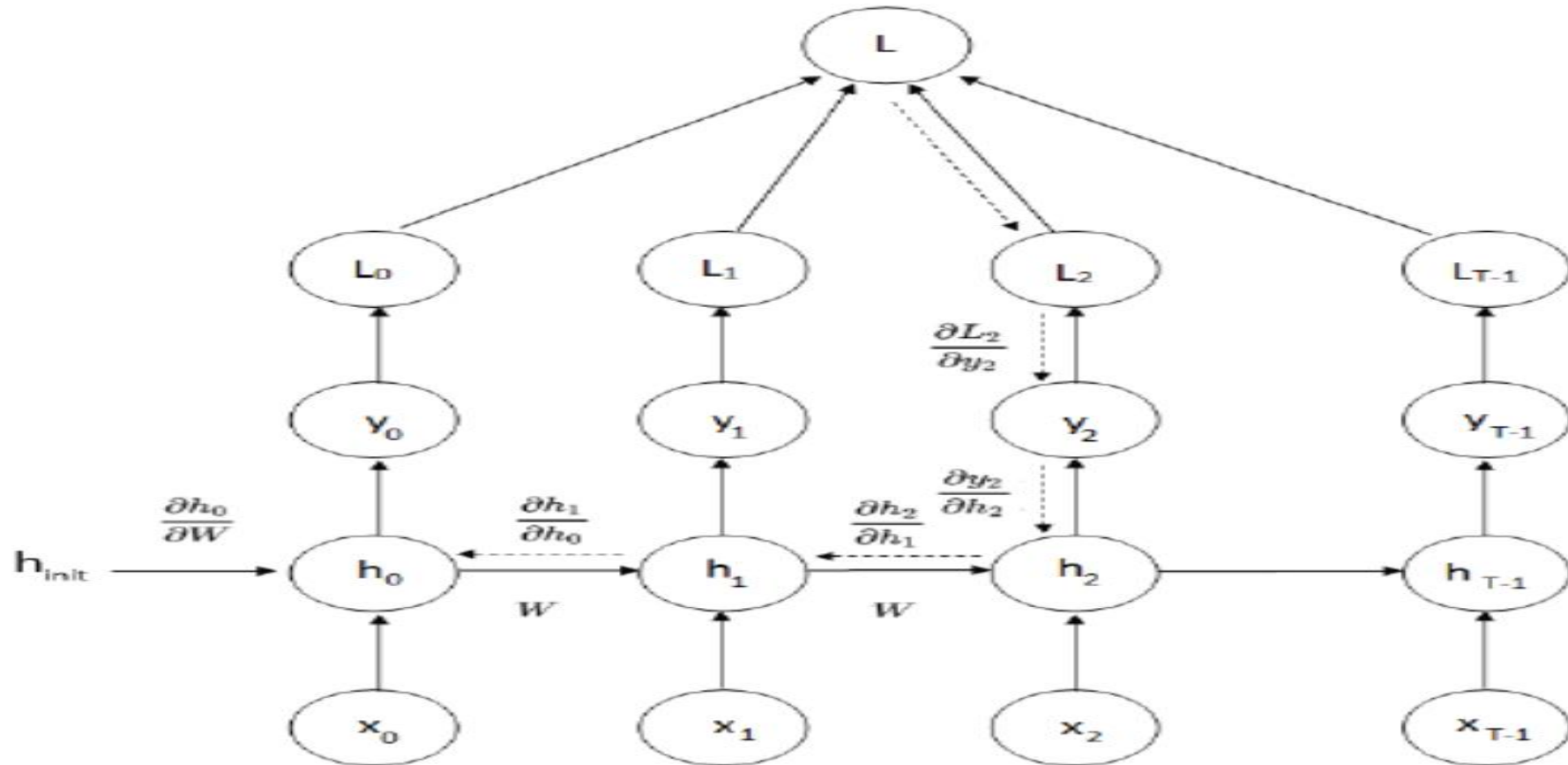
So, we need to calculate the derivative with respect to h_1 ,

$h_1 = \tanh(Ux_1 + Wh_0)$ is a function that is dependent on h_0 and W .

Thus, we need to calculate the derivative with respect to , as well

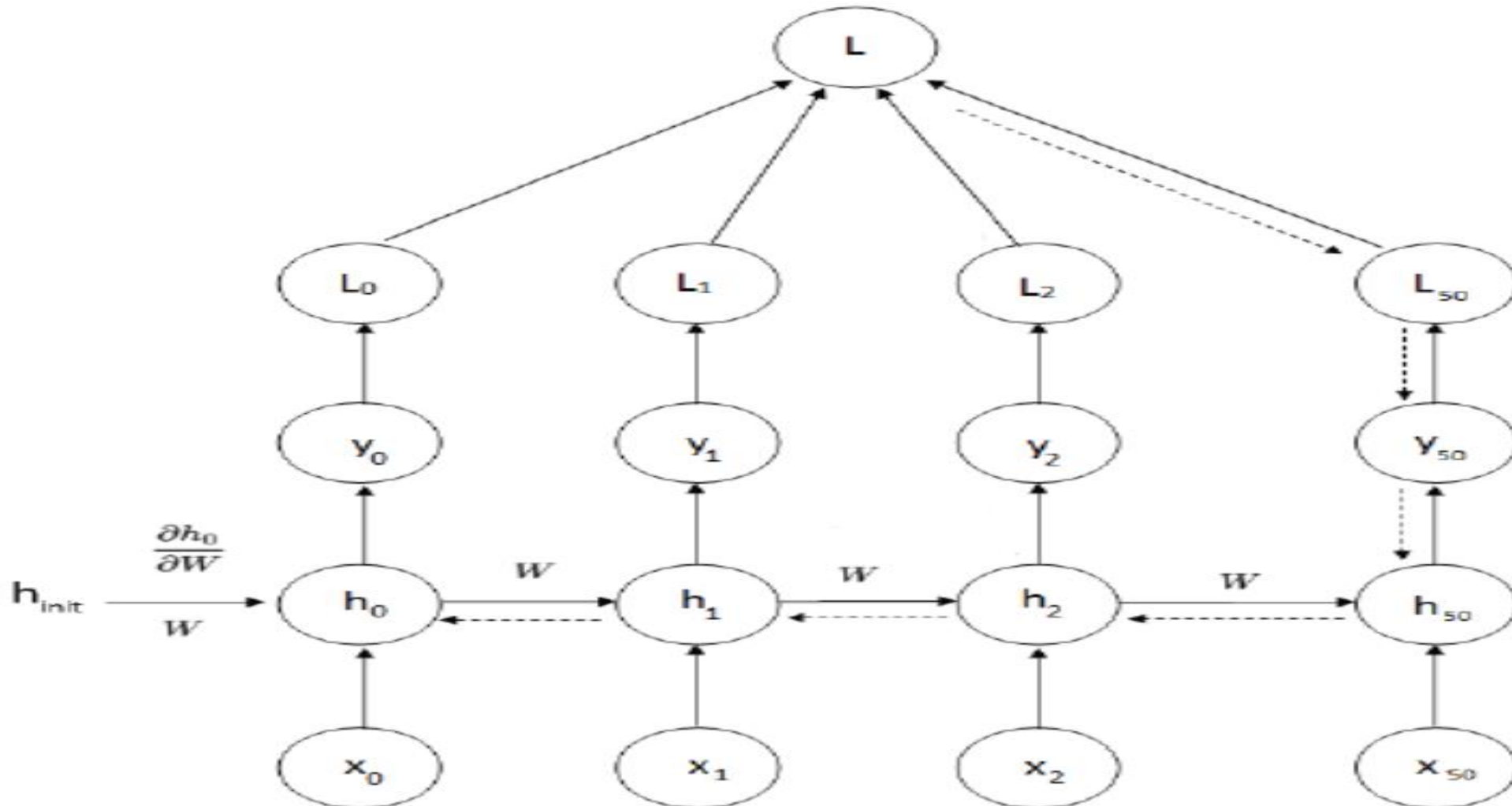
Vanishing and exploding gradients problem

- to compute the derivative of L_2 , we need to go all the way back to the initial hidden state, as each hidden state h_0 is dependent on its previous hidden state:



Vanishing and exploding gradients problem

- To compute the loss L_{50} , we need to traverse all the way back to h_0 ,



Vanishing and exploding gradients problem

- While back propagating towards the initial hidden state, we lose information, and the RNN will not backpropagate perfectly.
- $\mathbf{h}_t = \tanh(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1})$
- Every time we move backward, we compute the derivative of $\mathbf{h}_t$.
- **A derivative of tanh is bounded to 1.**
- **We know that any two values between 0 and 1, when multiplied with each other will give us a smaller number.**

Vanishing and exploding gradients problem

- We usually initialize the weights of the network to a small number. Thus, when **we multiply the derivatives And weights** while backpropagating, we are essentially multiplying smaller numbers.
- So, when we multiply smaller numbers at every step while moving
- backward, our gradient becomes infinitesimally small and leads to a number that the computer can't handle; this is called the **vanishing gradient problem**.

Vanishing and exploding gradients problem

equation of gradient of the loss with respect to the W that we saw in the *Gradients with respect to hidden to hidden layer weights, W* section

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{j=0}^{T-1} \sum_{b=0}^j (\hat{y}_j - y_j) \prod_{m=k+1}^j \underbrace{(W^T)}_{\text{Weights}} \underbrace{\text{diag}(1 - \tanh^2(W h_{m-1} + U x_m))}_{\text{Activation function derivative}} \otimes h_{k-1}$$

As you can observe, we are multiplying the weights and derivative of the tanh function at every time step.

Repeated multiplication of these two leads to a small number and causes the vanishing gradients problem.

Vanishing and exploding gradients problem

- The vanishing gradients problem occurs not only in RNN but also in other deep networks where **we use sigmoid or tanh as the activation function**.
- So, to overcome this, **we can use ReLU as an activation function instead of tanh**.
- However, we have a variant of the RNN called the **long short-term memory (LSTM)** network, which can solve the vanishing gradient problem effectively.

Vanishing and exploding gradients problem

- Similarly, when we initialize the weights of the network to a very large number, the gradients will become very large at every step.
- While backpropagating, we multiply a large number together at every time step, and it leads to infinity.
- This is called the **exploding gradient problem**.

Long Short Term Memory Networks (LSTM)

- The drawback of a **recurrent neural network (RNN)** is that it will not retain information for a long time in memory.
- We know that an RNN stores sequences of information in its hidden state but when the input sequence is too long,
- it cannot retain all the information in its memory due to the **vanishing gradient problem**
- **long short-term memory (LSTM)** cell, which resolves the vanishing gradient problem by using a special structure called a **gate**.
- Gates keep the information in memory as long as it is required.
- They learn what information to keep and what information to discard from the memory.

Long Short Term Memory Networks (LSTM)

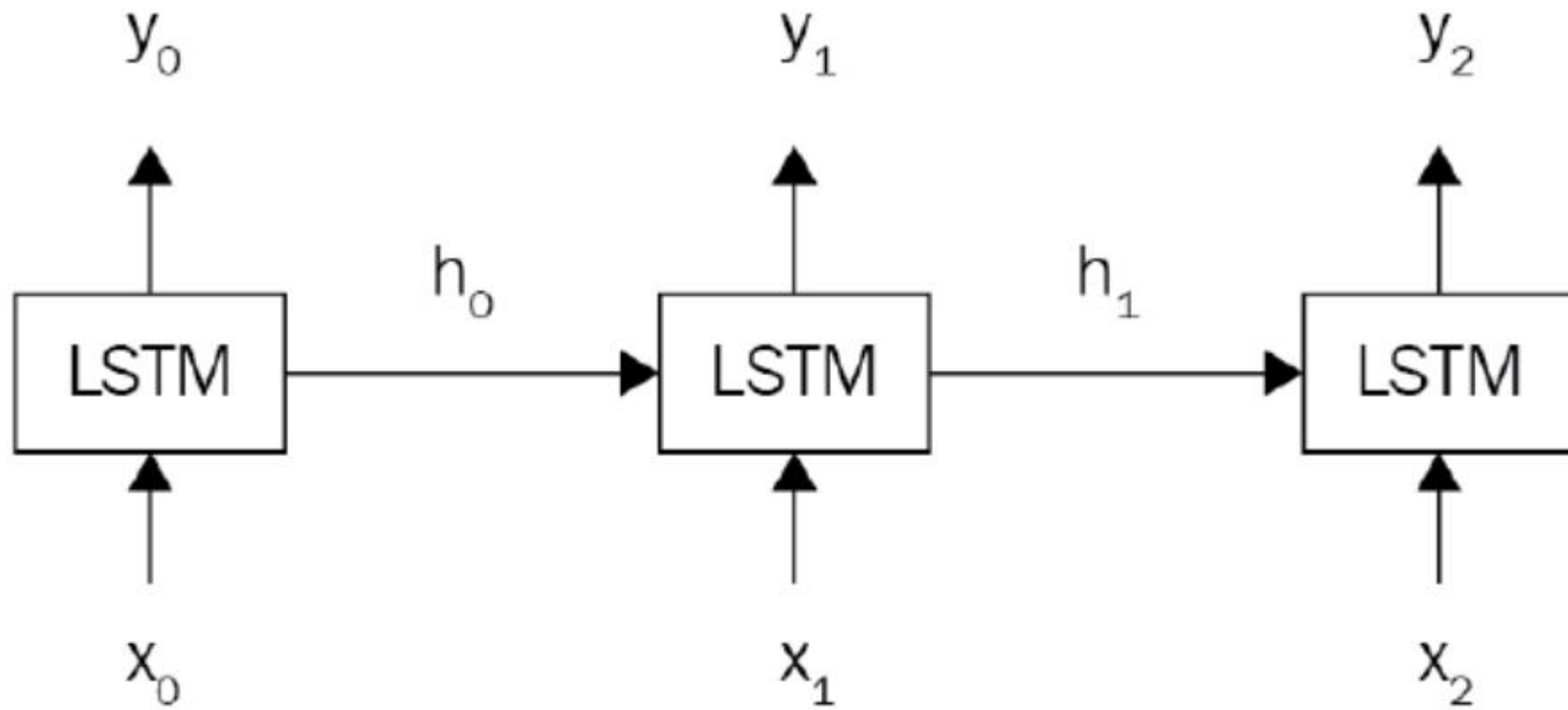
- Let's consider the following sentence to understand the problem better:
- ***Archie lived in China for 13 years. He loves listening to good music. He is a fan of comics. He is fluent in ____.***
- Now, if we were asked to predict the missing word in the preceding sentence, we would predict it as **Chinese**, but how did we predict that?
- We simply remembered the previous sentences and understood that Archie lived for 13 years in China.
- This led us to the conclusion that Archie might be fluent in Chinese.

Long Short Term Memory Networks (LSTM)

- An RNN, on the other hand, cannot retain all of this information in its memory to say that Archie is fluent in Chinese.
- **Due to the vanishing gradient problem**, it cannot recollect/remember information for a long time in its memory.
- **That is, when the input sequence is long, the RNN memory (hidden state) cannot hold all the information.**
- **To alleviate this, we use an LSTM cell.**
- LSTM is a variant of an RNN that resolves the vanishing gradient problem and retains information in the memory as long as it is required.

Basically, **RNN cells are replaced with LSTM cells in the hidden units**

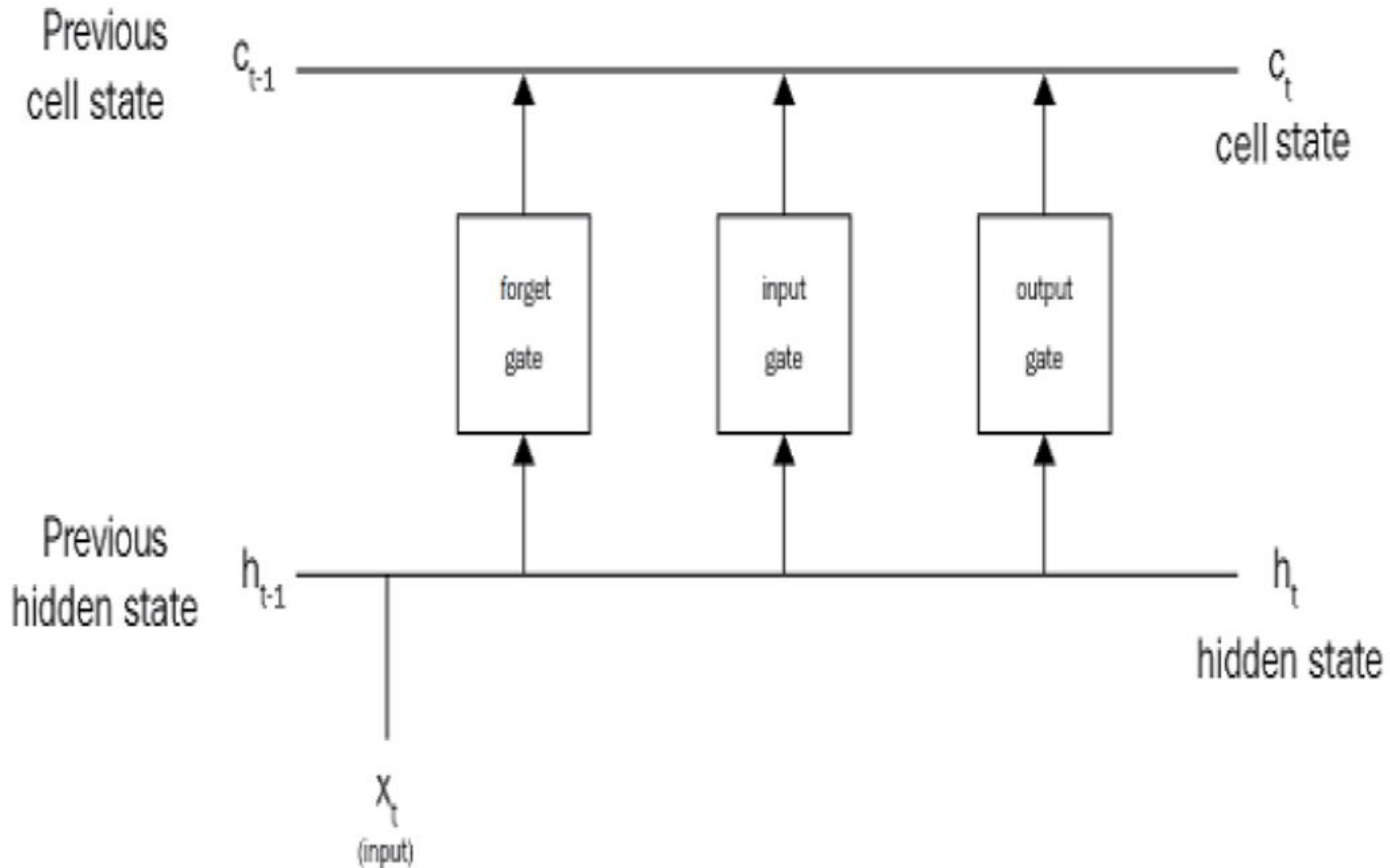
Long Short Term Memory Networks (LSTM)



Long Short Term Memory Networks (LSTM)

- How do LSTM cells achieve long-term dependency?
- How does it know what information to keep and what information to discard from the memory?
- This is all achieved by special structures called **gates**
- LSTM cell consists of three special gates called
- **the input gate, output gate, and forget gate:**

Long Short Term Memory Networks (LSTM)



Long Short Term Memory Networks (LSTM)

- These three gates are responsible for deciding what information to add, output, and forget from the memory.
- With these gates, an LSTM cell effectively keeps information in the memory only as long as required.
- In an RNN cell, we used the hidden state, h_t , for two purposes:
- one for storing the information and the other for making predictions.
- Unlike RNN, in the LSTM cell we break the hidden states into two states, called the **cell state** and the **hidden state**:
- **cell state is called internal memory and is where all information will be stored**
- **The hidden state is used for computing the output, for making predictions**

Long Short Term Memory Networks (LSTM)

- **Forget gate** : The forget gate, f_t , is responsible for deciding what information should be removed from the cell state (memory).
- ***Harry is a good singer. He lives in New York. Zayn is also a good singer.***
- As soon as we start talking about Zayn, the network will understand that the subject has been changed from Harry to Zayn, and the information about Harry is no longer required.
- Now, the forget gate will remove/forget information about Harry from the cell state.

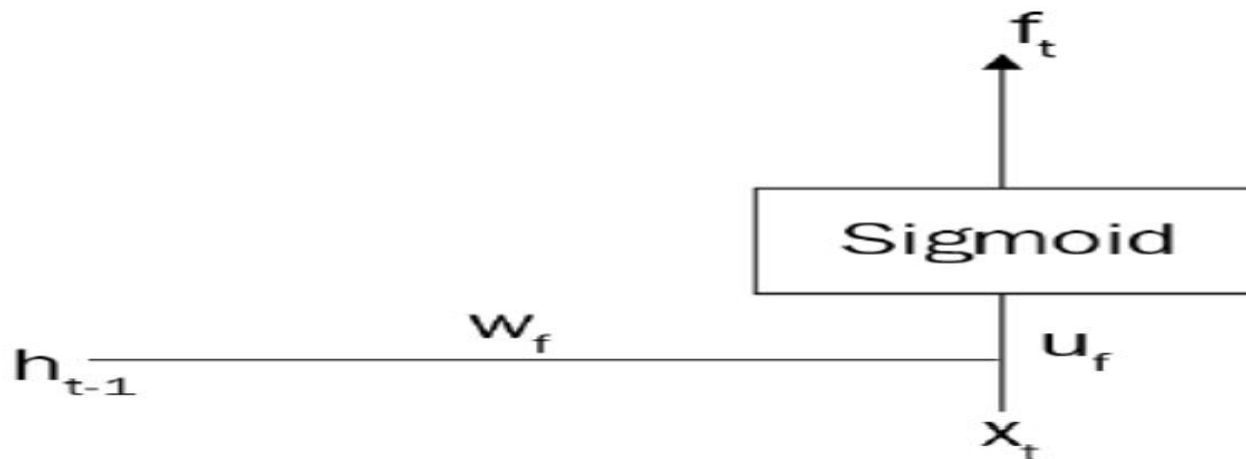
Long Short Term Memory Networks (LSTM)

- **Forget gate :**
- The forget gate is controlled by a sigmoid function.
- At time step , we pass input x_t , and the previous hidden state, h_{t-1} , to the forget gate.
- It return 0 if the particular information from the cell state should be removed
- and returns 1 if the information should not be removed.
- The forget gate, f , at a time step, t , is expressed as follows:

$$f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$$

Long Short Term Memory Networks (LSTM)

- **Forget gate :** $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
 - U_f is the input-to-hidden layer weights of the forget gate
 - W_f is the hidden-to-hidden layer weights of the forget gate
 - b_f is the bias of the forget gate



Long Short Term Memory Networks (LSTM)

- **Input gate** : The input gate is responsible for deciding what information should be stored in the cell state.
- *Harry is a good singer. He lives in New York. Zayn is also a good singer.*
- After the forget gate removes information from the cell state, the input gate decides what information it has to keep in the memory.
- since information about Harry is removed from cell state by **forget gate**,
- input gate decides to update the cell state with information **about Zayn**.

Long Short Term Memory Networks (LSTM)

Input Gate.

$$i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$$

- Similar to the forget gate, the input gate is controlled by a sigmoid function
- that returns output in the range of 0 to 1.
- If it returns 1, then the particular information will be stored/updated to the cell state,
- and if it returns 0, we will not store the information to the cell state.

U_i is the input-to-hidden weights of the input gate

W_i is the hidden-to-hidden weights of the input gate

b_i is the bias of the input gate

Long Short Term Memory Networks (LSTM)

Input Gate.

$$i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$$



LSTM

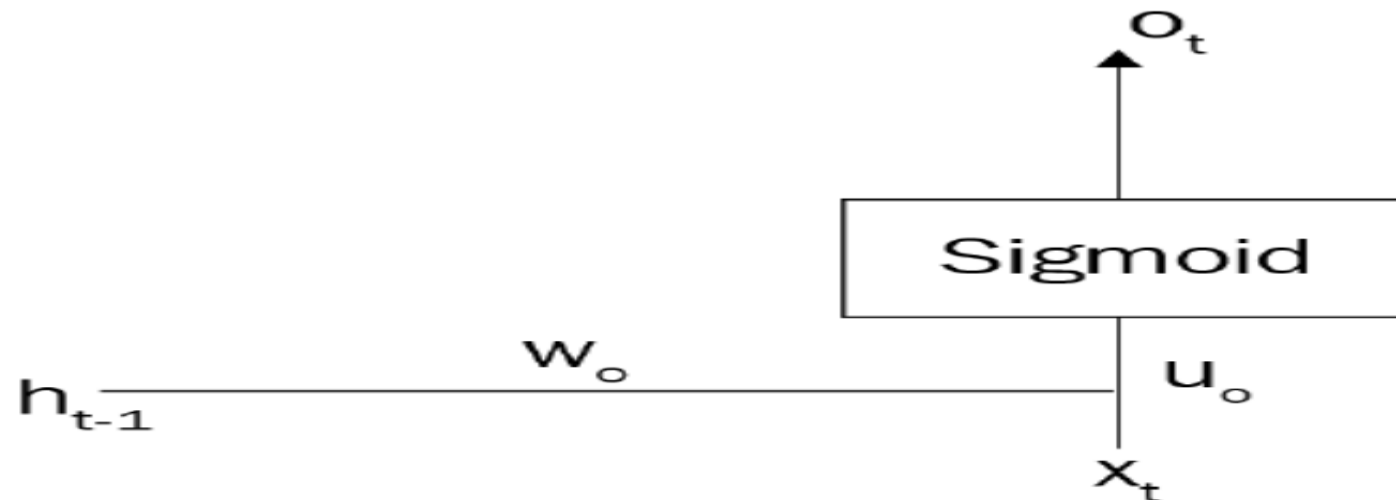
- **Output gate :**
- We will have a lot of information in the cell state (memory).
- The output gate is responsible for deciding what information should be taken from the cell state to give as an output.
- *Zayn's debut album was a huge success. Congrats _____.*
- The output gate will look up all the information in the cell state and select the correct information to fill the blank.
- Here, congrats is an adjective that is used to describe a noun.
- So, the output gate will predict **Zayn** (noun) to fill the blank.

LSTM

- **Output gate :**

$$o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$$

- U_o is the input-to-hidden weights of the output gate
- W_o is the hidden-to-hidden weights of the output gate
- b_o is the bias of the output gate



LSTM

- **Updating the cell state**
- First, we will see how to add new relevant information to the cell state.
- To hold all the new information that can be added to the cell state (memory), we create a new vector called gt . It is called **candidate state** or **internal state vector**.
- Unlike gates that are regulated by the sigmoid function, the
- candidate state is regulated by the tanh function, but why?
- The sigmoid function returns values in the range of 0 to 1, that is, it is always positive.
- We need to allow the values of gt to be either positive or negative.
- So, we use the **tanh function**, which returns values in the range of -1 to +1.

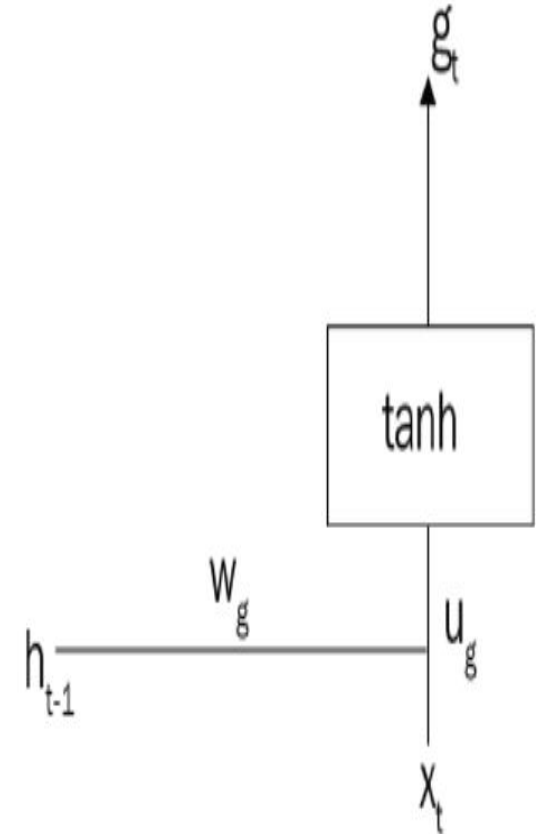
LSTM Updating the cell state

The candidate state, g , at time t is expressed as follows:

$$g_t = \tanh(U_g x_t + W_g h_{t-1} + b_g)$$

Here, the following applies:

- U_g is the input-to-hidden weights of the candidate state
- W_g is the hidden-to-hidden weights of the candidate state
- b_g is the bias of the candidate state



LSTM Updating the cell state

- How do we decide whether the information in the candidate state is relevant?
- How do we decide whether to add new information or not from the candidate state to the cell state?

We learned that the input gate is responsible for deciding whether to add new information or not,

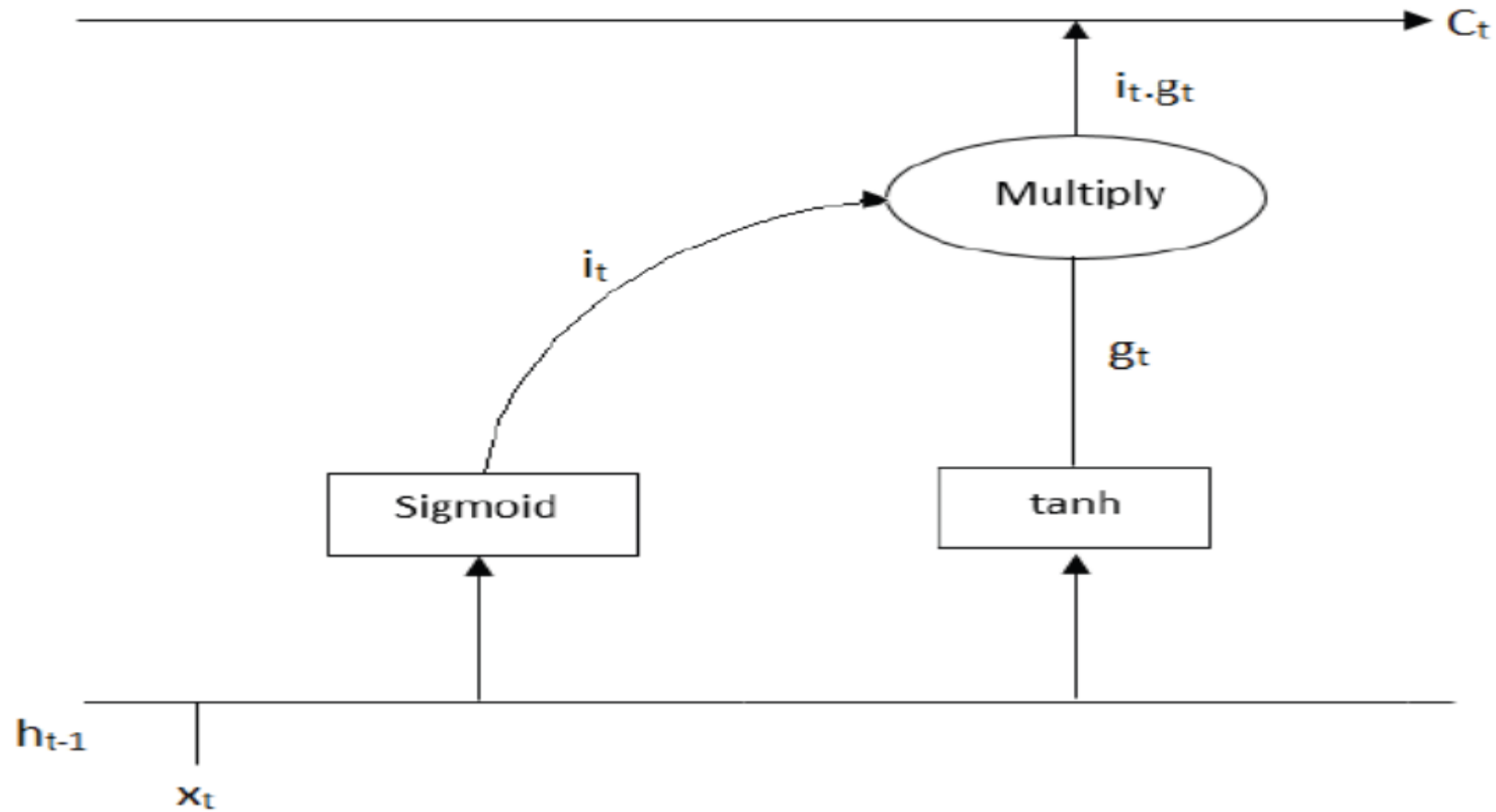
so if we multiply g_t and i_t , we only get relevant information that should be added to the memory.

- That is, we know the input gate returns 0 if the information is not required and 1 if the information is required

LSTM Updating the cell state

- Say , $i_t = 0$ then multiplying g_t and i_t gives us 0,
- which means the information in g_t is not required and we don't
- want to update the cell state with .
- When $i_t = 1$, then multiplying g_t and i_t gives us ,
- which implies we can update the information in g_t to the cell state.

LSTM Updating the cell state

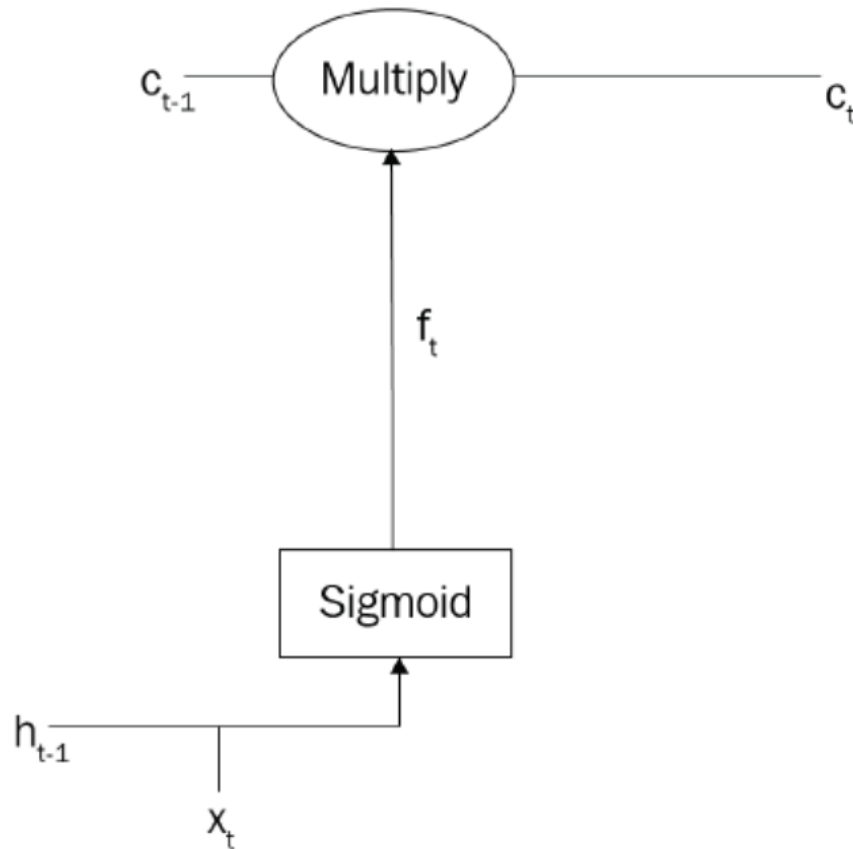


Adding the new information to the cell state with input gate i_t and candidate state g_t is shown in the following diagram

LSTM Updating the cell state

- how to remove information from the previous cell state that is no longer required.
- We learned that the forget gate is used for removing information that is not required in the cell state.
- So, if we multiply the previous cell state, c_{t-1} , and forget gate, f_t , then we retain only relevant information in the cell state.
- **Say , $f_t=0$ then multiplying c_{t-1} and f_t gives us 0**, which means the information in cell state, c_{t-1} , is not required and should be removed (**forgotten**).
- **When $f_t=1$, then multiplying c_{t-1} and f_t gives c_{t-1}** , which implies that information in the previous cell state is required and should not be removed.

LSTM Updating the cell state



can express the cell state equation as follows:

$$c_t = f_t c_{t-1} + i_t g_t$$

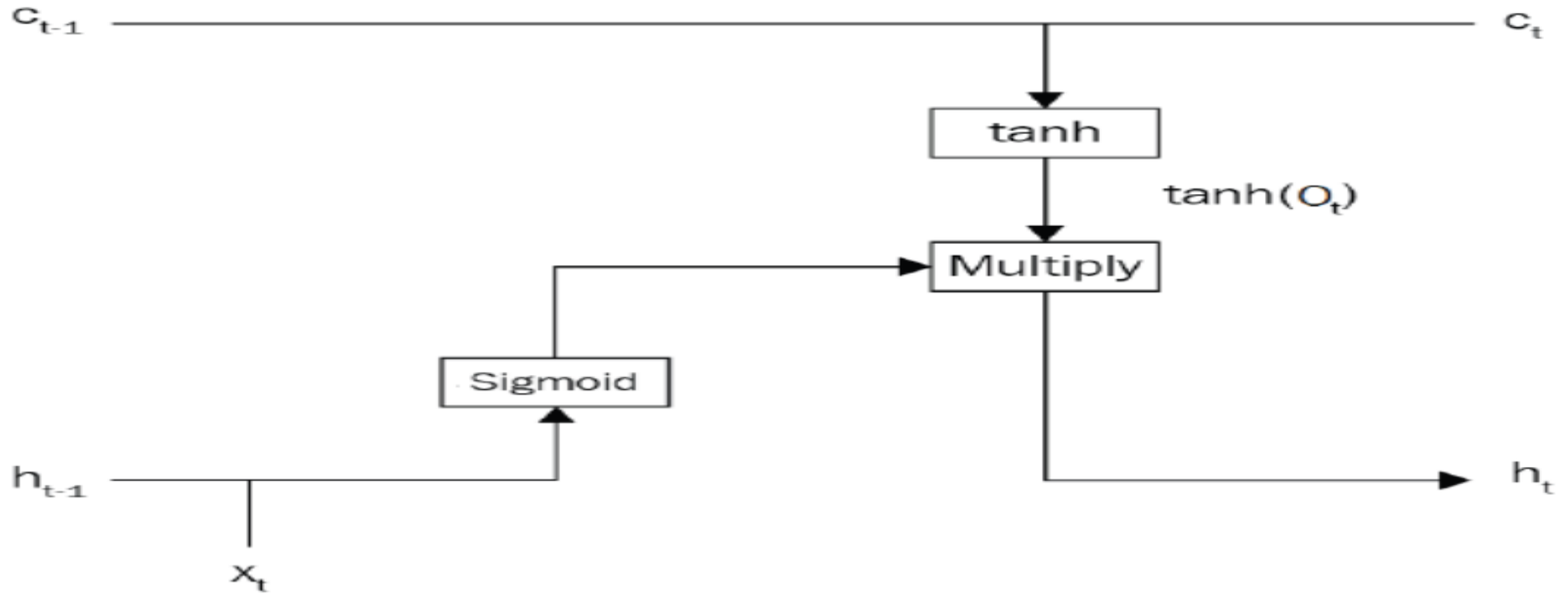
Removing information from the previous cell state, c_{t-1} , with the forget gate, f_t , is shown in the diagram:

LSTM -Updating hidden state

- We learned that the hidden state, h_t , is used for computing the output, but how can we compute the output?
- We know that the output gate is responsible for deciding what information should be taken from the cell state to give as output. Thus, multiplying o_t and tanh (to squash between -1 and +1) of cell state, $\tanh(c_t)$, gives us the output.
- $h_t = o_t \tanh(c_t)$

LSTM -Updating hidden state

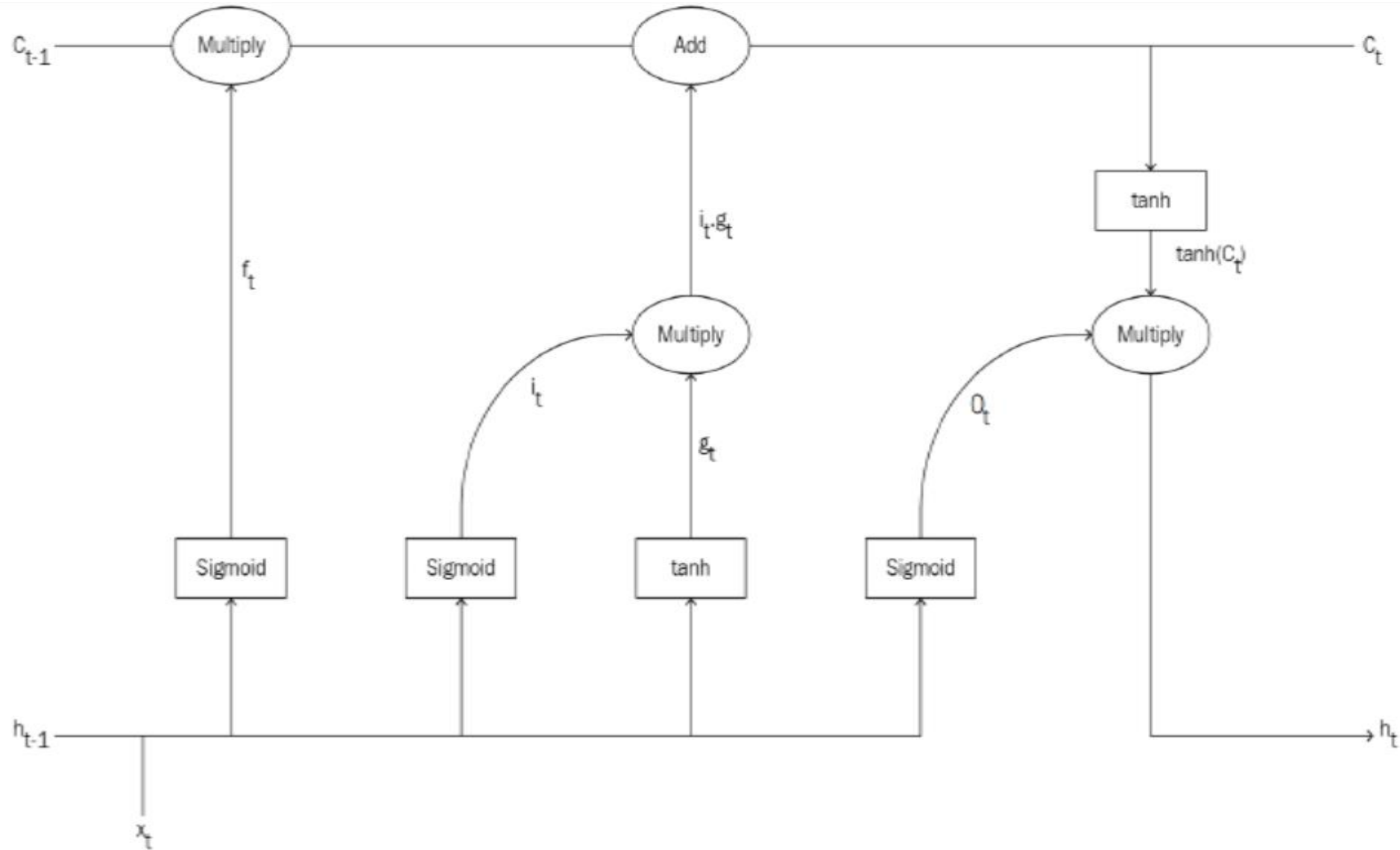
- $h_t = o_t \tanh(c_t)$



LSTM -Updating hidden state

- $h_t = o_t \tanh(c_t)$
- Finally, once we have the hidden state value, we can apply the softmax function and compute as follows:
- $\hat{y}_t = \text{Softmax}(V h_t)$
- Here, V is the hidden-to-output layer weights.

Forward propagation in LSTM



Forward propagation in LSTM

1. **Input gate:** $i_t = \sigma(U_i x_t + W_f h_{t-1} + b_i)$
2. **Forget gate:** $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
3. **Output gate:** $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$
4. **Candidate state:** $g_t = \tanh(U_g x_t + W_g h_{t-1} + b_g)$
5. **Cell state:** $c_t = f_t c_{t-1} + i_t g_t$
6. **Hidden state:** $h_t = o_t \tanh(c_t)$
7. **Output:** $\hat{y}_t = \text{softmax}(V h_t)$

Long Short Term Memory

- **Applications of LSTM**
- machine translation, and text summarization, grammatically correct sentences by learning the dependencies between words in a sentence.
- **Speech Recognition:** transcribing speech to text and recognizing spoken commands.
- They can be trained to recognize patterns in speech and match them to the corresponding text.
- **Time Series Forecasting:** predicting stock prices, weather, and energy consumption

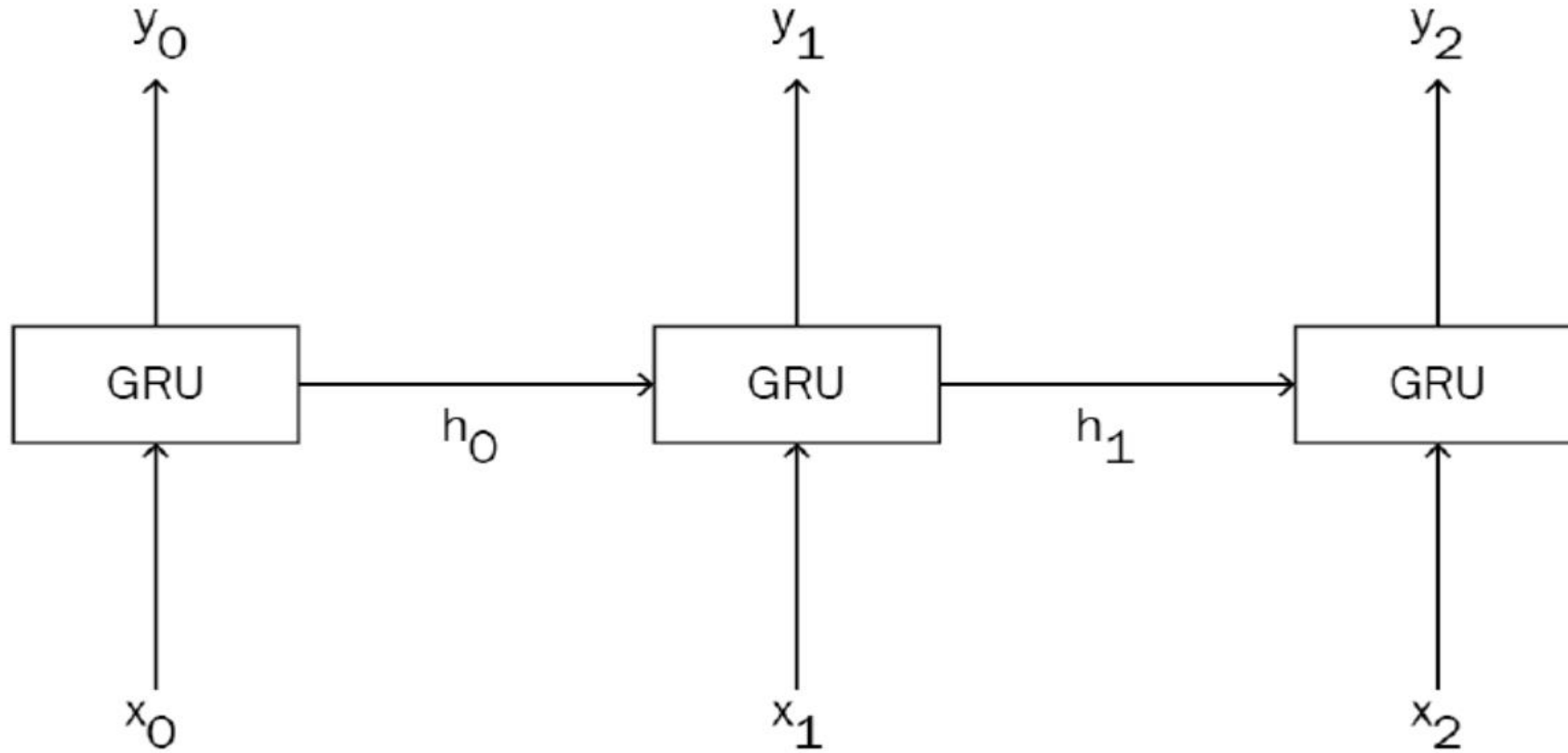
Long Short Term Memory

- **Anomaly Detection:**
 - detecting fraud and network intrusion.
- **Recommender Systems:** recommending movies, music, and books.
- **Video Analysis:** object detection, activity recognition, and action classification.
- They can be used in combination with other neural network architectures, such as Convolutional Neural Networks (CNNs),
 - to analyze video data and extract useful information.

Gated recurrent units

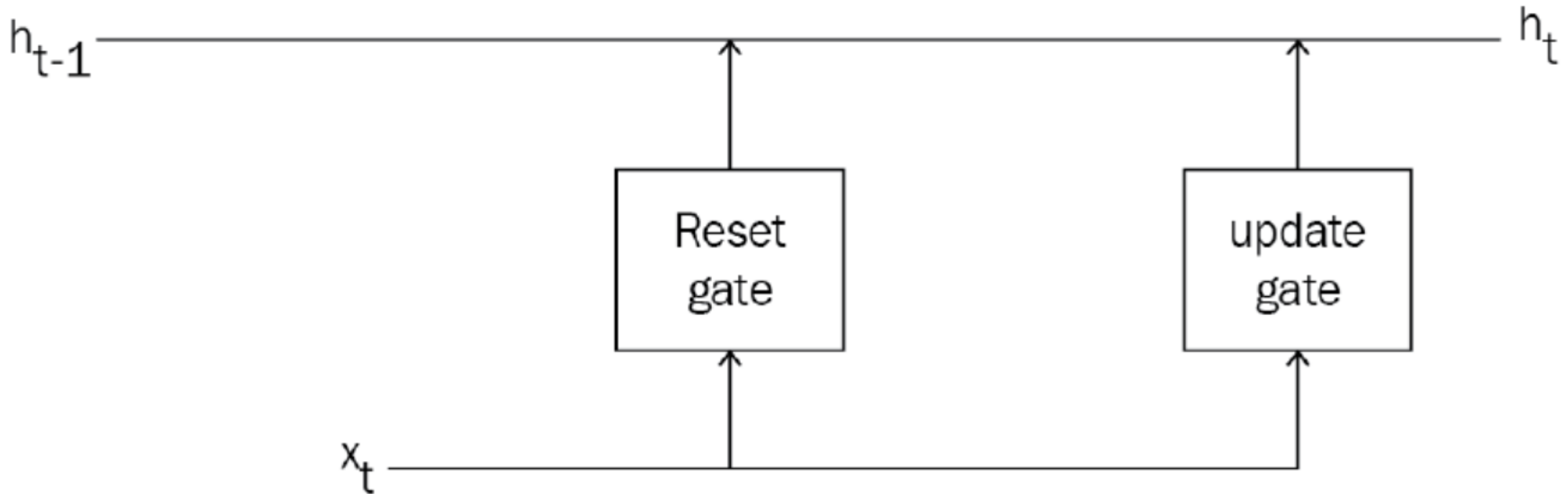
- So far, we have learned about how the LSTM cell uses different gates and how it solves the vanishing gradient problem of the RNN.
- But, as you may have noticed, the LSTM cell has too many parameters due to the presence of many gates and states.
- Thus, while backpropagating the LSTM network, we need to update a lot of parameters in every iteration.
- This increases our training time. So, we introduce the **Gated Recurrent Units (GRU)** cell, which acts as a simplified version of the LSTM cell.
- Unlike the LSTM cell, the GRU cell has only two gates and one hidden state

Gated recurrent units



Gated recurrent units

- a GRU cell has only two gates, called the update gate and the reset gate, and one hidden state.



Update gate

- The update gate helps to decide what information from the previous time step, h_{t-1} , can be taken forward to the next time step, h_t .
- It is basically a combination of an input gate and a forget gate, which we learned about in LSTM cells.
- input is x_t , multiplied with U_z , and the previous hidden state, h_{t-1}
- , 0 and 1:

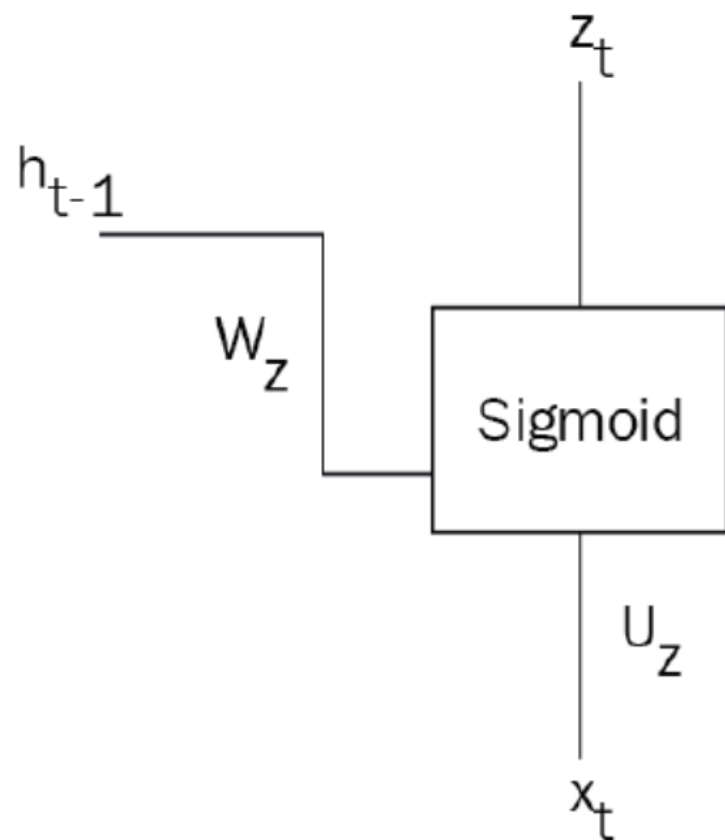
Update gate

The update gate, z_t , at time step t is expressed as follows:

$$z_t = \sigma(U_z x_t + W_z h_{t-1} + b_z)$$

Here, the following applies:

- U_z is the input-to-hidden weights of the update gate
- W_z is the hidden-to-hidden weights of the update gate
- b_z is the bias of the update gate



Reset gate

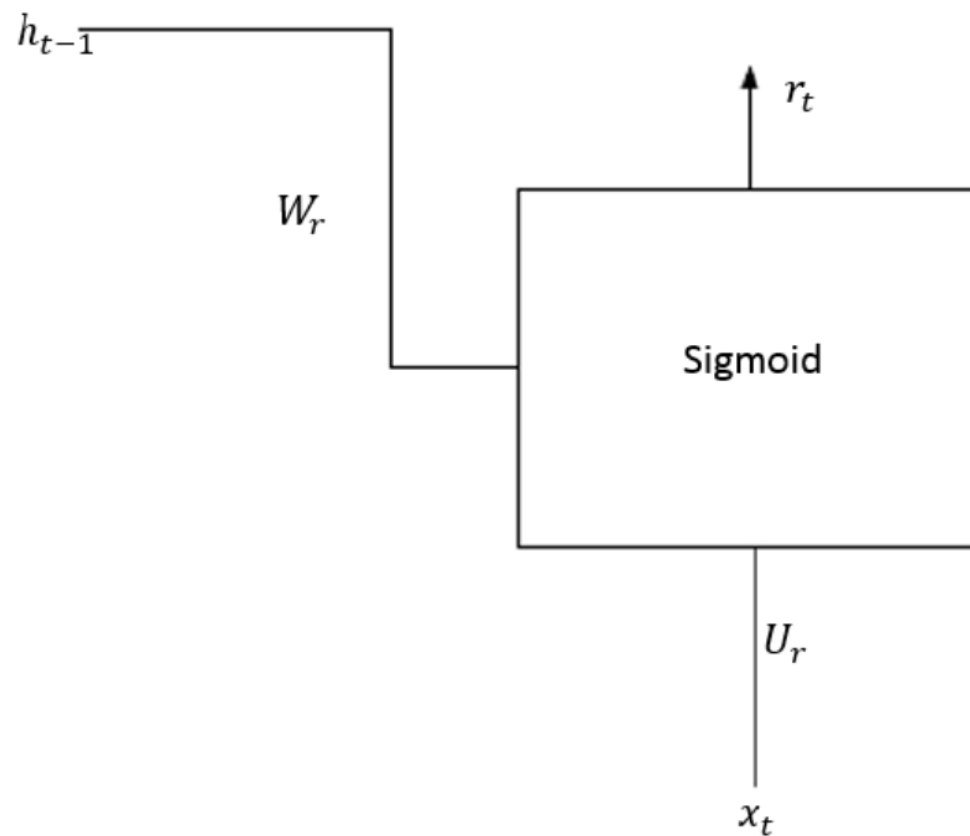
- The reset gate helps to decide how to add the new information to the
- memory, that is, how much of the past information it can forget

$$r_t = \sigma(U_r x_t + W_r h_{t-1} + b_r)$$

Here, the following applies:

- U_r is the input-to-hidden weights of the reset gate
- W_r is the hidden-to-hidden weights of the reset gate
- b_r is the bias of the reset gate

Reset gate



$$r_t = \sigma(U_r x_t + W_r h_{t-1} + b_r)$$

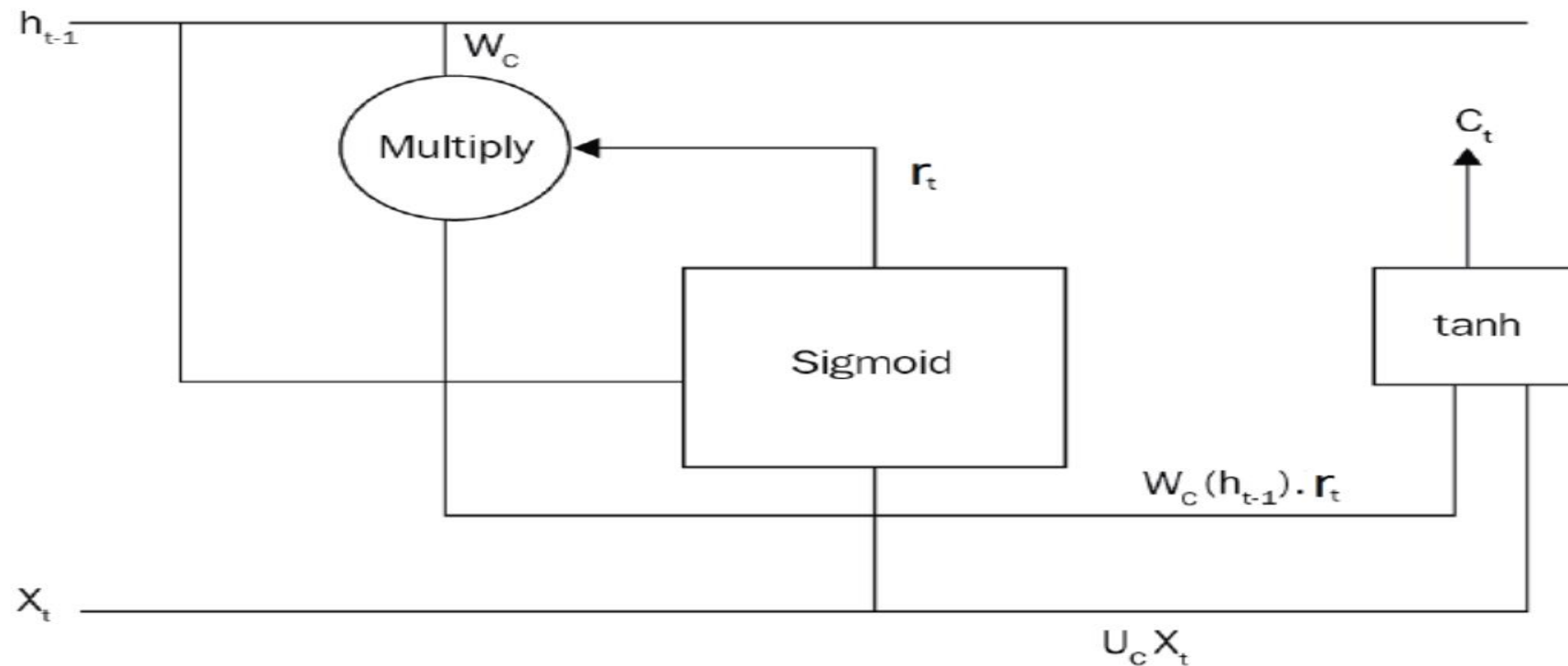
Updating hidden state

- First, we will see how to add new information to the hidden state.
- We create a new state called the **content state**, c_t , for holding the
- information. We know that the reset gate is used to remove information that is not required.
- So, using the reset gate, we create a content state, c_t , that holds only the required information.

$$c_t = \tanh(U_c x_t + W_c(h_{t-1}) \cdot r_t)$$

Updating hidden state

$$c_t = \tanh(U_c x_t + W_c(h_{t-1}) \cdot r_t)$$



Updating hidden state

- Now we will see how to remove information from the hidden state.

The hidden state is then updated as follows:

$$h_t = (1 - z_t) \cdot c_t + z_t \cdot h_{t-1}$$

Once the hidden state is computed, we can apply the softmax function and compute the output as follows:

$$\hat{y}_t = \text{Softmax}(Vh_t)$$

Forward propagation in a GRU cell

- **Update Gate:**

$$z_t = \sigma(U_z x_t + W_z h_{t-1})$$

- **Reset Gate:**

$$r_t = \sigma(U_r x_t + W_r h_{t-1})$$

- **Content State:**

$$c_t = \tanh(U_c x_t + W_c(h_{t-1}) \cdot r_t)$$

- **Hidden State:**

$$h_t = (1 - z_t) \cdot c_t + z_t \cdot h_{t-1}$$

- **Output:**

$$y_t = \text{softmax}(V h_t)$$

References

- 1. Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning (2016) An MIT Press book, <http://www.deeplearningbook.org>.(T2)
- ,MOHAMED ELGENDY, Deep Learning for Vision Systems
- Skansi S., Introduction to Deep Learning - From Logical Calculus to Artificial Intelligence, 1st Edition, Springer International Publishing, 2018.(T2)
- Buduma N., Fundamentals of Deep Learning, 1st Edition, O Reilly Media, 2016.(R1)
- Sudharshan Ravichandiran Hands-On Deep Learning algorithm with Python
- <http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Teaching/pdf/Lecture9.pdf>